
Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim



Inhoudsopgave

Introductie	1
Changelog:	1
Het voorbije decennium	2
Voor 2010	2
2010: En dan verscheen Stuxnet	3
2013 : Onze privacy te grabbel gegooid	5
2014: hoe veilig is <i>the cloud</i> ?	6
2015: Stuxnet bewaarheid	9
2015: Scheidingen en zelfmoord	9
2016: Internet-of-horrors	11
2017: Ransomware wordt gemeengoed	12
En nu: de geopolitieke macht van sociale media en grote datalekken	12
En de toekomst?	13
Het is erger, maar...	14
GDPR	15
Wat omvat GDPR	15
Wat beschermd GDPR	16
Meldplicht datalekken	16
Uitzonderingen meldplicht individu	17
Boetes	18
Cryptografie	21
Waarom cryptografie?	21
CIA en het security model	21
Encryptie	22
Kerckhoffs principe	23
De eerste algoritmes	24
Substitutie: Caesar encryptie	25
Transpositie: scytale encryptie	26
Combinatie	27

Cryptanalyse	28
Sleutellengtes en brute forcen	29
Symmetrische encryptie	30
Sleuteloverdracht	31
Block en Stream cipers	32
Streamciphers	32
Blockciphers	38
Asymmetrische encryptie	48
Het probleem met symmetrische encryptie	48
Publieke cryptografie	49
Dualiteit van publieke cryptografie	50
Diffie-Hellman sleutel uitwisseling	51
RSA	52
Boodschappen ondertekenen	54
Digitale certificaten	57
HTTPS en TLS	62
Wifi security	63
De problemen van Wifi	63
Onbeveiligde managementframes	66
De 802.11 standaard qua beveiliging	68
Verbinden met een netwerk	68
WEP	71
Hoe WEP faalde	75
Probleem 4: Sleutelmanagement	84
Hoe WEP oplossen?	84
WPA1	86
802.1X	86
TKIP	89
WPA1: alle blokjes samen	91
WPA2	92
Encryptie en MIC creatie	93
Helaas, aan alles komt een einde	94
WPA 3 (“Wifi 6”)	94
Authenticatie	97
Veilige paswoorden	97

Hoe paswoorden opslaan	98
Paswoorden als plaintext bewaren	98
Paswoord hashing	99
Rainbow table attack	99
Salting	102
CRAM en SCRAM	103
Multifactor authentication	104
Iets wat je weet: paswoorden	105
Iets wat je bent: biometrics	106
Iets wat je hebt: hardware	107
Federation en single-sign on (SSO)	107
Dankwoord	109
Bronnen	111

Introductie

Nu dat Zie Scherp en Zie Scherper ingeblikt zijn, wordt het tijd voor een nieuw project. De teksten in deze PDF zijn pre-alpha. Dit wil concreet zeggen dat:

- De teksten nog niet zijn nagelezen qua inhoud én spelling.
- De teksten nog niet compleet zijn.
- Veel afbeeldingen nog niet gecleard zijn om gebruikt te worden qua rechten.

Alle feedback is in deze fase zéér welkom:

- Schrijffouten.
- Zinnen die onduidelijk zijn.
- Paragrafen die je niet begrijpt.
- Zaken die ZEKER niet meer aangepast mogen worden omdat ze geniaal zijn.

Changelog:

Dit is versie 0.1.1 (15/5/21) * 0.1.1 * Hoofdstuk Authenticatie klaar. * 0.0.9 * Hoofdstuk Wifi volledig vertaald * Eerste helft van hoofdstuk "Authenticatie" beschreven * 0.0.8 * Uitleg en tekeningen certificaten toegevoegd. Hiermee kan hoofdstuk crypto (voorlopig) als afgerond beschouwd worden. * 0.0.7 * Aantal tekeningen omgezet * 0.0.6 * Wifi hoofdstuk vervolledigd (deels Engelstalig) Afbeeldingen moeten nog goed gezet worden. * 0.0.5 * Hoofdstuk 1 en 2 aangevuld en op schrijffouten verbeterd. # Wordt het erger?

De laatste 10 jaar is de (cyber)security wereld erg veranderd. Ze doet dit niet omdat ze daar zin in heeft, maar wel als antwoord op wat er gebeurt in de wereld in zake cyberaanvallen, geopolitieke situaties, etc.

Het is altijd een kat en muisspel, waarbij de boswachters helaas by definition steeds zullen achterlopen op de stropers. De kwaadwillige hackers hoeven maar 1 klein gaatje te vinden in je peperdure beveiliging en ze zijn binnen. Terwijl jij wel aan alles moet (proberen te) denken. Dat het erger wordt is eigenlijk daarom bijna automatisch een evidentie. De wereld van de beveiliging gebeurt per opbod en

hoe beter de boswachters het bos kunnen verdedigen, hoe complexer de technieken zullen worden die de stropers hanteren.

In de volgende secties gaan we een klein historisch overzicht geven van enkele belangrijke, interessante of spectaculaire gebeurtenissen die hebben plaatsgevonden in de cyberwereld de voorbije jaren. Dit laat ons toe om enerzijds enkele begrippen te duiden, anders om de vraag “wordt het erger?” te beantwoorden.



We gebruiken de term cyber om alles aan te duiden dat zich afspeelt op de digitale snelweg, namelijk het internet, de cloud, het www, en alle synoniemen en aanverwanten.

Het voorbije decennium

We gaan geregeld terug in de tijd gaan om te bekijken hoe bepaalde cyberverdedigings- en aanvalstechnieken zijn ontstaan, maar in dit hoofdstuk gaan we enkel terug tot 2010. Het jaar waarin Mack Zuckerberg, CEO van Facebook, door Time Magazine tot persoon van het jaar werd uitgeroepen en ook waarin Apple de eerste iPad tablet aan het publiek toonde. Het was helaas ook het jaar waarin het boorplatform, Deepwater Horizon, voor één van de ergste milieurampen ooit zorgde, alsook het jaar dat 33 mijnwerkers anderhalve maand lang 700 meter diep opgesloten zaten in een mijn. Voor ons, als toekomstige cyberboswachters, is echter de meest cruciale gebeurtenis het ontdekken van **Stuxnet**.

Voor 2010

Voor het ontdekken van Stuxnet in 2010 leek de cyberbeveiligingswereld wel een wereld van (relatieve) peis en vree. Het ergste dat kon gebeuren was dat je computer een virusje opdeed waardoor je mogelijk wat data, en vooral veel werkuren, kwijt raakte. Virussen, wormen en spam waren alomtegenwoordig maar eigenlijk niet meer dan luizen in de pels van de eindgebruikers. Tuurlijk, er werd een grondig gevloekt als een virus je foto's verwijderde of wanneer een worm zichzelf verspreidde naar je contacten - denk maar aan het *ILOVEYOU* virus dat als een soort *social engineer* mensen deed geloven dat ze een potentiële liefdesmatch hadden waarop ze vol spanning de bijlage openden en zo de worm *in huis haalden*. Of wat te denken van de *Conficker* worm die in 2008 meer dan 3 miljoen systemen kon besmetten en telkens de update-services van het Windows toestel uitschakelde. Uiteraard, dat was irritant, maar menig mens zou maar al te graag terug naar die tijd willen gaan als daarmee ransomware, botnets en door overheden gesponsorde cyberaanvallen onbestaande zouden zijn.

//TODO: jan g vragen voor beschrijving over hoe het op bedrijven was



De termen worm, virus en malware zal geregeld door elkaar worden gebruikt in deze cursus. Alle drie betekenen net niet hetzelfde, maar toch:

- Een **virus** is een kwaadaardig programma dat, eens het op een computer of apparaat staat, digitale schade kan aanbrengen.
- Een **worm** daarentegen is ook een virus, maar eentje dat zichzelf kan *voortplanten* naar andere systemen, iets wat een virus niet kan. Een worm kan zichzelf dus verspreiden zonder menselijke hulp.
- **Malware** is een overkoepelende term voor alle programma's en code die digitale schade aan een systeem of netwerk brengen. Virussen en wormen behoren dus tot deze groep.

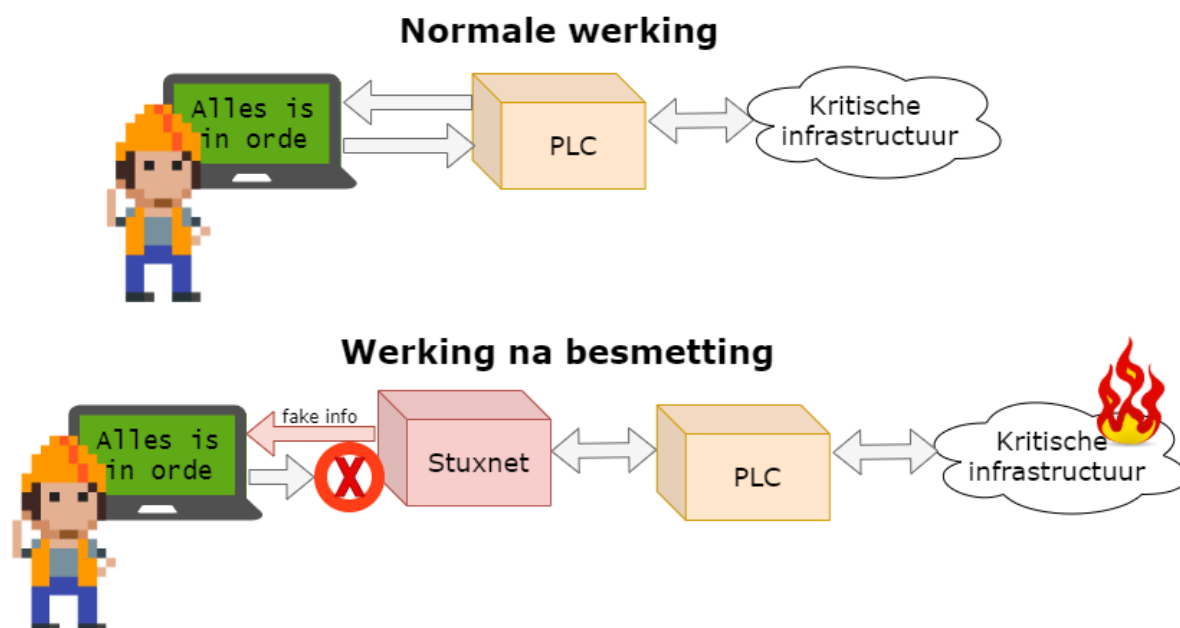
Naast deze 3 termen zullen we ook nog enkele andere malware types tegenkomen zoals Adware, spyware, spam, trojans, backdoors en rootkit. Wanneer nodig zullen we deze termen toelichten. Onthoud nu alvast dat malware de algemene naam is voor alle malafide software.

2010: En dan verscheen Stuxnet

Virussen konden computers (tijdelijk) onbruikbaar maken. OK, lastig, maar niet het einde van de wereld. De Stuxnet worm die in 2010 plots op de radar verscheen kon dat potentieel wel, de wereld eindigen. Het was een worm die op maat was gemaakt om zogenaamde *Programmable Logic Computers (PLC)* te controleren. Héél veel van onze kritische systemen zijn geautomatiseerd met behulp van deze PLCs, krachtige apparaten die machines kunnen aansturen zoals liften, robotarmen aan assemblagelijnen, zuiveringsinstallaties en zelfs kernreactors. PLCs vormen de interface tussen machines (hardware) en de computers (met software op) die de operatoren gebruiken om deze machines opdrachten te geven. Operators kunnen op hun systemen de machines bedienen en ten allen tijde controleren of deze naar behoren werken.

Wat Stuxnet deed was zich tussen de hardware en de software nestelen. Als een virus besmette het laptops en computers en zocht het of er PLC-bedieningssoftware aanwezig was op het toestel. Als dat het geval was dan plaatste het zichzelf er tussen in zodat het:

1. de machines opdrachten kon geven zonder dat de operator dit wist.
2. aan de operator kon vertellen dat alles in orde was, terwijl in realiteit de PLC mogelijk desastreuze opdrachten aan de hardware gaf.



Figuur 0.1: Werking stuxnet

We moeten er geen tekeningetje bij maken wat de effecten kunnen zijn indien een Stuxnet variant bewust werd ingezet om bijvoorbeeld de machinerie in een kerncentrale of waterdam te saboteren. Zonder in details van Stuxnet in te gaan is het duidelijk dat het verschijnen van dit virus een ommekeer betekende in wat de gevolgen van cyberterrorisme konden betekenen: mensenlevens stonden plots op het spel, niet enkel onze dierbare e-mails en digitale vakantiefoto's.



Stuxnet bleek een bewust ontworpen virus te zijn dat probeerde een specifiek soort centrifuge te saboteren. Namelijk de centrifuges die Iran gebruikte om uranium te verrijken. De inlichtingendiensten van de Verenigde Staten en Israël zouden klaarblijkelijk Stuxnet hebben ontworpen om zo dit verrijkkingsproces van uranium van Iran te dwarsbomen.



Wens je meer te weten over stuxnet? Dan raden we je de uitstekende, in 2006 verschenen, documentaire “Zero Days” door Alex Gibney aan. Deze documentaire geeft een zeer ontluisterend én boeiend beeld over de zoektocht naar de oorsprong van het Stuxnet virus.

2013 : Onze privacy te grabbel gegooid

Surfen op het internet was altijd een risico. We wisten in 2013 al dat onze datapakketjes over tientallen apparaten doorheen het internet wordt getransporteerd om zo tot bij hun doel te geraken. Cookies volgden al geregeld onze stappen en ook de aanbevelingen van Amazon waren soms akelig accuraat. Dat was de prijs die we moesten betalen om de ontelbare bronnen van het internet te kunnen gebruiken. En als je echt wat meer privacy nodig had, dan schakelde je de “incognito” modus in van je browser. Maar ook dan was je je ervan bewust dat zelfs je ISP (*internet service provider*) en je doel konden meekijken. Hoe erg kan dat zijn?

Die gedachte leefde bij veel mensen tot dan: het Internet was een nuttige tool en je wist dat er kon meegekeken worden door *derden* als ze dat echt wilden. Maar zou dit nu echt consequent en op grote schaal gebeuren? Neen toch?!

Dat glazen huisje werd in 2013 hardhandig aan diggelen geslagen door twee belangrijke personen:

- **Edward Snowden:** als (contractueel ingehuurd) systeembeheerder bij de NSA, de Amerikaanse inlichtingendienst gericht op elektronische spionage, had hij toegang tot het doen en laten van de dienst. Snowden lekte een grote hoeveelheid documenten naar de bevolking die onder andere het *PRISM*-programma uit de doeken deed. Hieruit bleek dat de NSA complexe samenwerking had met enkele grote Amerikaanse internetbedrijven (o.a. Microsoft, Google, Facebook, Apple, etc.) die zogenaamde *taps* op hun systemen hebben staan zodat de NSA “kan meeluisteren” op de netwerken (dit programma heette *PRISM*).
- **Julian Assange:** Er zijn altijd klokkenluiders (*whistleblowers*) geweest die om persoonlijke overtuigingen vonden dat bepaalde informatie met het publiek moesten worden gedeeld. Zeker in dictatoriale middens kan dit levensgevaarlijk zijn. Aan het eind van de 20e eeuw kregen deze klokkenluiders echter een erg krachtig apparaat om hun nieuw te verkondigen: het Internet. Julian Assange beseftte dit en richtte daarom in 2006 Wikileaks op. Een site waar klokkenluiders op een anonieme manier documenten konden *leken*. Tot 2010 zijn zo enkele erg controversiële documenten met de wereld gedeeld die soms verregaande diplomatieke of geopolitieke gevolgen hadden.



Het is niet evident om over mensen als Assange en Snowden te praten zonder in een controversiële modderpoel te geraken. Voor de één is Snowden een held, voor de ander een verrader. In dit boek trachten we een objectief beeld te geven, zonder waardeoordelen te geven. Wat wel buiten kijf staat is dat zowel Snowden als Assange de wereld getoond hebben dat er in de duistere wandelgangen van de veiligheidsdiensten zaken gebeuren waar “normale stervelingen” zelden weet van hebben.

Het waren uiteraard vooral de onthullingen van Snowden die bij velen de oogkleppen deed afvallen. Op het Internet ben je hoegenaamd niét anoniem. Grote (en kleine) mogendheden en privéfirma's kunnen ons doen en laten op het internet bekijken, bewaren en analyseren. Privacy en het internet zijn een *oxymoron*: een contradictorische term zoals zwarte sneeuw. Wanneer je je op het internet begeeft, op welke manier ook, gooi je een deel van je privacy te grabbel. En dat is iets dat helaas de voorbije 10 jaar er niet op verbeterd is (alhoewel het **Tor** netwerk met z'n *onion routing* toch wel een stevige extra privacy laag kan aanbieden).



Een interessant debat dat altijd opduikt bij deze problematiek is de “Ik heb toch niets te verbergen”-houding. In het kleine maar fijne boekje “Je hebt wél iets te verbergen” van onderzoeksjournalisten Maurits Martijn en Dimitri Tokmetzis (ISBN 9789082821611) wordt onherroepelijk brandhout gemaakt met deze stelling. Finaal zijn we volledig afhankelijk van de diensten die we gebruiken wat ze met onze data nu én belangrijker, in de toekomst zullen doen. Privé-informatie over jou die nu ogenschijnlijk ongevaarlijk lijkt, kan dat potentieel in de toekomst wel zijn wanneer normen, waarden of wetten veranderen.



Alhoewel het **HTTPS** al sinds 1995 bestond, werd het in 2013 amper door websites aangeboden. Nochtans geeft https een extra defensielaag tijdens de communicatie van jouw computer met die waar een website op *gehost* staat. Htps zal namelijk je communicatie versleutelen (zie hoofdstuk crypto) zodat enkel zender en ontvanger kunnen lezen wat er gezegd wordt. Met http is dat niet: al je communicatie kan door eender wie gelezen worden die zich tussen jouw computer en je eindbestemming nestelt. Pas in 2017 boden meer dan de helft van de website wereldwijd https aan. In 2021 gebruikt ongeveer 70% van alle website https als standaard communicatiemiddel aan (vroeger waren er al websites met https, maar http was de standaard oplossing).

[//https://transparencyreport.google.com/https/overview](https://transparencyreport.google.com/https/overview)

2014: hoe veilig is *the cloud* ?

In 2014 vond er een controversale plaats die vooral de puberende tiener zich zal herinneren: “*The fapping*”. Deze naam laat weinig aan de verbeelding over en helaas de lading goed. In het jaar dat Oekraïne Russische legers zag binnenrollen om de Krim (terug) in te palmen, verschenen er duizenden privéfoto's van een honderdtal *celebrities* op het internet. Deze privéfoto's hadden kwaadwillige hackers van de Apple iCloud accounts van de slachtoffers gestolen en vervolgens gepubliceerd via Reddit, 4chan, etc. De foto's verspreidden zich als een lopend vuurtje en de slachtoffers konden enkel

toezien hoe hun in de privésfeer opgenomen beelden door miljoenen mensen werden gedownload.

De beroemdheden hadden hun foto's in de cloud-opslag van Apple bewaard zoals op dat moment duizenden gebruikers ook al deden. Deze dienst zorgt ervoor dat je je als eindgebruiker van eender waar aan je foto's kan, daar ze "in the cloud" staan, wat door de spectaculaire groei van de smartphones (en iPhones in dit geval) een populair concept was geworden. Ook diensten zoals Dropbox, Onedrive (toen nog SkyDrive) toonden aan dat er een grote vraag was naar online opslag van informatie.

Om dergelijke diensten te gebruiken dien je natuurlijk een veilig paswoord te hebben, daar eender wie, van eender waar, kan proberen zich een weg naar je data te verkrijgen door jouw paswoord te raden. In 2014 waren concepten zoals 2-factor-authentication (2FA) en biometrische beveiliging (meer daarover later) nog niet zo populair en dus was je geheime paswoord je enige bescherming tegen onrechtmatige toegang tot je data.

De diefstal van de foto's werd echter vergemakkelijkt om 2 redenen:

- Sommige slachtoffers gebruikten makkelijk te raden, of snel te bruteforcen paswoorden.
- Andere hadden de beveiligingsvragen ter goeder trouw ingediend. Veel diensten stellen een extra beveiligingsvraag (zoals "Wat is de naam van je eerste huisdier" of "Op welke school zat je vader") die ze kunnen gebruiken om jouw identiteit te verifiëren indien je je paswoord vergeten bent en je dit wenst te resetten. Wanneer jij of ik dit soort vragen invullen dan is dit 90% van de tijd informatie die nergens te vinden valt, enkel in de grijze massa in je hoofd en misschien in een gênant dagboekje uit je jeugd. Bij de Amerikaanse beroemdheden wiens iCloud werd gehackt, is dat niet zo. Hun leven kan volledig gereconstrueerd worden aan de hand van de ontelbare interviews die ze al hebben gegeven. Gegarandeerd dat ooit een interviewer al heeft gevraagd wat de kleur van zijn of haar eerste auto was, of in welk dorp hij of zij is opgegroeid.



Het is een verkeerde reflex om in dit soort zaken ogenblikkelijk aan *victim shaming* te doen (kijk maar naar het recentere voorval waarbij enkele Bekende Vlamingen het slachtoffer waren van catfishing). Ieder doet en laat wat hij wenst in z'n privésfeer - daarom heet het ook privé- maar het is belangrijk te beseffen dat als je zaken *in the cloud* bewaard, de kans bestaande is dat iets of iemand er ooit onrechtmatige toegang tot zal krijgen. U weze gewaarschuwd.



Niemand verplicht je om op de beveiligingsvragen een eerlijk antwoord te geven. Er bestaat geen leugendetector in die systemen of een mama die op je vingers komt tikken. Lieg er dus op los, maar onthoudt uiteraard je antwoord. Dankzij die beveiligingsvragen hou je echter een stok achter de hand moest je je paswoord vergeten zijn.

Ook in 2014: als landen vechten

Het was te verwachten dat ook op cyberniveau er ooit een cyber-wapenwedloop zou starten zoals, helaas, ook in de echte wereld plaatsvindt. Daar waar landen voorheen nog vooral defensieve cyber-mogelijkheden hadden, begon halverwege het vorige decennium dit arsenaal meer en meer uitgebreid te worden met offensieve cyberwapens. Het was met andere woorden wachten tot de eerste cyberaanvallen door soevereine staten op andere staten.

Het probleem met cyberaanvallen is dat het als aangevallen mogendheid ongelooflijk moeilijk is om

1. De bron van de aanval te identificeren. Wie weet lijkt de aanval te komen vanuit land X, terwijl het eigenlijk land Y is dat gewoon de aanval via land X routeert. En het laatste dat je natuurlijk wil doen is het verkeerde land beschuldigen.
2. En wat als een hacker, zonder staatsinmengingen, beslist om op eigen houtje een cyberaanval te initiëren. Is het land van herkomst van die aanvaller dan verantwoordelijk voor de schade?
3. De “schade” van een cyberaanval is niet altijd duidelijk. Wanneer een raket op een stad wordt afgestuurd is dat een duidelijk *act of aggression* en is er grote kans op een militair conflict (indien de diplomatieke weg geen soelaas brengt). Maar is een cyberaanval, zoals een denial-of-service (DoS, zie later), genoeg reden om de oorlog buiten het cyberdomein te escaleren? Zijn er eigenlijk *rules of engagement*? Is er een conventie van Genève? Neen op dit alles. Daar cyberaanvallen zo moeilijk te traceren en identificeren zijn, blijft het allemaal het betere nattevingerwerk werk.

Een bewuste cyberaanval op een soevereine staat was al een enkele keer gebeurt (denk maar aan de cyberaanvallen in 2007 van Rusland op Estland) maar in 2014 was er helaas een nieuwe primeur. Ook nu moeten we naar Hollywood keren, niet om de beroemdheden en hun privé-kiekjes, maar omwille van een film waar een andere staat niet om kon lachen.

In 2014 zou Sony een nieuwe film uitbrengen, getiteld “The Interview”, met James Franco en Seth Rogan. In deze komedie worden 2 journalisten gevraagd om tijdens hun interview van een *fictieve* Noord-Koreaanse dictator hem te doden. De vergelijkingen met de Noord-Koreaanse leider Kim Jong-un waren voor iedereen duidelijk. Dat Noord-Korea op de tenen zou zijn getrapt stond dan ook in de sterren geschreven.

De daaropvolgende gebeurtenissen zijn nog steeds niet uitgeklaard, maar vast staat wel dat een selecte groep hackers toegang had gekregen tot confidentiële data op de servers van Sony en deze vervolgens op het internet lekte. De financiële schade voor Sony was enorm. De daders wisten onuitgebrachte films, e-mails, salarissen en andere privé-informatie te stelen. Heel lang werd gedacht dat de groeperingen die achter de aanval stond door Noord-Korea was aangestuurd. Echter, nu nog steeds heeft men dat niet kunnen bevestigen. Zou zijn er ook experts die denken in het bewijsmateriaal de hand van Rusland en/of China te zien. Kortom, zoals eerder gezegd, cyberaanvallen zijn verdomd lastig om in kaart te brengen.

Als het toch Noord-Korea zou zijn geweest - wat nog steeds meer dan mogelijk is- dan hebben ze hiermee dus een dubieuze primeur: een soeverein land voert een cyberaanval uit op een bedrijf in een ander land. En de vraag die vervolgens veel experts zich stelden was: “vanaf wanneer is dit een *act of aggression*?”, en ook: “moet een land op dit soort aanval reageren namens het bedrijf, of namens de hele natie waar het bedrijf zich bevindt?” Wie zal het zeggen. Dit boekje helaas niet, maar het geeft wel voer voor discussie.

2015: Stuxnet bewaarheid

Keer op keer zullen we dit moeten herhalen: we kunnen nooit met 100% zekerheid de origine van een cyberaanval vaststellen. Heel af en toe, met dank aan klokkenluiders zoals Snowden, of verregaand (al dan niet journalistiek) onderzoek worden specifiek aanvallen volledig uit de doeken gedaan (maar helaas vaak jaren na datum).

In 2015, iets meer dan een jaar na de Sony aanval, werd dat waar we voor vreesden bewaarheid: hackers waren er in geslaagd om de elektriciteitsinfrastructuur van Oekraïne deels plat te leggen op 23 december, vlak voor kerstavond. Een groepering slaagde er zo in om bijna een kwart miljoen mensen gedurende meerdere uren zonder stroom te zetten. Iedereen keek uiteraard ogenblikkelijk in de richting van Rusland - Oekraïne maakte vroeger deel uit van de voormalige Sovjetrepubliek- daar de aanvallen kwamen van IP-adressen die waren toegewezen aan Rusland. Maar zelfs als dat zou zijn, zonder harde bewijzen dat de hackers ook handelden in naam van de Russische overheid blijft het koffiedik kijken wie *op de vingers getikt* moet worden voor de aanval.

Wie het ook waren, één ding staat wel vast: in 2015 waren hackers er voor het eerst in geslaagd om met één welgemikte aanval honderdduizenden mensen in problemen te brengen die verder gingen dan het verwijderen van enkele bestanden.

Enkele jaren later, in februari 2021, zou het trouwens weer bijna prijs zijn. Een hacker kreeg toegang tot de systemen die de waterzuiveringsinstallaties van Oldsmar (Florida, VS) bediende. Een operator zag op tijd dat z'n muis plots bewoog en de maximum toegelaten hoeveelheid natriumhydroxide waarde van de filters op een dodelijk niveau zette. Er werd gelukkig tijdig ingegrepen (en detectoren verderop in het systeem hadden het vergiftigde water sowieso gedetecteerd), maar het deed wederom de vraag rijzen of onze kritische systemen wel voldoende beveiligd zijn tegen dit soort *lone wolves*.

2015: Scheidingen en zelfmoord

In 2015 wordt de schaal van cyberaanvallen steeds groter. De hoeveelheden informatie die hackers van bedrijven kunnen bemachtigen kunnen al lang niet meer op een A4'tje afgedrukt worden. Wanneer hackers toegang krijgen tot de privé-servers van hun doelwit kunnen ze vlotjes ettelijke gigabytes, tot

zelfs terabytes, aan privé informatie stelen. Tegenwoordig hebben we in Europa de GDPR wetgeving die probeert bedrijven duidelijk te maken (en te straffen) dat zij verantwoordelijk zijn om onze data op een veilige manier te bewaren (zie volgende hoofdstuk). In 2015 was dat veel minder. De berichten van datalekken in die tijd spraken boekdelen: van zodra cybercriminelen toegang hadden tot de privéservers konden ze de data zonder problemen lezen. Paswoorden, kredietkaartgegevens, rijksregisternummers, alles stond vaak onbeveiligd (*ongeëncrypteerd*) op de systemen.

De website Ashley Madison was Tinder voor mensen die een relatie wilden naast hun “officiële relatie”. De site hielp kortom mensen aan een affaire. Hun leuze, *“Life is short. Have an affair”*, wond er geen doekjes rond. En met hun meer dan 20 miljoen “klanten” was het duidelijk dat ze een lucratief idee hadden. Dat ze tegenwind zouden krijgen was te verwachten. Maar een wind van 12 beaufort? Dat niet.

In juli 2015 plaatste een groep hackers een ultimatum op een website aan het adres van de eigenaars van Ashley Madison: *“Haal jullie moreel dubieuze website van het internet, of wij plaatsen meer dan 60 gigabyte aan gestolen data online”*. De site werd niet offline gehaald en de hackers “hielden woord.” De gevolgen waren immens, niet zo zeer voor Ashley Madison, wel voor z’n klanten. Plotsklaps kreeg de wereld een lijst te zien waarin honderdduizenden klanten open en bloot aan de schandpaal werden genageld. Mensen werden publiekelijk vernederd. Er is sprake van minstens 2 of 3 zelfmoorden rechtstreeks als gevolg van de lek. Mensen werden ontslagen. Kortom, het lekken van wat uiteindelijk maar een hoop binaire data- nullen en eentjes- was had gevolgen voor relaties, mensenlevens en carrières.

Als positieve noot in dit verhaal halen we hier uit dat dit soort gigantische lekken mensen heeft doen inzien dat ze twee maal moeten nadenken voor ze hun persoonlijke informatie weggeven aan één of andere internet-gigant (het is helaas een les die we jaarlijkse lijken te vergeten, kijk maar naar de populariteit van Tik Tok, Facebook, etc.).



Een stelregel die bedrijven nu hanteren is de volgende: vraag je niet af **of** je gaat gehackt worden maar vraag je af **wanneer** je zal gehackt worden. Dit is een heel andere manier van tegen je beveiligingsprobleem aankijken. Vergelijk het met het in huis halen van een koffer met daarin 10 miljoen euro aan diamanten. Als je je afvraagt of er ooit inbrekers zullen binnen geraken en daar naar beveiligt -waakhonden, videocamera’s rond het huis, dubbel slot- dan heb je daar niets aan als ze vervolgens toch binnen geraken en je koffertje stelen. Veel beter kan je én je huis beveiligen, én ervan uitgaan dat ze de koffer gaan bemachtigen: en je dus maar beter ook ervoor zorgt dat de dieven niets met de diamanten kunnen doen (door bijvoorbeeld je naam er in te graveren).

Kortom: bedrijven moeten data die ze van ons opslaan minstens encrypteren en systemen inbouwen die de data onbruikbaar maken als ze toch gelekt zou worden.

2016: Internet-of-horrors

In onze huizen verschenen steeds meer apparaatjes die via het, meestal draadloze, netwerk met elkaar en het internet konden communiceren. De term Internet-of-Things (IoT) bracht een weelde aan nieuwe oplossingen in onze levens. Domotica, wearables, slimme thermostaten, beveiligingssystemen, weegschalen, alles werd én slimmer gemaakt én aan het internet gehangen.

Een inherent probleem met veel van deze kleine toestellen is dat ze op het gebied van beveiliging ondermaats presteren. Dergelijke IoT-apparaten werken vaak op batterijen en de makers willen natuurlijk de batterijduur zo lang mogelijk houden. Iedere extra feature die de designers in het apparaat willen steken heeft een kost op die levensduur. Een aspect zoals beveiliging werd dan ook vaak achteraan de lijst van potentiële features geplaatst. Komt daarbij dat IoT-apparaten updaten (met bijvoorbeeld nieuwe security patches) soms onmogelijk of tenminste omslachtig is. Als er dus een beveiligingslek wordt gevonden in een apparaat dan is de kans bestaande dat dit lek voor altijd aanwezig zal blijven én dus ook kan misbruikt worden.

Het was dus wachten tot de eerste aanvallen, specifiek gericht op IoT apparaten, zouden plaatsvinden. De meest opvallende aanval gebeurde eind 2016. Malware, genaamd Mirai, verspreide zich als een lopend vuurtje over IoT-apparaten waarvan de malware de standaard paswoord en username kende. Gebruikers die vergeten waren het paswoord aan te passen dat het apparaat heeft wanneer je het uit de doos haalde zaten zo plotseling met een ogenschijnlijk perfect werkend apparaat. Echter, de malware nestelde zich onzichtbaar op het apparaat en wachtte op commando's van de Mirai makers. De malware creëerde met andere woorden een zogenaamde *botnet*, een groot netwerk van besmette apparaten die allemaal commando's kunnen uitvoeren van de *botnet herder* (i.e. degene die de malware in de eerste plaats is beginnen verspreiden). De Mirai-makers hadden zo een leger minicomputers onder hun bevel die ze konden zeggen "surf nu allemaal naar die website". Een website die plots tienduizenden gebruikers onverwacht extra te verwerken krijgt zal vaak onder de druk bezwijken en crashen. Kortom, dit Mirai botnet kon zo grote distributed denial-of-service (DDOS) aanvallen uitvoeren, allemaal omdat gebruikers de paswoorden van hun gloednieuwe apparaatjes niet hadden aangepast. In hun verdediging, het is vaak een erg omslachtig, technisch, proces om het paswoord van een IoT-apparaat aan te passen.



De wildgroei van Internet-Of-Things apparaten heeft ervoor gezorgd dat hackers een grote hoeveelheid extra mogelijkheden hebben bijgekregen om op huis en bedrijfsnetwerken te infiltreren.



Een dubieuze (deels betalende) site, **shodan.io**, heeft als enige doel alle internet-of-things apparaten in kaart te brengen die zichtbaar zijn vanop het internet. Deze “Google voor IoT” toont zelfs om wat voor apparaten het gaat en welke bijvoorbeeld nog steeds het standaard (default) paswoord hebben.

2017: Ransomware wordt gemeengoed

De virussen in de vorige eeuw durfden al eens je data te verwijderen. Lastig, maar erg duidelijk: je data was je kwijt, tenzij je ergens een backup had liggen. De nieuwe virussen gingen echter een stapje verder: ze versleutelden al je data (vaak ook je backups als je zo dom was geweest deze op het zelfde apparaat te hebben) én vroegen vervolgens losgeld in ruil voor je data. De naam *ransomware* kon, helaas, niet beter gekozen zijn. Menig particulier betaalde ogenblikkelijk om de eenvoudige reden dat de ransomware de gebruiker nog een uitweg aanbood daar ransomware vaak werd “binnengehaald” door een gênante actie (bv illegale software downloaden, op reclame voor vage pornosites klikken, etc). Voor particulieren was ransomware vooralsnog irritant. Maar wat als de ransomware bedrijf kritische data begon te encrypteren? Dit is exact wat er gebeurde in 2017 met de WannaCry en Petya ransomwares. Deze malwares sloegen er in om banken, hospitalen, havenbedrijven en menig ander groot (en klein) bedrijf te besmetten. De economische schade werd geschat op bijna 4 miljard dollar vanwege het lamleggen van de productie (of in het geval van enkele ziekenhuizen: het redden van mensenlevens).

De schade én de losgeldbedragen worden ook steeds groter. Zo was er het voorval met Garmin in de zomer van 2020 dat niet alleen de populaire sport-tracking diensten gedurende meerdere dagen uit de lucht haalde, maar er ook voor zorgde dat menig vliegtuig niet mocht vliegen omdat de Garmin Pilot apps niet werkten waardoor de piloten geen up-to-date aeronautische plannen voorhanden hadden. Het is nooit geweten of Garmin het losgeld (10 miljoen dollar!) wel of niet heeft betaald, vast staat wel dat ransomware tegenwoordig een, helaas, erg lucratieve handel is geworden voor cybercriminelen.

En nu: de geopolitieke macht van sociale media en grote datalekken

2016 ging er een schokgolf doorheen de wereld. Tegen alle verwachtingen in won Donald J. Trump de presidentsverkiezingen na een bitsige strijd tegen Hillary Clinton. Dat social media een belangrijke rol zouden spelen dit decennium was al lang voorspeld. Facebook, Twitter, Google en konsoorten hadden miljarden gebruikers die met plezier hun privacy te grabbel gooiden in ruil voor dagelijkse dopamine-shots dankzij *likes* en *retweets*. Met dank aan gigantische *troll farms* (organisaties die duizenden fake social media accounts aanmaken en zo mee de social media algoritmes beïnvloeden om bepaalde

informatie te *nudgen* in de gewenste politieke richting) kon Trump honderdduizenden potentiële twijfelaars doen inzien dat hij het juiste antwoord was tijdens de verkiezingen.

De inmenging van Rusland in een buitenlandse verkiezing (een daad waar menig land zich reeds schuldig aan heeft gemaakt) was ontluisterend vanwege de schaal waarop het gebeurde. Het toonde de almacht van de grote Silicon Valley bedrijven en hoe zij op geopolitiek niveau een belangrijke speler zijn geworden. Iets dat vervolgens nogmaals bevestigd werd door het simultaan plaatsgevonden Cambridge Analytica schandaal, dat niet alleen mede ervoor gezorgd heeft dat Trump president werd, maar dat hoogstwaarschijnlijk ook een grote groep twijfelaars heeft kunnen overhalen om een pro-Brexit stem uit te brengen.

Drie jaar later zagen onderzoekers een soortgelijk fenomeen tijdens de verkiezingen van de Europese Unie in 2019. Een rapport toonde aan dat Rusland actieve misinformatie campagnes organiseerde om zo de verkiezingen te beïnvloeden. Ze gebruiken hierbij zogenaamde *bad actors*, fake social media accounts die (al dan niet fake) nieuws verspreiden dat “in de winkel van Rusland past”. Uit het onderzoek bleek dat de toenmalige belangrijkste EU-mandatarissen (Juncker, King, Tajani, etc.) soms tot 20% volgers op Twitter hadden die eigenlijk *bad actors* waren.

De voorbije jaren is er een (lichte) kentering bezig inzake de macht van de social media bedrijven, maar het blijft een feit dat momenteel wij alleen grotendeels afhankelijk zijn van door artificiële intelligentie aangedreven algoritmes die bepalen welke informatie wij willen/zouden/moeten consumeren, ieder uur van de dag. Zolang echter data het nieuwe goud is en bedrijven dit vrij kunnen bewaren (GDPR poogt dit in te perken) zullen zij hun algoritmes kunnen blijven voeden en trainen om nog griezeliger/knapper te maken.

Het gevolg van die *datahoarding* is echter ook dat datalekken ook steeds nefastere gevolgen hebben. Daar waar het bij Ashley Madison nog ging om een dikke 20 miljoen user accounts, medio 2021 zijn het aantal accounts dat bij een datalek betrokken zijn soms vertienvoudigd - de MGM Grand Hotels keten zag in de zomer van 2020 plots 142 miljoen van z’n gast-accounts te koop staan voor een schamele 3000 dollar in bitcoin.

En de toekomst?

Een glazen bol hebben we niet, maar vast staat dat het er niet op zal verbeteren. Zoals gezegd gaan bedrijven uit van *when* in plaats van *if* als het gaat over de vraag of ze al dan niet ooit gehackt zullen worden. Daarnaast zien we dat hoogtechnologische producten meer en meer ingeburgerd geraken in het cybercrimemilieu. Deep fakes video van bekenden zijn nu nog “grappig”, maar de realistische beelden (afbeeldingen, video én nu zelfs ook spraak) die ze kunnen produceren zijn al even niet meer te onderscheiden van het echte en zullen dus meer en meer kunnen gebruikt worden voor sextortion en soortgelijke schandalen. Dit soort trends zullen nog jaren *fake news* hoogtij laten vieren waardoor

ook toekomstige verkiezingen interessante doelen blijven voor andere mogendheden om hun stempel te drukken op geopolitieke tegenstanders.

Het is erger, maar...

Dus ja, het wordt helaas erger. De wapenwedloop in de cyberwereld gaat beangstigend snel vooruit en het wordt moeilijker en moeilijker om als “normale sterveling” er een antwoord op te geven. Als een IoT botnet van 10 miljoen apparaten morgen beslist om de infrastructuur van jouw KMO plat te leggen, dan zullen ze daar in slagen, ongeacht de miljoenen euro’s die je hebt geïnvesteerd in *firewalls, intrusion detection systems, virusscanners en honeypots*.

Aan de andere kant heeft de wapenwedloop er wel voor gezorgd dat onze infrastructuur ook steeds complexere aanvallen kan weerstaan. Hierdoor wordt het voor huis-tuin-en-keuken malware een pak moeilijker om nog computers van thuisgebruikers te besmetten.

Maar laten we deze cyberstropers geen vrij spel geven! Laten we leren van hun technieken om zo zelf onze systemen en personeel te *hardenen* en te beschermen tegen wat niet anders dan het “wilde westen van het Internet” (dixit komiek Steven Wright) kan genoemd worden.

GDPR

De General Data Protection Regulation (2018), of in het Nederlands de *Algemene Verordening Gegevensbescherming* is een Europese richtlijn om de privacy van Europese burgers, eender waar in de wereld, te beschermen. Of zoals de EU zelf op haar website beschrijft “the toughest privacy and security law in the world” (wat ook effectief zo is: het is de enige wet momenteel die er in slaagt om wereldwijd de regels ervan af te dwingen). De wet zelf bestaat uit 99 verschillende artikels die de rechten, personen en verplichtingen van bedrijven die onder de verordening vallen, worden uitgelegd. Ieder bedrijf, eender waar in de wereld, die data bijhoudt van EU-burgers dient zich aan deze wet te houden op risico van een boete als ze dit niet doen.



We gaan in dit hoofdstuk enkel een high-level overzicht geven van GDPR opdat we niet verzanden in de taal der rechters en advocaten, het *legalese*.

Wat omvat GDPR

Het recht om persoonlijke gegevens te wissen of het recht om vergeten te worden. GDPR laat je toe, als EU-burger om eender welke site die ‘jou kent’ te verplichten alle, of specifieke, informatie over je te verwijderen. Als je dus vindt dat Google niet moet weten waar je huisadres is, dan heb je het recht google te verplichten deze informatie te verwijderen.



Google kreeg een tijd terug een zeer stevige boete omdat het niet inging op de vraag van een EU-burger om *vergeten te worden*. Sindsdien zijn ze een pak behulpzamer in het naleven van de GDPR-wetgeving.

Een ander belangrijk aspect van GDPR is het zogenaamde **recht op overdraagbaarheid van gegevens**. Voor GDPR bestond was het soms een helse opdracht om bijvoorbeeld van internetprovider te veranderen. Iedere provider werkte met eigen dataformaten waarin de klantgegevens werden bewaard. Bij een overdracht moest dit dan altijd omgezet worden naar het formaat van de nieuwe provider. GDPR verplicht dat dit soort data overdrachten van persoonsgegevens nu volgens een vast formaat moeten gebeuren. En belangrijker: deze overdracht moet zo transparant én zo geautomatiseerd

mogelijk gebeuren. Hierdoor kan de data quasi ogenblikkelijk in de database van de ontvanger geïntegreerd worden en zal jij als klant dus veel sneller de overstap kunnen maken zonder alle bijhorende administratieve rompslomp en vertragingen.



De voorganger van de GDPR was de DPD (Data Protection Directive). Dit was geen Europese wet (verodering) maar een aanbeveling en was een stap in de goede richting maar nog niet ver genoeg. GDPR heeft ervoor gezorgd dat individuele burgers geen speelbal meer zijn van de grote bedrijven die de data van burgers als het nieuwe goud verzamelen en ermee doen wat ze zelf willen.

Wat beschermd GDPR

De GDPR wet zorgt ervoor dat bedrijven veel bewuster met gebruikersdata omgaan. Ze kunnen namelijk verantwoordelijk gehouden worden indien gebruikersdata gestolen of gelekt wordt. De GDPR is een privacy-wetgeving in de eerste plaats: het wil de privacy van het individu beschermen. Volgende data valt dan ook onder de GDPR-wetgeving:

- Persoonlijke identificeerbare informatie: namen, adressen, geboortedatum en rijksregisternummer.
- Op internet gebaseerde gegevens: zoals gebruikerslocatie, IP-adres en cookies.
- Gezondheids-en genetische gegevens.
- Biometrische gegevens (denk maar aan je irisscan of vingerafdruk).
- Raciale en/of etnische gegevens.
- Politieke meningen.
- Seksuele geaardheid.

Meldplicht datalekken

Bedrijven die onder de GDPR-wetgeving vallen (alle bedrijven die data van Europese burgers bewaren vallen hier onder, ongeacht hun locatie in de wereld) hebben een meldplicht indien zij een datalek hebben.



Onder een datalek verstaan we *“Een inbreuk op de beveiliging die per ongeluk, of op onrechtmatige wijze leidt tot de vernietiging, het verlies, de wijziging of de ongeoorloofde verstrekking van of de ongeoorloofde toegang tot doorgezonden, opgeslagen of anderszins verwerkte gegevens.”*

Van zodra het bedrijf weet heeft van een (mogelijk) datalek dienen zij dit ogenblikkelijk te melden:

- Aan de **toezichthoudende autoriteit**: Ieder land heeft z'n eigen autoriteit en in België is dat de **CBPL**, de *Commissie voor de bescherming van de persoonlijke levenssfeer*. Deze commissie zal iedere datalek *case-by-case* beoordelen om te zien of het bedrijf een GDPR-overtreding heeft begaan of niet.
- Aan **ieder individu waar de data betrekking op had**: als de gelekte data ook maar op één manier kan gelinkt worden aan een individu, dan dient deze van de lek op de hoogte gebracht te worden.
- Dit dient **binnen de 72 uur na ontdekking van het datalek te gebeuren**.

Merk op dat tegenwoordig een datalek zelden zo snel wordt ontdekt, dat beseft GDPR ook. Daarom is de regel dat 72 uur ingaan vanaf het moment dat het bedrijf weet heeft van het datalek. Het is dus perfect mogelijk dat de datalek reeds maanden eerder plaatsvond maar dat het bedrijf het nu pas ontdekt.

Uitzonderingen meldplicht individu

Bij een datalek zijn er enkele mogelijke uitzonderingen waarom het bedrijf niet noodzakelijk het individu op de hoogte moet stellen van de datalek:

- Indien het bedrijf kan aantonen dat de gestolen data zodanig beveiligd is dat deze onbruikbaar is. Als dus de data bijvoorbeeld geëncrypteerd is dan zou het bedrijf dit als argument kunnen gebruiken om de meldplicht te verzaken. Een kanttekening is hier natuurlijk op z'n plaats: er bestaat altijd nog de mogelijkheid dat de data finaal nog kan gedecrypteerd zal worden en er dus alsnog private gegevens in verkeerde handen komen.
- Stel dat het bedrijf de persoonlijke voorkeuren van de gebruikers in een database heeft staan en de identificeerbare persoonsgegevens in een andere. Als nu blijkt dat enkel de database met persoonlijke voorkeuren werd gehackt, dan kan deze data nooit gelinkt worden aan personen en is er dus ook geen sprake van een privacy schending.



Het is sowieso altijd een goede gewoonte om het adagio “*don't put all your eggs in one basket*” te hanteren. Het is wijzer om de data over verschillende databases te bewaren. Zo voorkom je dat bij een datalek automatisch steeds alle data wordt gestolen.



Indien er een erg grote hoeveelheid mensen moeten ingelicht worden na een datalek, dan mag het bedrijf ook beslissen om in de plaats daarvan een publieke aankondiging te doen. Uiteraard zijn de bedrijven hier minder happig op omdat dit sowieso een PR-nachtmerrie is (iedere aankondiging van een datalek is dat trouwens).

Boetes

Als er uiteindelijk toch een inbreuk op de GDPR-wetgeving wordt vastgesteld dan zal er dus een boete opgelegd worden afhankelijk van de ernst van de inbreuk:

- Minder ernstige inbreuken: boete tot € 10 miljoen, of 2% van de wereldwijde jaaromzet van het bedrijf uit het voorgaande boekjaar, afhankelijk van welk bedrag hoger is.
- Meer ernstige inbreuken (inbreuken die ingaan tegen de principes van het recht op privacy en het recht om te worden vergeten, die de kern vormen van GDPR): boete van maximaal € 20 miljoen, of 4% van de wereldwijde jaaromzet, afhankelijk van welk bedrag hoger is.

Of en hoe groot de boete zal zijn is gebaseerd op een aantal criteria:

- Ernst en aard van de overtreding.
- Intentie: is de datalek het gevolg van een slordigheid of opzettelijk gebrek aan beveiliging?
- Schadebeperkende omstandigheden.
- Genomen voorzorgen.
- Geschiedenis van overtredingen.
- Medewerking: in hoeverre werkt het bedrijf mee met de toezichthoudende autoriteiten?
- Gegevenscategorie
- Kennisgeving: heeft men zich aan de meldplicht gehouden?
- Certificering: heeft het bedrijf in het verleden reeds via een externe audit aangetoond aan de GDPR wetgeving te voldoen?
- Verzwarende of verzachtende factoren.



Merk dus op dat het schenden van de GDPR wetgeving niet automatisch in een boete resulteert. Als het bedrijf kan aantonen dat ze echt alles in het werk hebben gesteld om de data te bewaren zoals een goede huisvader betaamt, dan kan het zijn er geen boete zal volgen.



De voorbije jaren (mei 2018 tot eind 2020) werden reeds 272 miljoen euro aan boetes geïnd ten gevolge van GDPR overtredingen. De wet is zo effectief, dat ze ook ondertussen dient als inspiratie wereldwijd wanneer landen of groeperingen nieuwe privacy-wetgevingen willen construeren.

Cryptografie

Waarom cryptografie?

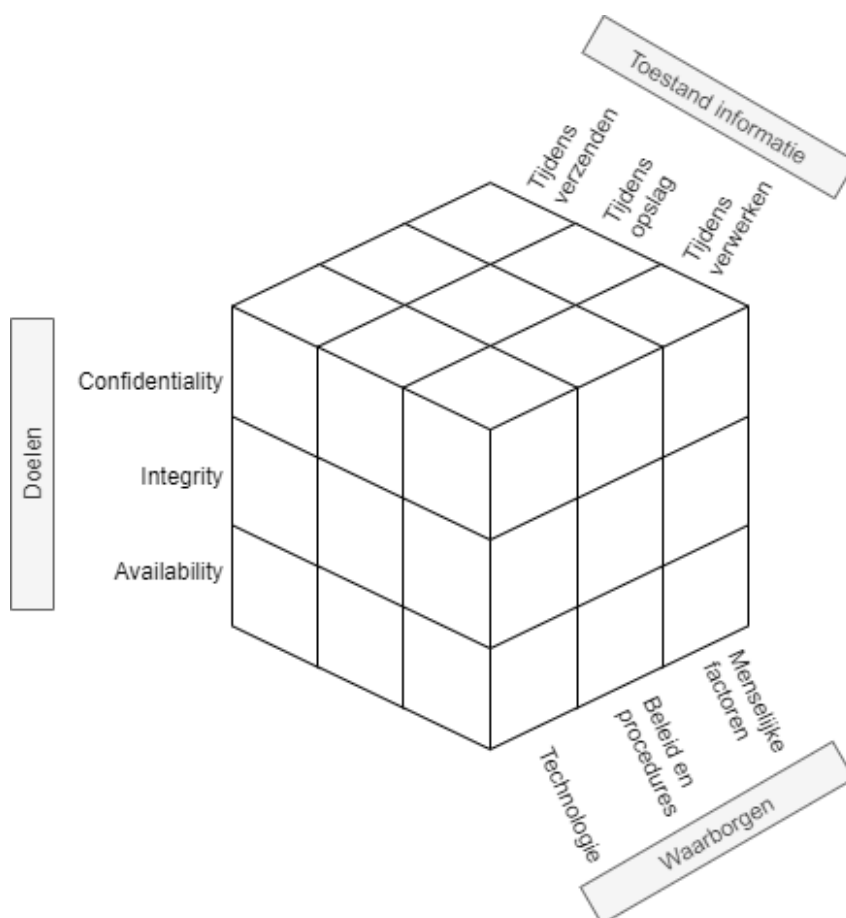
CIA en het security model

Data (of informatie), in welke vorm dan ook (berichten over een netwerk, bestanden op een harde schijf, tekst in een database), moet beschermd worden, dat beseffen we nu. Het doel van onze data is dat deze voldoet aan het acroniem **C.I.A** wat staat voor:

- **C** voor “**confidentiality**”: vertrouwelijkheid. De data kan enkel door zij die er recht toe hebben gebruikt worden. We gaan dit onder andere oplossen met behulp van encryptie en paswoorden.
- **I** voor “**integrity**”: integriteit. We moeten weten of onze data onbeschadigd is en niet werd aangepast door derden (of storingen). Bij bestanden gaan we bijvoorbeeld werken met zogenaamde (secure) hashes.
- **A** voor “**availability**”: beschikbaarheid. Data die niet door rechtmatige gebruikers kan bereikt worden is onbestaande data. Zogenaamde “denial-of-service” (dos) aanvallen hebben als doel deze pijler van CIA aan te vallen. availability is een breed veld en wordt onder andere opgelost door backups, redundante servers enerzijds, en preventieve maatregelen anderzijds zoals firewalls, load balancers, etc.

De zogenaamde McCumber kubus, ontwikkeld door John McCumber in 1991, geeft een goed beeld weer waarom het steeds belangrijk is goed te beseffen binnen welke context we praten. Deze kubus stelt een model voor dat kan gebruikt worden om je ervan te vergewissen dat je aan alles denkt wanneer je je informatie wilt beschermen. De kubus bestaat uit 3 dimensies en iedere dimensie bestaat uit een aantal aspecten. **Enkel wanneer we alle aspecten van alle dimensies in onze beveiligingsaanpak voorzien kunnen we hopen dat we onze C.I.A. doelen hebben bereikt.**

Om ons doel te bereiken (C.I.A.) moeten we ervoor zorgen dat we dit toepassen op alle vormen die onze data kan hebben (opslag, verzenden, verwerken). Dit kunnen we tewerkstellingen door technologische oplossingen (zoals encryptie wat zo meteen wordt uitgespit), maar een niet onbelangrijke factor zijn ook de mensen die met de data moeten werken. Als zij zich niet aan de afspraken (procedures) houden en hun paswoorden gewoon op post-its aan hun scherm hangen, dan mag je een nog zo’n dure firewall hebben, het zal niet baten.



Figuur 0.1: De McCumber kubus

Encryptie

Een grote pijler van CIA, confidentiality, wordt opgelost met behulp van encryptie, namelijk het versleutelen van onze data met behulp van een geheime sleutel. Door deze te versleutelen wordt deze onleesbaar voor personen die de geheime sleutel niet hebben (en bijgevolg niet geautoriseerd zijn om de data te mogen lezen). De moeilijkheid van een goed cryptografisch systeem is dat de data op een zodanig manier met versleuteld worden dat het quasi onmogelijk is om zonder sleutel de originele data terug te vinden. We spreken hierbij over de originele data als de *plaintext* en de geëncrypteerde data als *ciphertext*. De ontvanger van een ciphertext moet deze, als hij de juiste sleutel heeft, terug kunnen omzetten naar de originele plaintext.

Er zijn al veel encryptie algoritmes de revue gepasseerd doorheen de geschiedenis van de mens. Al van in de tijd van de Romeinen werd er aan cryptografie gedaan. Mensen hebben altijd gevoelige data gehad waar vertrouwelijk mee moest om gesprongen worden. Naarmate de **cryptanalyse** (dat is het

proberen ontcijferen van een ciphertext zonder dat je de geheime sleutel hebt) evolueerde moesten ook de cryptografische algoritmes verbeteren. Ook hier zien we weer diezelfde wedloop tussen digitale stropers en cyberboswachters. Hoe sterker onze computers worden (met dank aan de wet van Moore) hoe krachtiger onze algoritmes moeten worden. De eenvoudigste vorm van cryptanalyse, bruteforcing, is rechtstreeks afhankelijk van de snelheid van de computer. Hoe meer sleutels per seconde een computer kan testen, hoe sneller de originele sleutel kan gevonden worden.



De volledige geschiedenis van de cryptografie hier vertellen zou ongeveer 1200 pagina's vereisen. Het briljante boek "The Codebreakers" van David Kahn is een aanrader voor eenieder die meer willen weten over deze boeiende geschiedenis. Laat de 1200 pagina's je niet afschrikken, het boek leest als een echte thriller.

Alle bestaande cryptografische systemen kunnen op verschillende manieren gekarakteriseerd worden (bron Network Security Essentials, door William Stallings):

- De **acties** die op de data wordt uitgevoerd om deze te encrypteren:
 - Substitutie: een teken door een ander teken vervangen.
 - Transpositie: een teken naar op een andere plek in de tekst zetten.
 - Product: een combinatie van meerdere substituties en transposities.
- Het **aantal sleutels** dat nodig is:
 - 1 sleutel, ook wel "private encryption" genoemd.
 - 2 sleutels, ook wel "public encryption" genoemd.
- De **manier** waarop de data wordt verwerkt:
 - Als een blok data, blok per blok.
 - Als een stream, teken per teken.

Kerckhoffs principe

We gaan zo meteen bekijken hoe encryptie effectief gebeurt, maar we willen al even een mythe onderuit halen. Binnen encryptie heb je de *principes van Kerckhoff*, 6 regels die in de 19e eeuw werden vastgelegd waarin encryptie-algoritmes moeten voldoen om als veilig beschouwd te worden. Sommige principes zijn, onder ander door onze steeds krachtige computers, niet meer relevant, meer 1 blijft ongelooflijk belangrijk: " het kennen van het gebruikte encryptiealgoritme door de *tegenstander* is geen probleem, het is enkel de geheime sleutel die ten allen tijde uit de handen van de tegenstander moet blijven."

Dit ogenschijnlijk eenvoudige zinnetje is veel zeggen: **je sleutel (of paswoord) is wat je het beste moet beschermen. Zonder kennis van de sleutel zou iemand nooit toegang mogen krijgen tot versleutelde data, ongeacht dat geweten is met welk systeem de data werd gecijferd.**

Security through obscurity is al lang een dubbel snijdend zwaard binnen de security wereld. Enerzijds is het niet aangeraden om met veel tamtam aan te kondigen hoe jij je data beveiligd. Anderzijds geeft het je mogelijk een vals gevoel van veiligheid (*snake's oil*) daar het geheimhouden van je algoritme en systemen geen garantie is dat deze ook effectief veilig zijn. In de 21e eeuw zijn de meest gebruikte encryptie-algoritmes publiek gekende algoritmes die door duizenden experts aan de tand zijn gevoeld. De kans dat er dus (bewuste) fouten in dergelijke standaarden zit is véél kleiner dan wanneer je met een zogenaamd *proprietary* systeem werkt waarvan de werking angstvallig geheim wordt gehouden.

Finaal draait alles op het geheimhouden van je sleutel, iets dat we telkens weer in dit boek zullen herhalen!



Let op encryptiesystemen die zichzelf verkopen met zinnen zoals “10 jaar nodig op een gewone laptop om alle sleutels te testen”. Dit zou kunnen doen vermoeden dat je dus voor minstens 10 jaar goed zit (we gaan er even vanuit dat de gemiddelde cryptanalist maar toegang heeft tot 1 laptop, wat uiteraard in de echte wereld niet zo is). De gemiddelde tijd van voorgaande systeem om te bruteforcen is 5 jaar, de helft.

Stel dat je een sleutel hebt die bestaat uit 8 karakters. Een karakter is een letter van a tot z (geen onderscheid tussen hoofd en kleine letters en geen getallen of speciale tekens). Er zijn 26^8 mogelijke sleutels (we gaan ervan uit dat de sleutel exact 8 karakters moet bevatten). Een computer kan 1 miljoen sleutels per seconde testen. De duur om alle mogelijke sleutels te testen is dus $(26^8)/1\,000\,000$, oftewel 208 827 seconden, pakweg 58 uur. Intuïtief zou je kunnen denken dat je dus meer dan 2 dagen “veilig” zit, wat niet zo is. De kans dat de eerste sleutel die je test reeds de juiste is, is even groot als dat het de laatste sleutel is. Kortom, gemiddeld gezien zal de sleutel in de helft van de maximum tijd gevonden was, oftewel 29 uur.

De eerste algoritmes

Het doel van ieder encryptie-algoritme is dus om data zodanig te versleutelen zodat enkel eigenaars van de gebruikte sleutel de originele tekst kunnen terugvinden. We vertelden net dat encryptiealgoritmes kunnen onderverdeeld volgens de actie die ze uitvoeren: substitutie, transpositie of een combinatie. We tonen van iedere variant nu een historisch voorbeeld.



Volgende tool, speciaal gemaakt om crypto te leren, is een erg handig iets om de verschillende cryptografische systemen te visualiseren én testen: [link](#)

Substitutie: Caesar encryptie

De Caesar encryptie (naar Julius Caesar) bestaat uit een eenvoudige substitutie algoritme. De sleutel is een getal tussen 1 en 25 en geeft aan door welk element uit het alfabet een teken wordt aangepast, als volgt:

- Ieder element wordt voorgesteld als een cijfer. A krijgt de waarde 1, B wordt 2,... Z wordt 26.
- Als de sleutel het getal 3 is, dan zal nu iedere letter A in de tekst vervangen worden door het teken 1+3, dus D. Iedere B wordt een E, enzovoort.
- Indien er een “overflow” is achteraan komen we uiteraard terug naar voor in het alfabet. Iedere Z wordt dus een C, iedere Y een B, enzovoort.



De modulo operator (%) is erg nuttig bij substitutie-algoritmes zoals bij Caesar encryptie. De modulo operator geeft de rest weer wanneer de linkse door de rechtse operator zouden delen. $19\%5$ geeft dus 4 als resultaat. Je kan de operator gebruiken om snel te weten wat de waarde van een teken wordt bij Caesar-encryptie als volgt:

`teken % sleutel => nieuw teken`

Als je dus een sleutel hebt met waarde 7 en je wilt weten wat de waarde van Y (element 25) wordt dan schrijf je:

`25 % 7`

Dit zal dus 4 worden, oftewel een D.

Het Caesercipher wordt ook wel kortweg *Rot* genoemd, naar het woord *rotatie*. Een cijfer erachter geeft dan aan welk de te gebruiken sleutel is. Rot4 wil dus zeggen dat alle elementen 4 plaatsen opgeschoven moeten worden.



Merk op dat Rot13 (ook wel *Caesaralfabet* genoemd) een speciale sleutel is. Als je namelijk 2 maal na elkaar Rot13 toepast op een tekst (eerst op de plaintext, dan op de resulterende ciphertext) dan verkrijgt men terug de originele tekst.

Uiteraard kan iedere weldenkende mens in de 21e eeuw een Caesar-encryptie bruteforcen. Het aantal mogelijk sleutels beperkt zich tot 25 mogelijkheden (sleutel 26 zal resulteren in géén encryptie: je plaintext en ciphertext zullen identiek zijn) en je kan dit dus snel testen.

Door **frequentieanalyse** op de ciphertext toe te passen kan men ook de plaintext terugvinden zonder te moeten bruteforcen. Indien de plaintext een tekst in ,bijvoorbeeld, het Nederlands is, dan kunnen we gebruik maken van de statistische eigenschappen van een taal. Zo weten we dat bepaalde letters in een standaard Nederlandstalige tekst meer of minder vaak voorkomen. De letter e komt bijvoorbeeld veel vaker voor dan de v. Daar iedere letter in de encryptie door een andere wordt vervangen, is het dus voldoende om te ontdekken (a.d.h.v. frequentieanalyse) welke letter(s) het meest of minst voorkomen om je zo een vermoeden te geven van de originele letter.



Dit verklaart ook waarom je best je te encrypteren berichten zo kort mogelijk houdt. Hoe minder tekens, hoe minder goed je frequentieanalyse zal werken. Een andere veelgebruikte fout (in klassieker encryptie) was dat de verzender bijvoorbeeld voorspelbare tekst ging encrypteren. Als je weet dat de verzender altijd begint met “Geachte” in z’n berichten, dan is de kans groot dat de eerste 7 tekens in de ciphertext deze plaintext voorstellen.

Het principe van Caesar-encryptie, de substitutie, blijft echter overeind staan en zal je nog zien terugkomen in de komende algoritmes.

Transpositie: scytale encryptie

Bij transpositie-algoritmen gaan we de positie van de karakters veranderen. De sleutel kan hierbij bepalen op welke manier dit moet gebeuren. De Oude Grieken gebruikten een zogenaamde scytale om aan transpositie-encryptie te doen. Een scytale was een lange stok bestaande uit 3 of meerdere lange zijden. De boodschap werd op een lang lint geschreven en dit lint werd dan over de skytale gedraaid. De sleutel gaf aan uit hoeveel vlakken de te gebruiken scytale moest bestaan. Ieder karakter van de plaintext (het lint) kwam op een andere zijde te liggen. Vervolgens werden alle letters op 1 zijde achter elkaar gezet, en dit werd herhaald voor iedere zijde: dit werd de ciphertext die werd doorgestuurd.

Om de nu de cipertext decrypteren werd een onbeschreven lint over de juiste scytale gelegd. Vervolgens werd de verkregen op dit lint, zijde per zijde, beschreven. Als de ontvanger dan het lint ontrolde kreeg hij terug de originele tekst te zien.



Het voordeel van een transpositiecipher is natuurlijk dat een zogenaamde “known plaintext” aanval iets moeilijker wordt. Een veel gebruikte manier die cryptanalisten vroeger gebruikten is de kennis die ze hadden van delen van de plaintext. Als geweten was dat het bericht altijd begon met “Beste dokter” dan was dit al een goede aanzet om van hieruit verder te werken.

Uiteraard zijn er tal van varianten mogelijk om transpositie te doen. Eerst kan je beslissen om je plaintext in een bepaalde vorm te plaatsen: bijvoorbeeld in 10 kolommen. Vervolgens kan je dan, gebaseerd op de sleutel, beslissen in welke volgorde je de kolommen achter elkaar plaatst om de originele tekst te krijgen. Dit is een zogenaamd *route cipher* wat onder andere werd gebruikt tijdens de Amerikaanse Burgeroorlog.

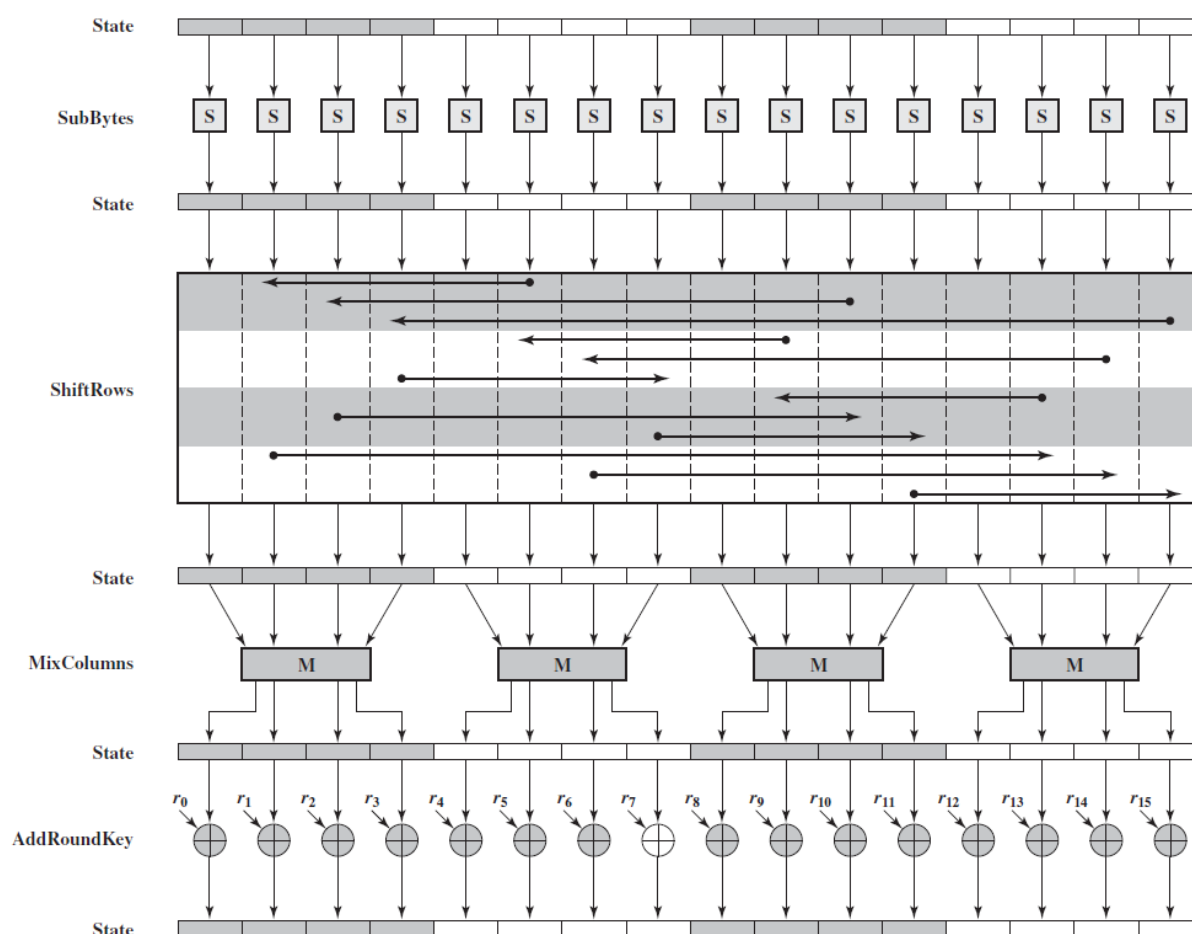


Figuur 0.2: Bron Wikipedia

Combinatie

Het spreekt voor zich dat een combinatie van een transpositiecipher en een substitutiecipher je encryptie nog versterkt. Veel moderne algoritmen kunnen nog steeds herleid worden tot een sequentie van meerdere basisvormen na elkaar.

De **Advanced Encryption Standard (AES)** is in de 21e eeuw zo'n beetje de de facto standaard als het aankomt op symmetrische encryptie (d.w.z. encryptie waar maar 1 sleutel voor nodig is, verder meer hierover). Als we echter eens het algoritme opengooien en een enkele *encryption round* bekijken (AES bestaat uit een sequentie van deze rondes) dan zien we dat de bits die bovenaan binnenkomen (*state*) vervolgens een combinatie van substituties (*sub*) en transposities (*mixcolumns* en *shiftrows*) ondergaat.



Figuur 0.3: Bron Wikipedia

We gaan AES nog terugzien opduiken wanneer we gaan bekijken hoe draadloze netwerken worden beveiligd. Als Belg mogen we trouwens erg fier zijn op deze wereldwijd gebruikte Amerikaanse standaard. Je zal later ontdekken waarom dat zo is!



Doel van dit hoofdstuk is ook aantonen dat je geen wiskundig wonder talent moet zijn om de basisconcepten van cryptografie te begrijpen. Hier en daar neem ik wat vrijheden om bepaalde stappen te vereenvoudigen, maar de essentie van de algoritmen blijft wel bewaard en daarmee, hopelijk, ook de eenvoudigheid (en dus elegantie) ervan.

Cryptanalyse

De term cryptanalyse is al enkele keren nu gevallen: de wereld van de cryptologie bestaat uit 2 delen, die elkaars tegengestelden zijn:

1. Cryptografie: de wetenschap van het versleutelen van informatie.
2. Cryptanalyse: de wetenschap van het ontcijferen van versleutelde informatie, zonder kennis van de gebruikte sleutel.

We gaan in dit boek niet te veel tijd aan de wondere wereld van cryptanalyse spenderen, daar dit ons te ver zou brengen. We vatten echter even de belangrijkste concepten hier samen.

Sleutellengtes en brute forcen

Zoals je je kan inbeelden is de sleutellengte evenredig met de tijd die cryptanalysten nodig hebben om je sleutel te bruteforcen. De maximale tijd die nodig is alle sleutels van een bepaalde lengte te berekenen kan je als volgt vinden:

$$MaximaleTijd = \frac{AantalMogelijkeTekens^{SleutelLengte}}{pogingen/seconde}$$

Een voorbeeld:

Een sleutel (of paswoord) bestaat uit 8 tekens, enkel kleine letters van a tot en met z zijn toegestaan. De berekeningen worden op een GeForce GTX 1080 gedaan die ongeveer 30 miljoen pogingen per seconde kan doen. We krijgen dan:

$$MaximaleTijd = \frac{26^8}{30000000}$$

Oftewel ongeveer 6960 seconden, wat neerkomt op ongeveer 1,9 uur tijd benodigd om alle mogelijke sleutels te testen (herinner je eraan dat deze tijd gehalveerd moet worden om te weten hoe lang het gemiddeld zal duren om de juiste sleutel terug te vinden).

Volgende tabel toont nog voorbeelden waarbij telkens dezelfde GeForce GTX kaart werd gebruikt. Het getal tussen haakjes geeft aan hoeveel mogelijke tekens er in dit type mogelijk zijn:

# tekens	enkel nummers (10)	kleine letters (26)	grote & kleine letters & nummers (62)	eender welk teken (95)
4	0,3 ms	15 ms	490 ms	2,7 s
5	3 ms	400 ms	31 s	4,3 min
6	33 ms	10 s	32 min	6,8 uur
7	330 ms	4,5 min	33 uur	27 dagen
8	3,3 s	1,9 uur	84 dagen	7 jaren
9	33 s	2,1 dagen	14 jaren	670 jaren

# tekens	enkel nummers (10)	kleine letters (26)	grote & kleine letters & nummers (62)	eender welk teken (95)
10	5,6 min	54 dagen	890 jaren	$6,3 * 10^4$ jaren
11	56 min	3,9 jaren	$5,5 * 10^4$ jaren	$6 * 10^6$ jaren
12	9,3 u	100 jaren	$3,4 * 10^6$ jaren	$5,7 * 10^8$ jaren
13	3,9 dagen	$2,6 * 10^3$ jaren	$2,1 * 10^8$ jaren	$5,4 * 10^{10}$ jaren
14	39 dagen	$6,8 * 10^4$ jaren	$1,3 * 10^{10}$ jaren	$5,1 * 10^{12}$ jaren
15	1,1 jaar	$1,8 * 10^6$ jaren	$8,1 * 10^{11}$ jaren	$4,9 * 10^{14}$ jaren
16	11jaar	$4,6 * 10^7$ jaren	$5 * 10^{13}$ jaren	$4,7 * 10^{16}$ jaren

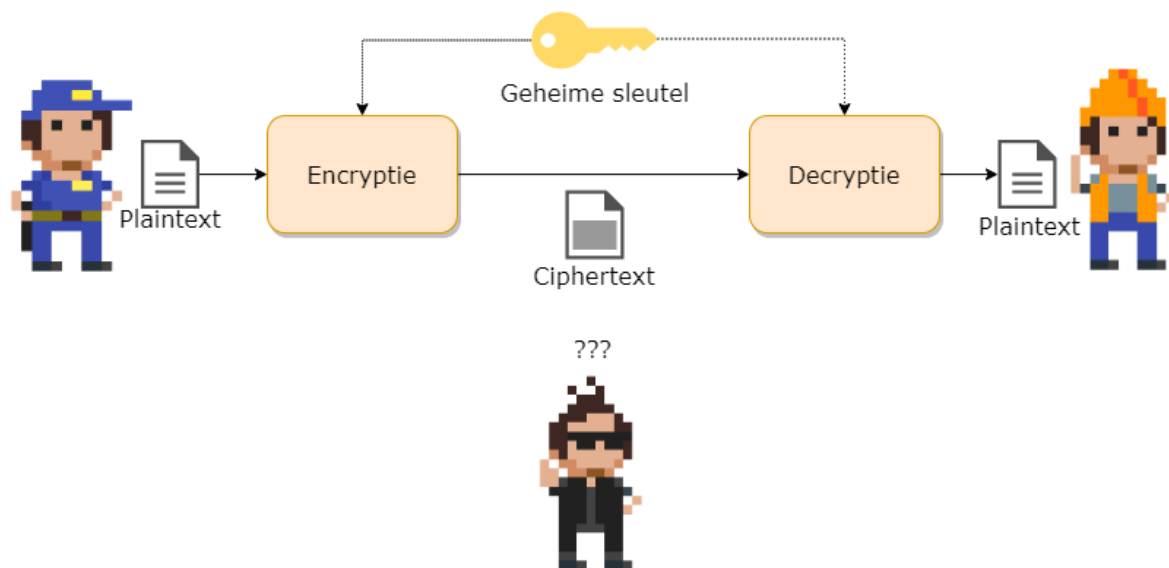


Per extra GeForce-kaart die de aanvaller zou gebruiken halveert de tijd in deze tabel.

- Sleutel lengte en permutaties berekenen afh van alfabetgrootte
- Tijd om te bruteforcen tabel
- cryptanalytics attack
- cryptanalytic attacks vs bruteforce
- Bruteforce vs dictattack
- Noot omtrent quantum computers

Symmetrische encryptie

Er zijn 2 soorten encryptiesystemen als we kijken naar het aantal sleutels. Symmetrische systemen zijn systemen waarbij maar 1 sleutel nodig is: zowel ontvanger als verzender gebruiken dezelfde sleutel. De term symmetrisch verwijst naar het feit dat het algoritme exact hetzelfde doet aan beide zijden. Het enige verschil is dat bij de verzender de plaintext in het systeem wordt gestoken, wat resulteert in een ciphertext. Terwijl de ontvanger de ciphertext in het systeem plaatst om een plaintext te krijgen.



- De symmetrische encryptiesystemen zijn de oudste vorm: alle klassieke algoritmes waren van dit principe. Asymmetrische systemen zijn pas in de 20e eeuw ontwikkeld (circa 1970).
- Voorbeelden van bestaande symmetrische encryptiesystemen zijn: AES, DES, IDEA, RC4, Blowfish, etc.

Dit type encryptie is nog steeds het meest gebruikt en wordt overal gebruikt waar data op een veilige (confidentiality) moet bewaard, verstuurd of verwerkt worden.

Sleuteloverdracht

De moeilijkheid bij symmetrische systemen is de sleuteloverdracht. Daar ontvanger en verzender dezelfde sleutel hanteren is het natuurlijk belangrijk dat deze de sleutel op een veilige manier kunnen uitwisselen. Dit probleem wordt niet opgelost door symmetrische cryptosystemen. Afhankelijk van de context kan deze uitwisseling op verschillende manieren gebeuren:

- Via een asymmetrisch encryptiesysteem dat wél sleutels op een veilige manier kan uitwisselen (zie verder).
- Via een beveiligd kanaal, in eender welke vorm (bijvoorbeeld fysiek de sleutel aan de andere persoon geven of zeggen, deze opsturen via een reeds bestand opgezet symmetrisch encryptie-kanaal, etc.)

Block en Stream cipers

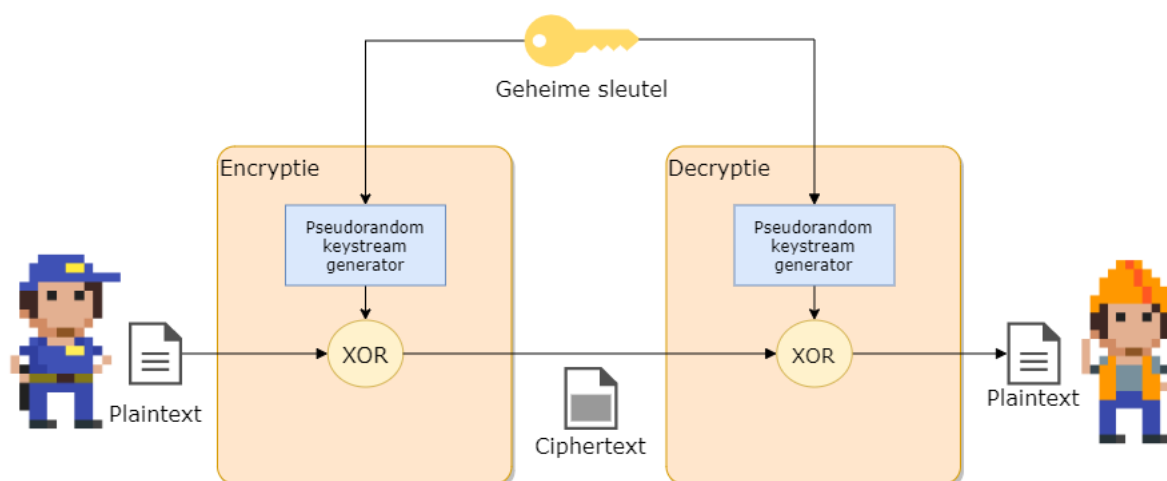
Er zijn 2 soorten symmetrische encryptiecipters als we kijken naar de manier waarop ze de te encrypteren data verwerken:

- **Streamciphers:** hierbij wordt de data letterlijk als een stream van tekens, teken per teken geëncrypteerd (RC4, etc.). Voor ieder teken dat verwerkt wordt zal er exact één geëncrypteerd teken gegenereerd worden. Dit soort algoritmes zijn over het algemeen sneller dan de blockciphers.
- **Blockciphers:** de data wordt in blokken (van bijvoorbeeld 128 tekens) verwerkt (AES, DES, 3DES, etc.) Deze vorm komt heden ten dage vaker voor.

Streamciphers

De werking van een symmetrisch stream cipher is verrassend eenvoudig en bestaat uit 2 delen:

- Een pseudorandom keystream generator: deze zal de sleutel als het ware expanderen naar een sleutel met de zelfde lengte als de stream, genaamd een **keystream**. Daar we met een stream werken zal deze generator teken per teken genereren. Hoe dit gebeurt leggen we verderop uit.
- De **xor** of “exclusieve of” functie: deze zal de plaintext naar een ciphertext omzetten door de plaintext met de keystream samen te voegen.



Aan de ontvanger zijde gebeurt exact hetzelfde. **Enkel indien de ontvanger dezelfde sleutel gebruikt, zal deze dezelfde keystream kunnen genereren, en bijgevolg enkel dan de originele plaintext verkrijgen.**

Het hart van een symmetrisch streamcipher is dus enerzijds de xor-functie én, belangrijker, de manier waarop de keystream wordt gemaakt.

De XOR functie

De waarheidstabel van de XOR-functie is de volgende:

Plaintext input	Keystream input	Ciphertext output
1	0	1
0	1	1
0	0	0
1	1	0

De xor-functie wordt in schema's aangeduid door een cirkel met een plusje in: $\oplus$

De XOR-functie heeft de fijne eigenschap dat je deze dus voor encryptie kan gebruiken.

Beeld je in dat we het bericht 1010 willen versleutelen, en we hebben een gegenereerde keystream 1101. Als we deze XOR'n dan geeft dit 0111. Dit is dus de ciphertext. Als de ontvanger dezelfde keystream kan genereren en deze xor'd met de verkregen ciphertext, dan krijgt deze terug de originele plaintext.

De keystream generator

De keystream generator heeft dus als doel om voor iedere karakter dat moet geëncrypteerd worden een bijhorend keystream karakter te maken. Deze karaktergeneratie moet onvoorspelbaar zijn (*random*) tegenover de sleutel die wordt gebruikt en het voorgaande karakter dat werd gemaakt. Echter, dit moet wel PSEUDO (*schijn*)-willekeurig zijn: dezelfde sleutel als begintpunt (*seed*) moet dezelfde reeks genereren.

De kracht (en zwakte) van een symmetrisch streamcipher ligt in de implementatie van de manier waarom deze keystream generator werkt. Mogelijke zwakheden kunnen bijvoorbeeld zijn dat de gegenereerde stroom informatie van de sleutel "lekt" naar de keystream (wat desastreuze gevolgen bleek te hebben bij de originele wifi-security (WEP) waarover later meer) of een voorspelbare "randomiteit" van de keystream?



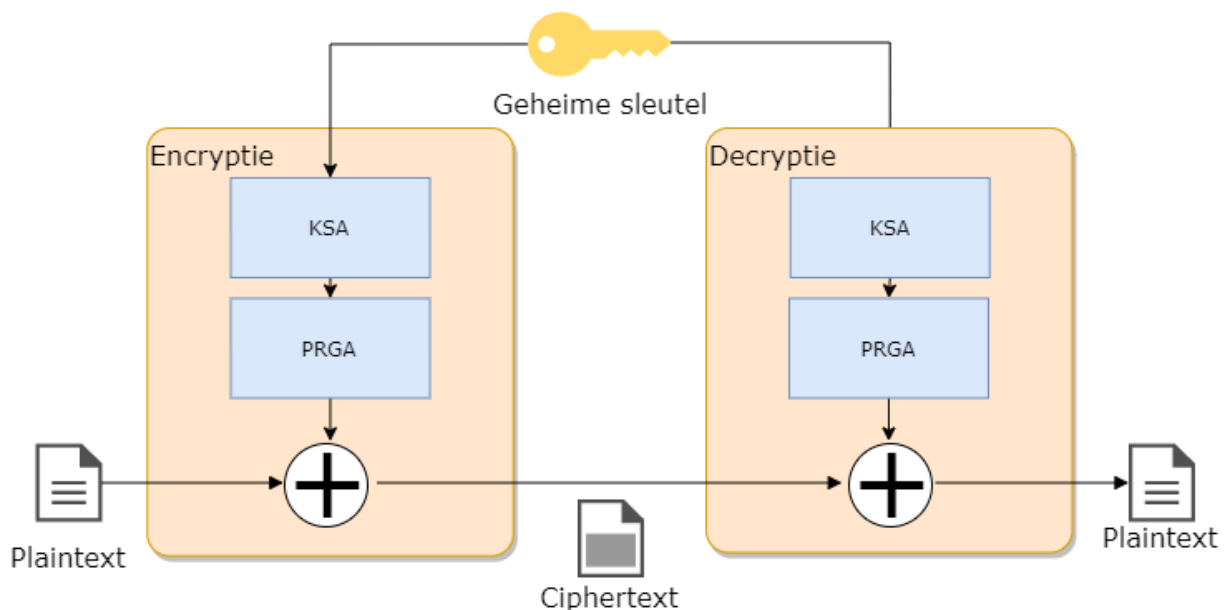
Om aan encryptie te kunnen doen hebben we systemen nodig die onvoorspelbaar zijn. Als de aanvaller kan voorspellen wat de uitvoer van een onderdeel van de encryptie zal zijn, dan kunnen we geen confidentiality en/integrity voorzien. Kortom, we hebben algoritmes nodig die willekeurige getallen kunnen generen die 100% onvoorspelbaar zijn. Net zoals het werpen van een dobbelsteen niet voorspeld kan worden, zo ook moeten onze algoritmes een (digitale) dobbelsteen hebben.

Digitale systemen die perfect willekeurige getallen genereren noemt men **random number generator** (RNG). Uiteraard moet een RNG geprogrammeerd kunnen worden: dat behelst dus een algoritme. Een algoritme is per definitie “voorspelbaar”. Alles hangt daarom af van de invoer die het algoritme gebruikt om random getallen te beginnen genereren. We spreken dan van een **pseudo random number generator** (PRNG), pseudo (**schijnbaar**) omdat de uitvoer afhankelijk is van het startgetal, de zogenaamde **seed**. Die seed kan bijvoorbeeld de encryptiesleutel zijn: enkel met dié sleutel zal het algoritme dezelfde reeks getallen generen. Er zijn echt ook systemen die bijvoorbeeld de staat van een flipflop als startpunt gebruiken: wanneer je een flipflop aanzet kan je niet voorspellen of deze op 1 of 0 zal staan (daar deze staat beïnvloed wordt door de elektromagnetische straling). Uiteraard is een dergelijke seed voor een keystreamgenerator nutteloos, daar zowel verzender én ontvanger dezelfde reeks getallen moeten kunnen genereren.

RC4 tot op het bot

Laten we eens één van de meest gebruikte streamciphers bekijken, het RC4 cipher. Dit algoritme, ontwikkeld Ron Rivest (de afkorting staat trouwens voor *Rons Cipher 4*), wordt gebruikt onder andere om een beveiligde SSL-tunnel (zie later) op te zetten en zit in het hart van veel geëncrypteerde communicatiekanalen. Het heeft weliswaar enkele zwakheden, maar mits juist toegepast kunnen die grotendeels omzeild worden.

RC4 werkt zoals we eerder verklaarden hoe een stream cipher werkt: het heeft een keystreamgenerator en zal de keystream vervolgens XO’r met de plaintext. Eerst zal de ingevoerde sleutel (die 40 tot 2048 bits lang mag zijn) omgezet worden naar een compatibele werksleutel met behulp van een **Key scheduling algorithm** (KSA). Deze werksleutel zal dan als seed gebruikt worden om een keystream in het **Pseudo-random generator algorithm** (PRGA) te maken.



KSA De **KSA** heeft al doel om, de ingevoerde 40 tot 2k-bit sleutel om te zetten naar een compatibele sleutel voor de PRGA . Het doet dit volgens een eenvoudig algoritme:

Stap 1: plaats de ingevoerde sleutel in een array (**T**) van 256 karakters. Herhaal de sleutel indien nodig.

Stap 2: Maak een array (**S**) aan, ook van 256 karakters, en plaats er de waarden 0 tot en met 255 in.

Stap 3: permuteer (*transpositie*) de elementen in de array T met behulp van de array S als volgt:

```
j = 0
for i = 0 tot 255
{
    j = (j + S[i] + T[i]) % 256
    Verwissel(S[i],S[j])
}
```

Deze stap zal dus de elementen in de sleutelarray S naar nieuwe posities in de T array plaatsen en deze ook de hele tijd van plek wisselen. De index j is afhankelijk van de sleutelwaarde en zorgt er dus voor dat iedere sleutel een unieke array T zal opleveren.

Uitgewerkt voorbeeld van KSA Stel dat onze sleutel “ab” is. De decimale ASCII-waarden van a en b zijn 97 en 98 respectievelijk. Onze array T zal dus bestaan uit 128 keer de waarden 97 en 98 na elkaar aan de start:

Index	Waarde
S[0]	97
S[1]	98
S[2]	97
etc.	

Stap 2 genereert de tabel T:

Index	Waarde
T[0]	0
T[1]	1
T[2]	2
etc.	

Als we dan stap 3 toepassen dan krijgen we na de eerste iteratie van de loop ($i=0$):

$$j = (0 + S[0] + T[0]) \% 256$$

oftewel

$$j = (0 + 0 + 97) \% 256 \Rightarrow j \text{ wordt } 97$$

De eerste wissel die in S zal plaatsvinden is dan `Verwissel(S[0], S[97])`

S ziet er dan als volgt uit na de eerste iteratie:

Index	Waarde
S[0]	97
S[1]	1
S[2]	2
...	
S[97]	0
etc.	

En dit herhalen we nog 255 keer.

PRGA Nu we een compatibele sleutel T hebben kan de keystream generatie van start gaan. Deze bestaat uit een loop die blijft doorgaan telkens een nieuw plaintext karakter binnenkomt, als volgt:

```
i = 0
j = 0
Herhaal telkens keystream karakter nodig is
{
    i = (i+1) % 256
    j = (j + S[i]) % 256
    Verwissel(S[i],S[j])
    k = S[(S[i]+S[j]) % 256]
    output k naar keystream
}
```

Zoals je ziet zal dus de array S de hele tijd van “gedaante” veranderen (hij blijft bestaan uit de getallen 0 tot en 255 maar al lang niet meer in volgorde) en telkens zal het algoritme één specifieke waarden (tussen 0 en 255) uit de array teruggeven als keystream karakter.

Uitgewerkt voorbeeld van PRGA Als we verder werken met de tabel K van het vorige uitgewerkte KSA voorbeeld, dan krijgen we dus bij iteratie 1 van de PRGA



We veronderstellen even dat we de KSA niet verder hebben uitgevoerd en de tabel S dus dezelfde is gebleven als op het einde van het voorbeeld wat in het echt niet zal zijn.

```
i = (0+1) % 256 => 1
j = (0+97) % 256 => 97
Verwissel(S[1],S[97]) => Verwissel(1,0)
k = S[ (S[1]+S[97])%256 ] => k = S[ (1 + 0) % 256]
we outputten de waarde die op S[1] staat
```

Finaal zal deze *geoutputte* waarde k ge-xor'd worden met het huidige karakter van de plaintext.



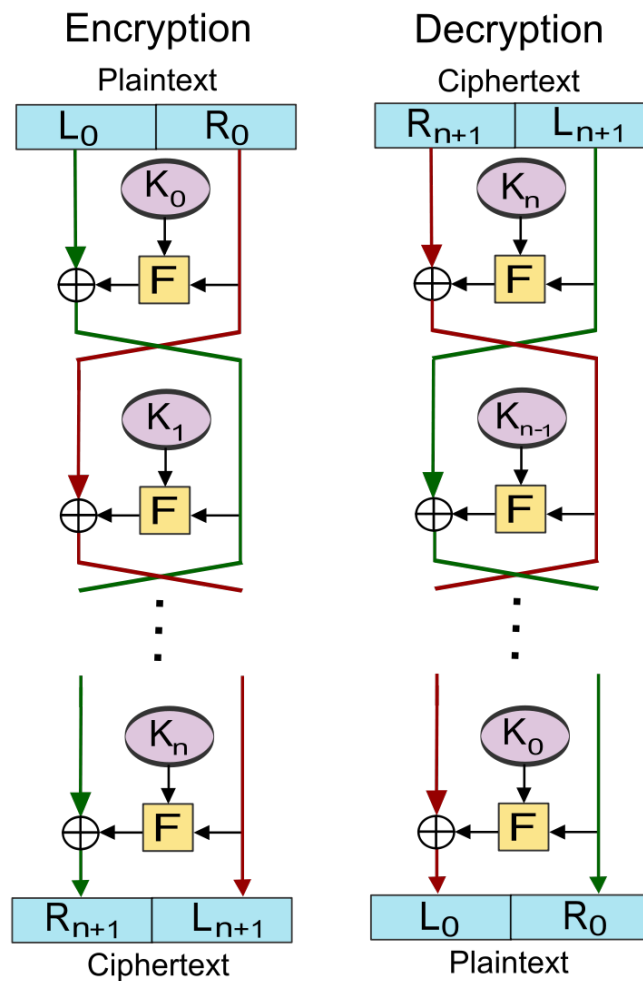
Zo, dat viel nog mee he? Zoals al gezegd, een belangrijke motivatie van dit boek is aantonen dat je niet bang hoeft te zijn van wat er achter de schermen van de cyberwereld gebeurt. De hoeveelheid wiskunde die we bijvoorbeeld nodig hadden is beperkt gebleven tot onze trouwe modulo (%) -operator en meer niet. Wanneer we zo meteen een block ciphers gaan uitkleden zal je ook daar ontdekken dat je best in staat bent schijnbaar complexe technologieën te begrijpen. Hop naar de blockciphers dus!

Blockciphers

Blockciphers, de naam zegt het al, zal eerst de plaintext in blokken karakters opdelen (bv 128 bits) en vervolgens blok per blok encrypteren.

Feistel structuren

Ook hier zullen we dezelfde soorten operaties (XOR, substituties en transposities) zien terugkomen. Echter, ook zogenaamde **Feistel**-structuren worden hier gebruikt: in deze operatie zal steeds de data in 2 helften worden gesplitst en wordt steeds een specifieke operatie (aangeduid met F van functie in de figuur), zoals een substitutie, op 1 helft uitgevoerd dat dan wordt ge-xor'd met de andere helft. Dit wordt meerdere keren herhaald, waarbij de linker (L) en rechterzijde (R) steeds afwisselend door de specifieke encryptie-operatie gaan. Net zoals bij RC4 zullen we ook vaak met een zogenaamd key scheduling algoritme werken zodat de sleutel niet constant doorheen het hele proces dezelfde is en we dus met **subkeys** werken (K in onderstaande figuur)



Figuur 0.4: Bron wikipedia

DES tot op het bot

Een van de oudste block ciphers is **DES** oftewel **Data Encryption Standard**. Deze standaard werd in 1977 geboren en vereiste toen blokken van 64 bits data en gebruikte een 56bit sleutel. Ondertussen is dit cipher zeer outdated, maar we gaan hem toch bekijken omdat deze enerzijds erg belangrijk was voor de wereld - grote delen van het bankwezen beschermde er hun financiële transacties mee (en nadien met de opvolger 3DES, zie verder)- en de standaard is erg duidelijk om een goed inzicht in block ciphers te verkrijgen.



Er was veel controverse rond deze standaard (gebaseerd op het door IBM ontwikkelde Lucifer cipher) omdat de originele versie (genaamd Lucifer) werkte met een dubbel zo grootte sleutel (128 bit) en bijgevolg dus veiliger was op lange(re) termijn. De Amerikaanse veiligheidsdienst, NSA, was niet zo happig om een dergelijk goed beveiligd algoritme commercieel te maken en zorgden er daarom voor dat de sleutellengte gehalveerd werd.

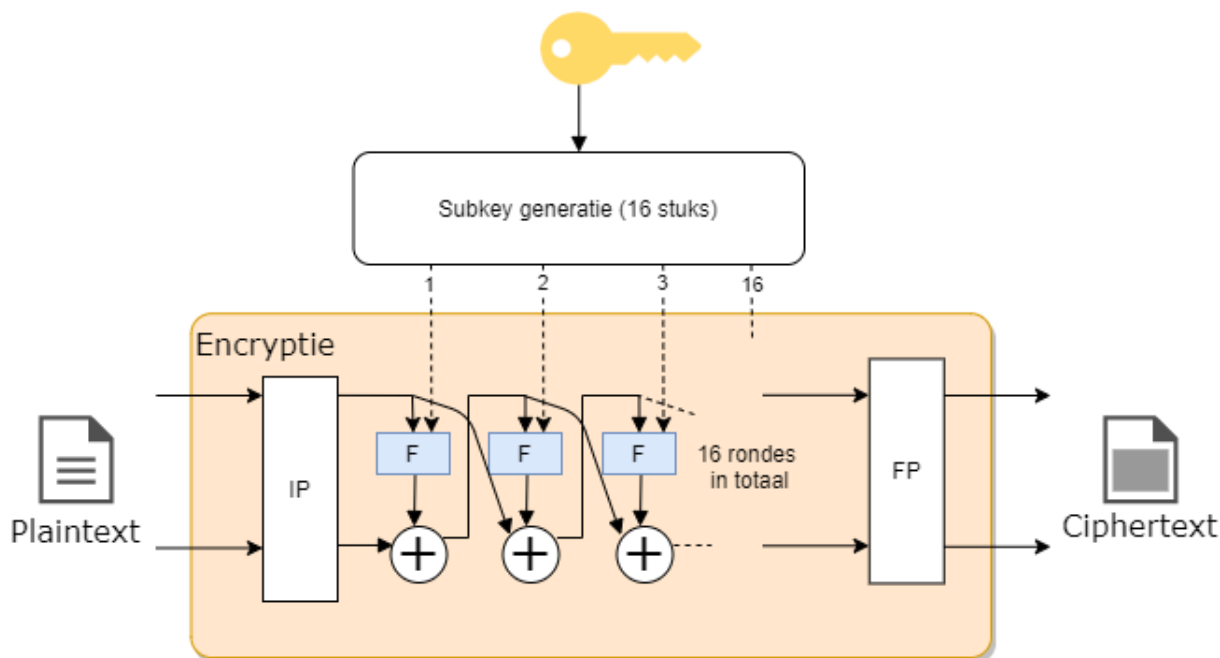
Data met DES encrypteren (en bijgevolg ook decrypteren daar het een symmetrisch cipher is) bestaat uit 2 onderdelen:

1. Versleuteling: Een reeks feistel-structuren na elkaar (**16 rondes**) die de data block per block versleutelen.
2. Subkeys maken: Een key scheduling algoritme dat 16 subkeys genereert (1 voor iedere encryptieronde), gebaseerd op de 56 bit sleutel.



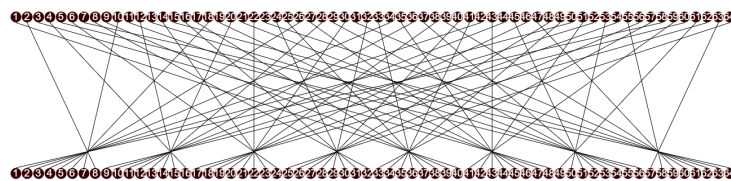
Je kan de DES standaard [hier](#) nalezen en ontdekken dat deze niet zo lang is zoals je zou verwachten van een wereldwijd geadopteerde standaard.

Versleuteling Volgende schema toont de encryptie bestaande uit 16 rondes:



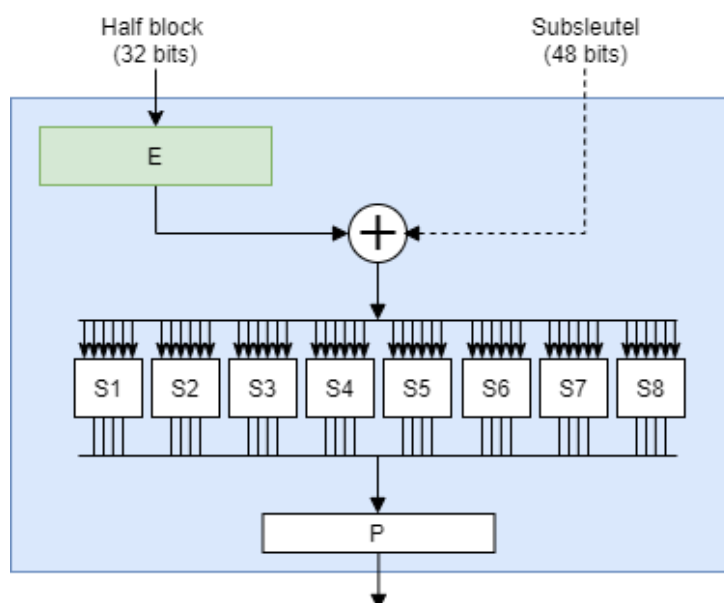
De data wordt blok per blok doorheen dit gedeelte gestuurd. Eerst gebeurt er een zogenaamde *Initiële permutatie* (IP in de figuur) waarbij iedere bit naar een andere plek wordt gestuurd volgens een vast

patroon. Achteraan gebeurt dit nogmaals in een *Finale permutatie (FP)*.



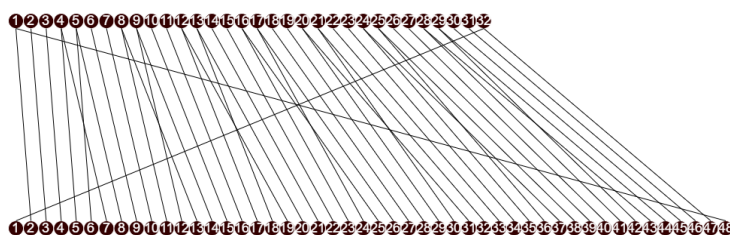
Figuur 0.5: De Initiële permutatie

Na de *IP* gaat de data door 16 feistel structuren die telkens het zelfde doen. De data wordt in 2 helften gesplitst waarbij de rechterzijde door de F-operatie gaat (die we zo meteen toelichten), het resultaat hiervan wordt ge-xor'd met de linkerhelft van de data. Het resultaat van deze XOR, een 32 bit blok, wordt nu het rechterblok in de volgende ronde en omgekeerd.



Figuur 0.6: Binnenin de F-operatie

In het F-blok wordt eerst het 32-bit blok uitgebreid (*E* in de figuur, van expansie) naar een 48 bit blok zodat deze even lang is als de subkey voor deze ronde. De expansie gebeurt, net als de initiële permutatie, volgens een vast patroon:



Figuur 0.7: De expansie van 32 naar 48 bits

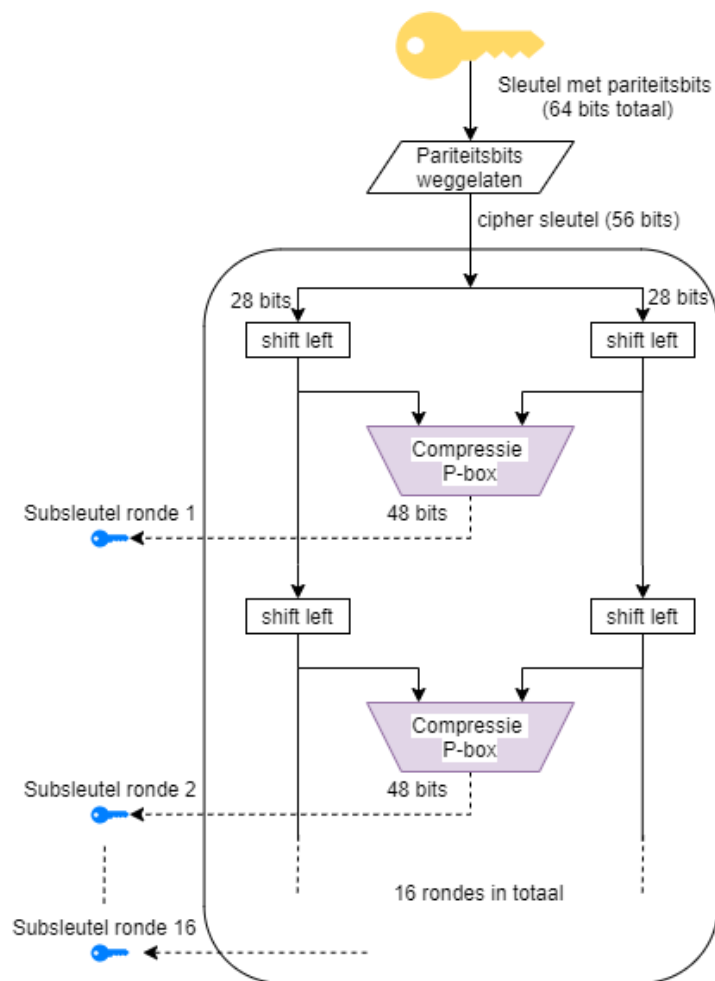
Nu worden deze 48 bits ge-xor'd met de subkey. Het resultaat wordt in blokjes van 6 bits door een *S*-blok gestuurd (zogenaamde *Selection blocks*). In dit blokje wordt 6 bit omgezet naar 4 bit. In de figuur hieronder zien we bijvoorbeeld hoe de omzetting in blok *S5* gebeurt. Ieder blokje heeft een soortgelijke tabel, maar met andere resultaten. De 6 bits bestaan uit de 2 outer bits, namelijk de eerste en de laatste bit, alsook de 4 innerbits. De figuur toont bijvoorbeeld dat de output 1001 zou zijn indien er 011011 in het blok wordt geplaatst.

S ₅		Middle 4 bits of input															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Outer bits	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

Figuur 0.8: Waarheidstabel van het *S5*-blok (Bron wikipedia)

Finaal krijgen we dus terug een 32 bit blok dat nog een laatste permutatie ondergaat die weer de bits van plaats verandert en de output hiervan is een geëncrypteerd blok data dat kan doorgestuurd worden naar de ontvanger.

Subkeys maken Iedere ronde tijdens de versleuteling vereist een sleutel. Om te voorkomen dat steeds de hoofdsleutel wordt gebruikt (en er zo potentieel dezelfde keystreams worden gemaakt) wordt deze sleutel doorheen een *round-key generator* algoritme gestuurd. Dit algoritme bestaat uit 16 rondes waarbij de 56 bit sleutel (8 van de 64 bits in de originele sleutel zijn zogenaamde pariteits-bits, die dienen om te controleren of de sleutel geen fouten bevat) steeds in 2 helften van 28 bits wordt geknipt.



Figuur 0.9: De 16 rondes die telkens 1 subkey maken

Iedere ronde wordt iedere helft van de sleutel 2 bits *geshift* (in ronde 1,2, 9 en 16 maar 1 bit). Dat wil zeggen dat alle bits 2 plekje opschuiven en de eerste (of laatste) bits komen dan achteraan (of vooraan) te staan. Deze 2 geshifte helften worden dan enerzijds doorgestuurd naar de volgende ronde, anders naar een *compressie P-box* waarvan het resultaat een 48 bits subkey zal zijn van die ronde.

De *compressie P-box* doet al vermoeden wat er gebeurt:

- Compressie: dus een aantal bits zullen wegvallen (er komt 56 bits in, maar we hebben maar 48 bits nodig)
- P-box: een permutatie oftewel transpositie dat alle bits van plek zal veranderen.

Er bestaan verschillende varianten hoe dit in z'n werk zal gaan, maar bij DES wordt volgende tabel gehanteerd:

1	2	3	4	5	6	7	8
14	17	11	24	01	05	03	28
15	06	21	10	23	19	12	04
26	08	16	07	27	20	13	02
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

Voor zij die graag puzzelen, welke getallen ontbreken?



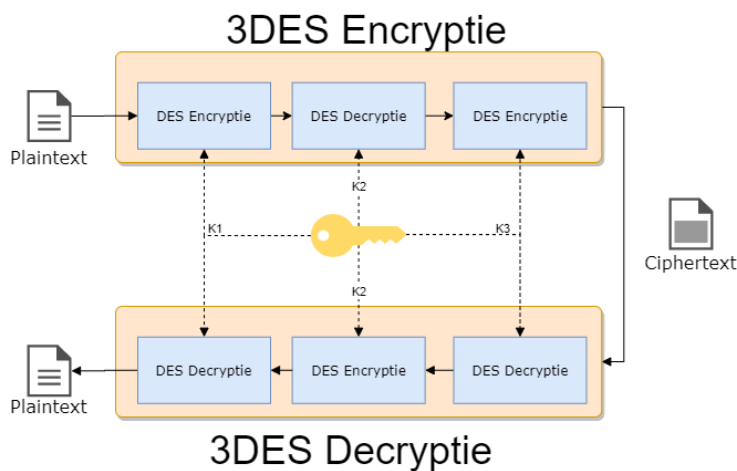
De bits op locaties 9, 18, 22, 25, 35, 43 en 54 worden geblokkeerd. Daar de sleutel steeds geshift wordt wil dit zeggen dat steeds andere bits *achtergelaten* geworden.

En zo hebben we het einde van de werking van DES bereikt. Dat viel al bij al nog mee, niet? Uiteraard hebben we nu vooral getoond *hoe* het werkt, maar niet *waarom* het werkt.

3DES

Al van bij de start gingen er stemmen op dat de originele sleutellengte voor DES (56 bits, 48 in effectiviteit vanwege de pariteitsbits) redelijk snel zou gebruteforced worden. Om die reden werd 3DES in het leven geroepen in 1995. De oplossing was een mooi staaltje compromis: het bood een verhoogde beveiliging doordat het een lange sleutel had (tot 168 bits lang) maar bleef tegelijkertijd compatibel met de bestaande DES hardware en software.

De werking van 3DES (*triple DES*) verrassend eenvoudig: ieder blok data wordt 3 keer doorheen een DES-cipher gestuurd. Hierbij wordt steeds een andere sleutel gebruikt. Om de bestaande DES hardware te gebruiken wordt hierbij de data eerst door de encryptie gestuurd, dan doorheen de decryptie en terug door de encryptie. Daar we een andere sleutel gebruiken in iedere fase heeft dit (dankzij de eigenschappen van symmetrische ciphers) als effect dat we dus effectief 3 maal na elkaar encrypteren met steeds een andere sleutel. Aan de ontvanger zijde gebeurt dan het omgekeerde: decryptie, encryptie, decryptie én dit dus allemaal met de bestaande DES hardware!



3DES laat dus ook (single) DES encryptie toe. Het enige dat je hiervoor moet doen is de subsleutels K2 en K3 gelijkstellen waardoor de 2 en derde fase tijdens de encryptie (en decryptie) eigenlijk niets doet, daar het gewoon de data encrypteerd in ronde 2 en dan ogenblikkelijk in ronde 3 terug gecrypteerd.



Het bankwezen gebruikt 3DES nog steeds (of varianten die erop gebaseerd) zijn om financiële transacties van onder andere Visa en Mastercard te beveiligen.

Block cipher modes

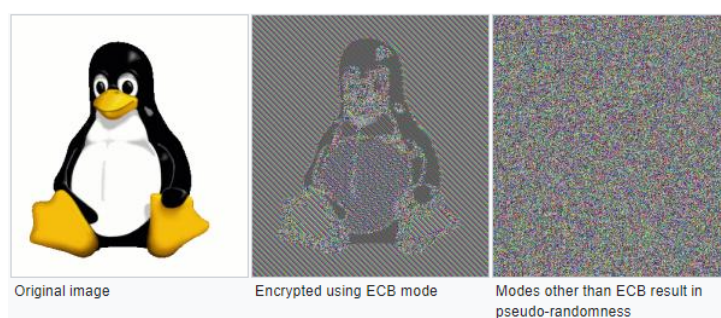
Tot hiertoe gingen we steeds blok per blok in het encryptiecipher sturen en het resultaat ervan doorsturen. Klaar. Oplettende mensen hebben hier mogelijk al een hiaat in gezien: wat als twee blokken exact dezelfde data bevatten? Beide zullen dezelfde ciphertext als resultaat genereren, daar we telkens dezelfde sleutel (en dus subkeys) gebruiken. Twee ciphertexts die identiek zijn willen we vermijden, daar het potentiële informatie over de plaintext zichtbaar maakt. Voorts laat dit soort werking ook replay attacks toe: de aanvaller kan een geëncrypteerd pakket bewaren en op een later moment terug opsturen, zonder dat hij moet weten wat de plaintext bevat.



Door pakketten te sniffen zou de aanvaller kunnen achterhalen wat de mogelijk inhoud van een pakket is. Ieder netwerkprotocol volgt de standaarden die ervoor beschreven zijn en zo kan dus de aanvaller heel veel informatie ontdekken over een ciphertext gewoon ten opzichte van wanneer een pakket wordt verstuurd tegenover de andere. Als het bijvoorbeeld het eerste pakket is dat verstuurd wordt, dan is de kans groot dat dit pakket de typische “*ik wi een verbinding opzetten*”-request is.

Voorgaande modus, waarin we ieder blok onafhankelijk van het vorige encrypteren, noemen we de **Electronic Codebook (ECB)** modus. Alhoewel deze modus dus duidelijk een veiligheidsprobleem met zich mee draagt, heeft deze modus ook één voordeel:

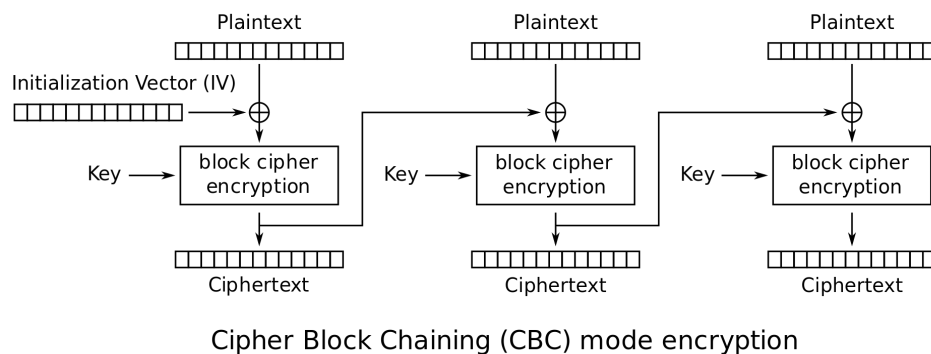
- Ieder blok wordt onafhankelijk van andere blokken gedecrypteerd. Als er dus een blok door wat voor fout niet gedecrypteerd worden, dan heeft dat geen invloed op de daaropvolgende. Dit is dus voor streaming-situaties nuttig: beeld je in dat je decryptie faalt halverwege het binnenkrijgen van een film die je aan het bekijken bent. Je zou helemaal opnieuw moeten beginnen.



Figuur 0.10: Volgende voorbeeld toont een (overdreven) manier waarom ECB minder veilig is dan de modes die we nog gaan behandelen (Bron wikipedia)

ECB is duidelijk een niet zo veilige manier om een block cipher toe te passen. Veel interessanter (veiliger) wordt het wanneer we extra informatie gebruiken om een blok te encrypteren. **Enkel het huidige blok en dezelfde sleutel gebruiken is namelijk níét veilig.** Er zijn verschillende modes om veiliger te encrypteren dan ECB:

- Cipher block chaining (CBC): de output van het vorige block (de ciphertext) wordt mee als input voor de encryptie van het volgende block gebruikt.
- Propagating CBC (PCBC): zelfde als CBC maar bij decryptie van een block zijn ook alle vorige blokken vereist.
- Cipher feedback (CFB): ongeveer zelfde als CBC alleen wordt het vorige block iets later in de encryptie van het volgende block gebruikt.
- Output feedback (OFB): het block cipher wordt als een stream cipher gebruikt.
- Counter-mode (CTR): een extra teller wordt gebruikt als input bij de encryptie van een blok. Deze teller wordt steeds verhoogd. Eén van de meest gebruikte modes (in onder andere WPA2 en IPSEC).



Figuur 0.11: CBC encryptie (Bron wikipedia)

Alle modes uit de doeken doen is hier niet aan de orde maar het moge duidelijk zijn dat ECB de minst veilige mode voorhande is en deze wordt dan ook best vermeden.



Het concept **Initializatie Vector (IV)** zal je veel zien terugkomen in ciphers. Een IV is een getal dat men als extra seed meegeeft tijdens de encryptie, naast de sleutel. Op deze manier voorkomen we dat steeds enkel de sleutel als seed wordt gebruikt en we dus effectief steeds met een *andere* sleutel werken. Uiteraard zal ook de andere zijde over dezelfde IV moeten beschikken en zal deze dus doorgestuurd moeten worden. Dit gebeurt meestal via de header van het bijhorende pakketje en is ongeëncrypteerd. Dit lijkt contra-intuïtief - de IV onbeveiligd doorsturen - maar is geen probleem. Uiteraard is het belangrijk dat er een goed *IV selectie algoritme* wordt gebruikt dat bepaald hoe steeds het volgende IV moet worden berekend (bv steeds met 1 verhogen, een willekeurig, etc.).



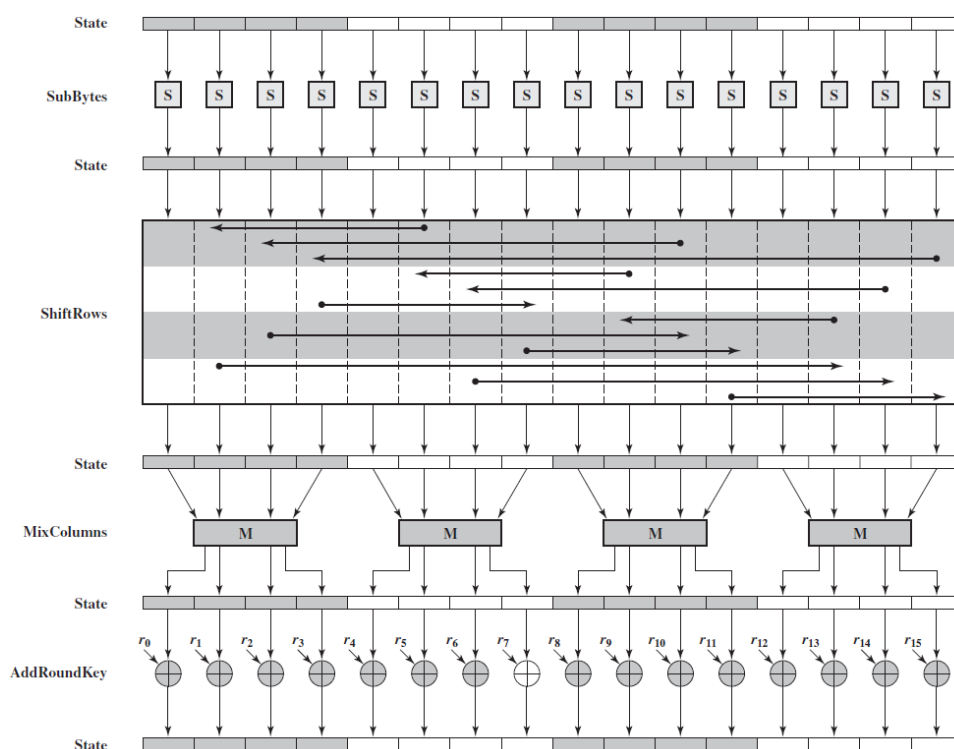
[De wikipedia pagina over block cipher] https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation modes geeft een zeer goed overzicht (én vergelijking).

AES

Alhoewel 3DES een verbetering op DES was, was er toch nood aan een nieuwe encryptie-standaard die langere tijd kon bestaan. In 2001 werd daarom de **Advanced Encryption Standard (AES)** onder het doopvont gehouden als de nieuwe defactor encryptiestandaard wereldwijd. Deze Amerikaanse standaard is gebaseerd op het **Rijndael** algoritme waar we als Belgen fier op mogen zijn: Rijndael is ontwikkeld door 2 Belgische KUL-cryptografen Vincent Rijmen en Joan Daemen.

AES is een symmetrisch block cipher dat data in blocks van 128 bits zal opsplitsen en sleutel van tot 256 bits lang toelaat. De volledige werking van AES gaan we hier niet uit de doeken doen, het voldoet te begrijpen dat in grote lijnen hetzelfde soort stappen worden doorlopen als DES en andere symmetrische ciphers:

- Er is een *key expansie* stap om een unieke sleutel p r ronde te hebben.
- De data wordt doorheen meerdere rondes gestuurd.
- Iedere ronde gebeuren er zaken zoals substituties en transposities, zowel van bytes als van hele rijen of kolommen data.



Figuur 0.12: AES encryptie (Bron wikipedia)

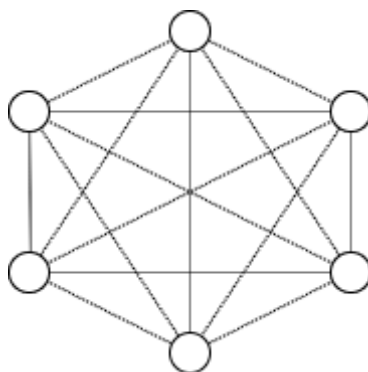
Asymmetrische encryptie

Het probleem met symmetrische encryptie

Wat als je bij symmetrische encryptie met meerdere mensen wilt communiceren zonder dat iedereen elkaars berichten kan zien? Bob kan onmogelijk dezelfde sleutel gebruiken om met Eve te communiceren die hij reeds gebruikte met Alice. Kortom, je hebt per *eindpunt* een aparte sleutel nodig. Het aantal sleutels dat je nodig hebt, zeker als ook alle gebruiker onderling nog eens willen communiceren wordt

erg snel erg groot. Je kan dit berekenen met de formule $n * \frac{(n-1)}{2}$ waarbij n het aantal gebruikers voorstelt:

- 6 gebruikers vereisen 15 sleutels.
- 7 gebruikers vereisen 21 sleutels.
- 10 gebruikers vereisen er al 45.
- 100 gebruikers vereisten er 4950!



Figuur 0.13: Bij 6 gebruikers zijn er al 15 sleutels nodig.

Symmetrische encryptie heeft dus een *key distribution problem* wanneer er encryptie op een grote schaal nodig is. Zeker als we spreken over online communicatie, over het internet, wordt de schaal ogenblikkelijk gigantisch groot en wordt het *sleutelmanagement* problematisch. Een andere oplossing is dus aan de orde.

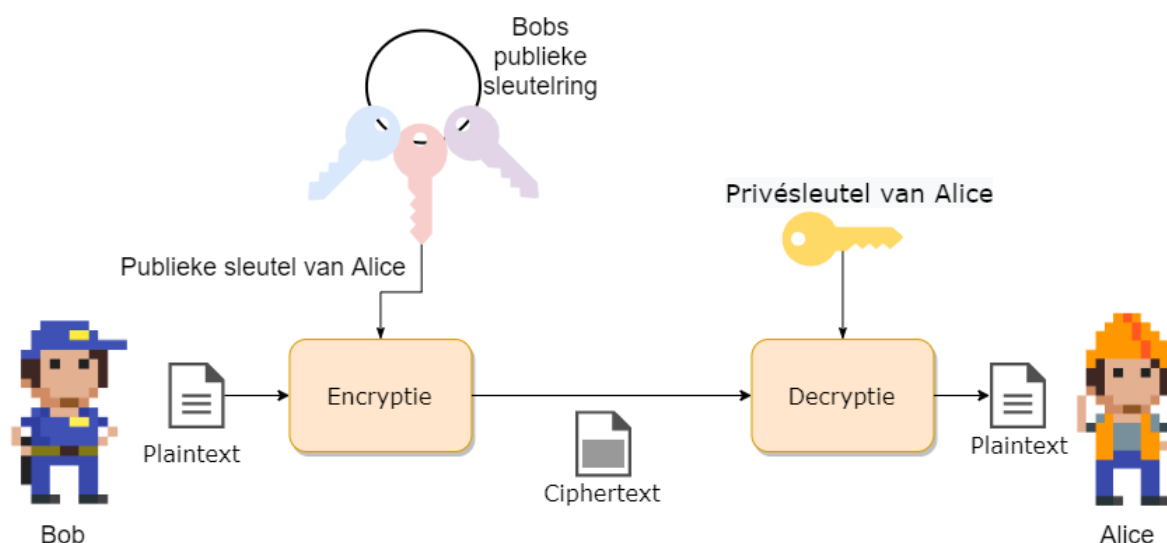
Publieke cryptografie

Publieke crypto oftewel **asymmetrische encryptie** zal het probleem met symmetrische encryptie oplossen doordat er gebruikt wordt gemaakt van 2 sleutels:

- 1 publieke sleutel
- 1 private sleutel

De publieke sleutel kan iedereen, vrij, gebruiken om versleutelde berichten mee aan te maken. Echter, enkel de eigenaar van de bijhorende private sleutel zal deze berichten kunnen decrypteren. Kortom, we lossen nu een deel van het sleutelprobleem op: iedereen heeft z'n eigen private sleutel (**en moet deze geheim houden!**) en kan via z'n publieke sleutel berichten krijgen.

De techniek werd in de jaren 70 door Diffie en Hellman ontwikkeld en had een grote impact op de manier waarop beveiligde communicatie op het internet mogelijk maakt.



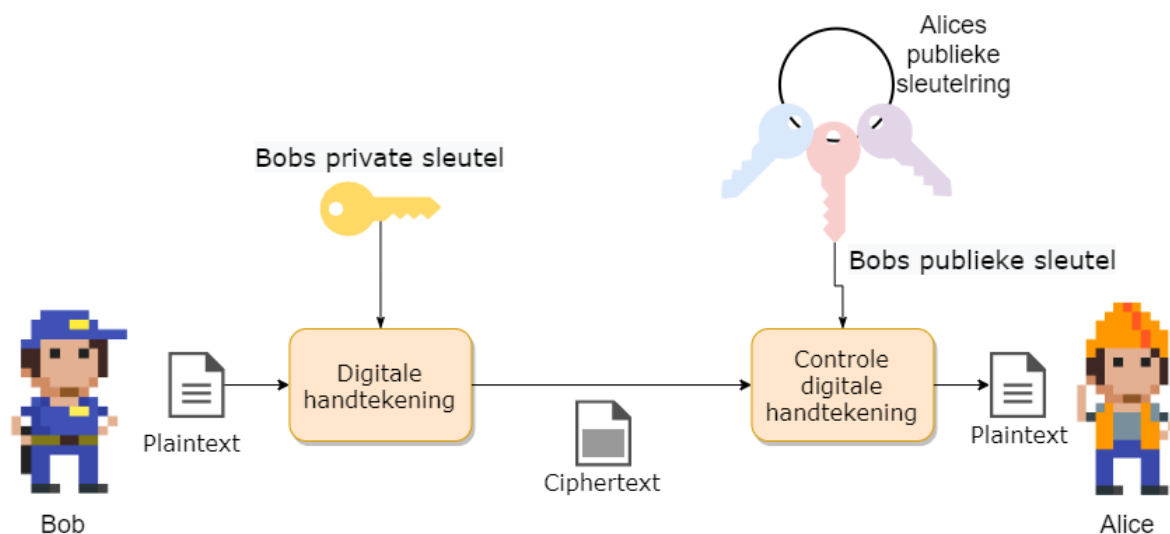
Je kan publieke encryptie beschouwen als volgt. De publieke sleutel, die niet geheim is, is een openstaande kist. Iedereen kan de kist gebruiken om een geheime boodschap in te plaatsen en vervolgens deze op slot te klikken. Enkel de eigenaar van deze kist heeft de bijpassende private sleutel die echter deze kist kan opendoen en de originele boodschap kan lezen (*decrypteren*).



Enkele van de bekendere publieke cryptosystemen zijn onder andere RSA, DSS en El Gamal.

Dualiteit van publieke cryptografie

Asymmetrische crypto zal niet alleen het sleutel-probleem oplossen, het heeft als extra eigenschap dat het publiekprivate sleutelpaar ook dienst kan doen als **digitale handtekening** om te controleren of een boodschap wel degelijk afkomstig is van een specifiek persoon. Hierbij zal een private sleutel gebruikt worden om het digitale bericht te ondertekenen. Daar de ondertekenaar de enige persoon kan zijn die deze private sleutel in z'n bezit heeft, kan men zijn identiteit bevestigen door de bijhorende publieke sleutel te gebruiken. Enkel de bijhorende publieke sleutel zal hiervoor gebruikt kunnen worden en zo hebben we een vorm van integriteit of data authenticatie.



We zullen dit concept verderop uitwerken, maar eerst gaan we bekijken hoe publieke crypto juist werkt.

Diffie-Hellman sleutel uitwisseling

Dankzij public crypto hebben we nu een systeem om sleutels op een veilige manier uit te wisselen. Het is namelijk zo dat symmetrische crypto sneller is én dus voor (realtime) communicatie interessanter is. We weten echter dat het sleutelmanagement bij symmetric crypto een probleem is als we met grote groepen gebruikers zitten. Het Diffie-Hellman sleuteluitwisselingsconcept helpt ons hierbij: het laat toe dat twee gebruikers sleutels over een onveilig kanaal kunnen uitwisselen op een veilige manier.

Zowel Bob als Alice genereren eerst een publiek/privaat sleutelpaar dat ze voor deze sessie wensen gebruiken ter communicatie. Vervolgens stuurt ieder z'n publieke sleutel naar de ander. De ontvanger zal deze publieke sleutel combineren met de eigen private sleutel wat zal resulteren in een nieuw *shared secret* dat beide nu kennen en kunnen gebruiken, bijvoorbeeld, als de symmetrische sleutel om verdere communicatie te bestendigen.

De reden dat dit werkt is met dank aan de modulo operator en de eigenschappen ervan. Een voorbeeld:

	Alice	Bob
Stap 1	Alice kiest een geheim getal A. Ze kiest $A = 3$	Bob kiest ook een geheim getal $B = 6$.
Stap 2	Alice berekent $7^A \% 11 \Rightarrow 343 \% 11 = 2$, genaamd X	Bob berekent $7^B \% 11 \Rightarrow 11764 \% 11 = 4$, genaamd Y

	Alice	Bob
Stap 3	Alice stuurt 2 naar B	Bob stuurt 4 naar Alice
Stap 4	Alice berekent $X^A \% 11 = 2^3 \% 11 = 9$	Bob berekent $Y^B \% 11 = 4^6 \% 11 = 9$

Zoals je merkt kunnen nu Alice en Bob het berekende getal 9 als gedeeld geheim kennen. Enkel zij 2 kennen dit getal.



Uiteraard zullen in de praktijk Bob en Alice véél grotere getallen kiezen dan 3 en 6.

RSA

Eén van de oudste, maar nog steeds populairste, publieke cryptosystemen is het in 1977 ontwikkelde RSA algorithm. RSA, wat staat voor de achternamen van de 3 ontwikkelaars (Rivest, Shamir en Adleman) gebruikt sleutels van 1536 tot 4096 bits lang. Het systeem is vrij traag maar heeft natuurlijk als voordeel dat het veilige sleuteltransmissie toestaat over een onveilig kanaal: we zien daarom vaak RSA gebruikt worden om eerst sessiesleutels uit te wisselen, vervolgens wordt overgeschakeld op een sneller symmetrisch cipher.

De exacte berekeningen die gebeuren tijdens encryptie en decryptie nemen ons iets te ver, maar volgend voorbeeld toont een vereenvoudigde wijze waarop RSA wordt toegepast:

Data encrypteren met behulp van asymmetrische versleuteling gebeurt op bijna dezelfde wijze als de Diffie-Hellman sleutel uitwisseling. Ook nu zullen beide zijde rekenen op de eigenschappen van de modulo-operator om over een onveilig kanaal veilige communicatie te kunnen doen.

Eerst dient een publieke sleutel aangemaakt te worden. Hiertoe dient Bob 2 grote priemgetallen, p en q te kiezen, bijvoorbeeld $p=17$ en $q=11$. Vervolgens berekent Bob N door deze priemgetallen met elkaar te vermenigvuldigen ($p \cdot q$ geeft $17 \cdot 11$, $N=187$). Bob kiest nu nog een priemgetal e , bijvoorbeeld 7. Bob kan nu zijn eigen geheime, private sleutel d maken, namelijk $e \cdot d = 1 \% ((p-1) \cdot (q-1))$ wat dus $7 \cdot d = 1 \% (16 \cdot 10)$ geeft of $7 \cdot d = 1 \% 160$. Om nu d te vinden moeten we een getal vinden zodat $7 \cdot d$ een veelvoud van $1 \% 160$ geeft, dus bijvoorbeeld 1, 161, etc. In dit geval vinden we $d=23$.



Om d te berekenen maken we gebruik van de zogenaamde *Uitgebreid Euclidisch algoritme*, een eeuwenoud algoritme gebaseerd op het *Algoritme van Euclides* dat we in het lager leerden gebruiken om de grootste gemene deler te berekenen van 2 getallen.

Bob heeft dus nu:

- Publieke sleutel bestaande uit $N = 187$ en $e = 7$.
- Private sleutel $d = 23$.

Iedereen die nu naar Bob iets wilt sturen kan dit via z'n publieke sleutel (N en e). Stel dat Alice het ascii-karakter X naar Bob wil sturen. De ascii-waarde van X is 88. De encryptie door Alice gebeurt dan als volgt: $C = data^e \% N$. De te versturen ciphertext C wordt dus: $(88^7) \% 187$ oftewel $C=11$.

Enkel Bob zal deze ciphertext met zijn private sleutel d kunnen decrypteren door $C^d (\% N)$ te doen, oftewel $11^{23} \% 187$ wat terug de plaintext 88 geeft!



de sterkte van publieke crypto stoelt dus op het feit dat ontbinden van (grote) getallen in factoren computationeel veel moeilijker is dan de omgekeerde stap, namelijk 2 getallen met elkaar vermenigvuldigen.

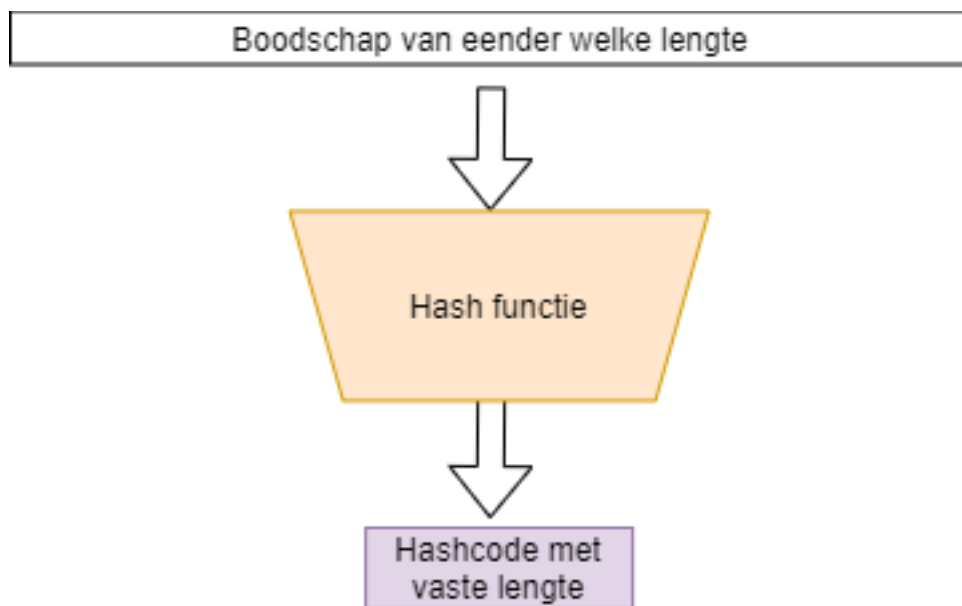
15621 in z'n factoren ontbinden is veel moeilijker dan de getallen 123 en 127 vermenigvuldigen (wat dus ook 15621 zal geven).



Zonder in detail te treden hoe cryptocurrencies en blockchains werken, is het nuttig om te vermelden dat bij cryptocurrencies ook de public crypto concepten worden gebruikt. Ook hier is je private sleutel uiterst belangrijk: enkel de eigenaar van de private sleutel “bezit” de bijhorende cryptocurrencies in de chain. Daarom is het uiterst belangrijk dat je NOOIT je private sleutel aan derden geeft, want zo geef je hen toegang tot jouw coins en kunnen ze vervolgens deze stelen door de private sleutel te vervangen.

Intermezzo: Hashes

Een zogenaamde hash is een concept uit de informatica die we gebruiken om te controleren of een digitaal stuk tekst werd aangepast of niet. Door de tekst in een hashfunctie te steken wordt een hash aangemaakt. Deze hash is een stuk code met een vaste lengte, ongeacht de originele input. Wanneer 1 bit of meer wordt aangepast in de originele boodschap dan zal deze in een totaal andere hash resulteren. Enkel dus wanneer een identiek stuk tekst als invoer (tot op bitniveau identiek) wordt gebruikt zullen twee hashen gelijk zijn.



Voorgaande is uiteraard onmogelijk: daar een hash meestal veel korter is dan de originele boodschap, is het mathematisch mogelijk dat 2 totaal verschillende teksten toch dezelfde hash geven. Het is de opdracht van een goede hashfunctie om dit soort **hash collisions** zo klein mogelijk te houden!

Een hash-functie is niet omkeerbaar: men mag onmogelijk aan de hand van een hash (ook wel *digest* of *hashcode* genoemd) terug de originele tekst kunnen achterhalen. Een hashfunctie is dus een eenrichtingsfunctie, ook wel afbeelding genoemd in wiskundige termen.

Er bestaan veel verschillende hash-functies. Enkele van de bekendere zijn:

- MD5, oftewel Message Digest 5: deze zal een 128-bit hashwaarde genereren.
- SHA-X, oftewel Secure Hash Algorithms. Zo is er SHA-256 wat een 256 bits hash zal genereren.

De sterkte van een hash algoritme zit hem in de grootte van de kans waarop hash collisions kunnen optreden. Zo zijn er bij MD5 al veel meer collisions gevonden dan bijvoorbeeld bij het recenter gepubliceerde SHA3-512 algoritme.

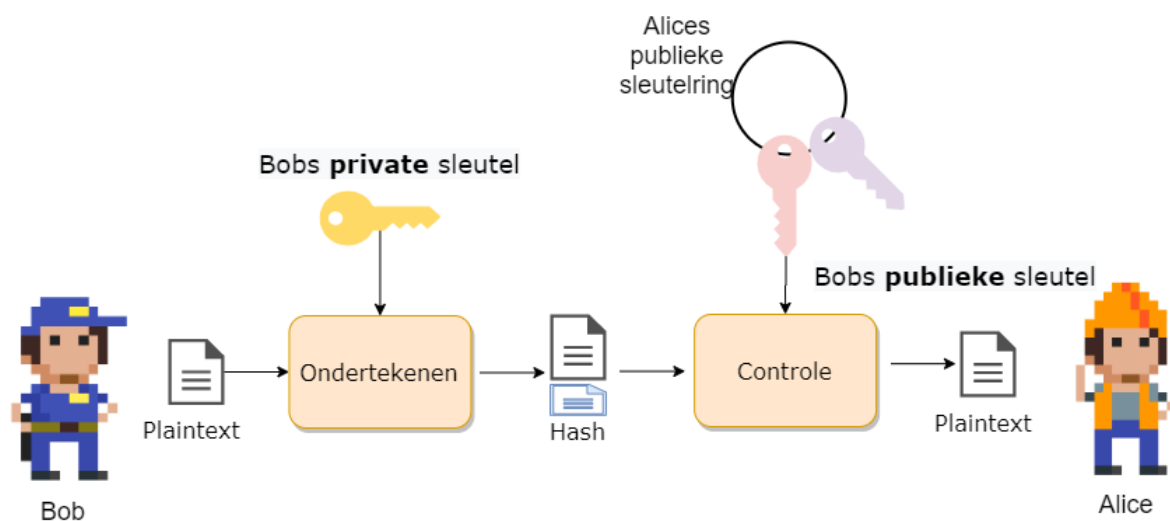
Boodschappen ondertekenen

Een probleem bij online communicatie is dat we geen zekerheid hebben dat het ontvangen bericht wel degelijk van de persoon komt van wie we verwachtten dat deze het bericht had opgesteld. We kunnen daarom het publieke crypto systeem gebruiken om berichten te ondertekenen. Daar enkel Bob de

bijhorende private sleutel kan hebben die bij z'n publieke sleutel hoort, is het bezit van deze private sleutel hebben het bewijs dat hij de rechtmatige eigenaar van een bepaalde publieke sleutel is.

Onder andere RSA laat toe om een digitale handtekening (**digital signature**) te plaatsen bij een bericht om zo te bewijzen dat de verzender de rechtmatige eigenaar van een bijhorende publieke sleutel is.

Een digitale handtekening wordt als extra bericht achteraan de te versturen boodschap geplaatst. De signature is als het ware een hash berekend aan de hand van de private sleutel. De ontvanger zal nu met de bijhorende publieke sleutel van de verzender kunnen verifiëren of de bijhorende private sleutel werd gebruikt om de handtekening te genereren.



Om een digitale handtekening te berekenen moeten we eerst een hash van het bericht berekenen. We gebruiken hier bijvoorbeeld MD5 of één van de SHA-algorithmes voor. Deze hash gaan we nu “verpakken” met de private sleutel.

Stel dat de hash 35 is en we hebben volgende sleutelpaar:

- publieke sleutel: $e=5$ en $n=91$
- private sleutel: $d=29$.

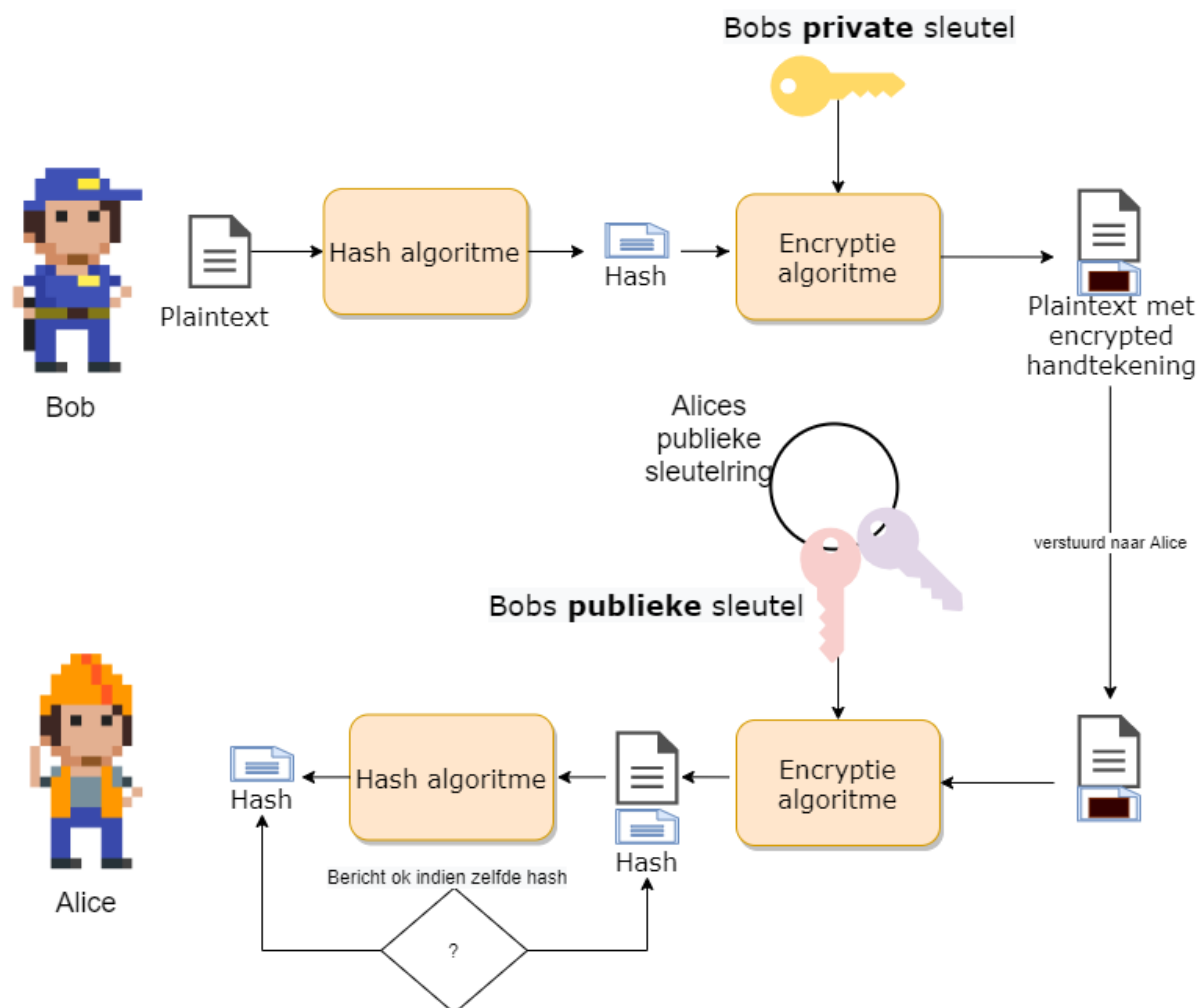
De handtekening wordt berekend door: $s = m^d \% n$ oftewel $s = 35^{29} \% 91$ wat 42 geeft.

We versturen dus naar de ontvanger de boodschap zelf en de bijhorende handtekening: 35, 42.

De ontvanger kan nu controleren of de ontvangen boodschap 35 dezelfde handtekening geeft. Hij gebruikt hiervoor de publieke sleutel e en berekent: $42^e == 35^n$, als dit overeenkomt dan weet de ontvanger dat hij het bericht kan vertrouwen.



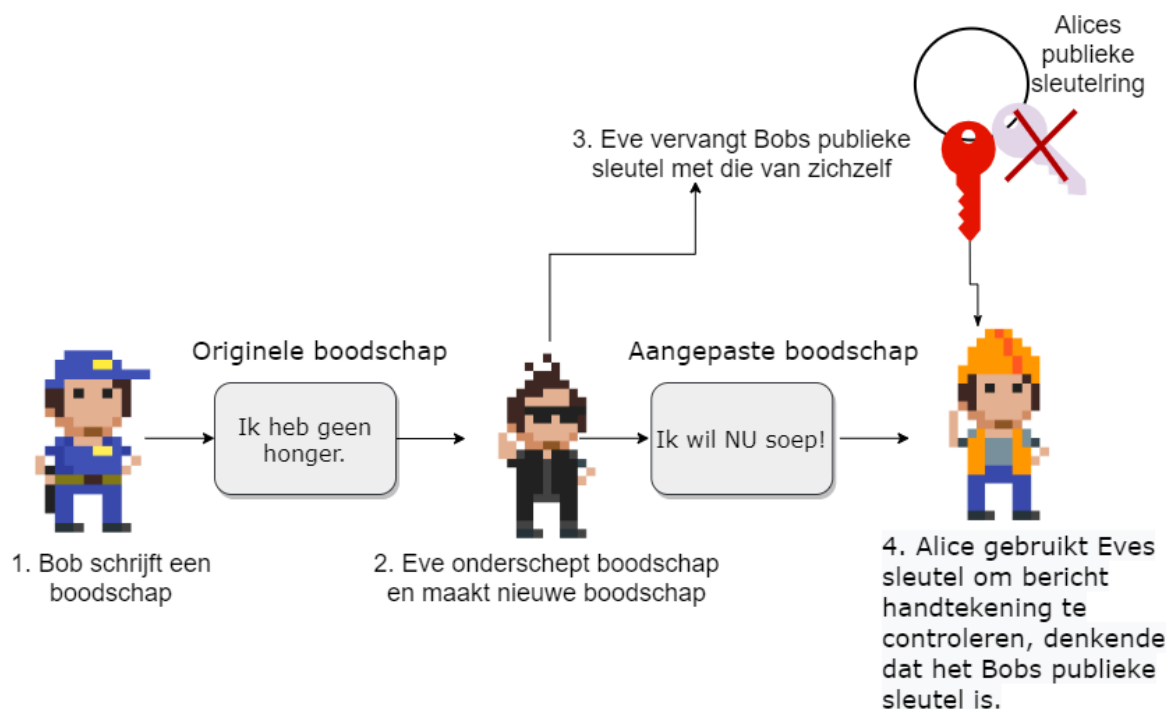
Getallen voorbeeld komt van hier



Het probleem met digitale handtekeningen

We hebben echter een probleem. Hoe weet je eigenlijk dat je wel de juiste publieke sleutel gebruikt. Publieke sleutels zijn, wel, publiek. Iedereen kan jou een publieke sleutel geven en zeggen “*Dit is de sleutel van persoon X*” zonder dat jij kan controleren of dat zo is.

We kunnen daarom als kwaadwillig persoon bijvoorbeeld een legaal bericht onderscheppen, aanpassen en dan vervolgens ondertekenen met onze eigen handtekening. Als we vervolgens aan de ontvanger kunnen wijsmaken dat jouw publieke sleutel zagezegd bij de originele verzender hoort, dan zal de ontvanger jouw aangepaste bericht “geloven”.



Kortom, we hebben een manier nodig om de **identiteit van de eigenaar** van een publieke sleutel te verifiëren. Kom binnen: **certificaten**.

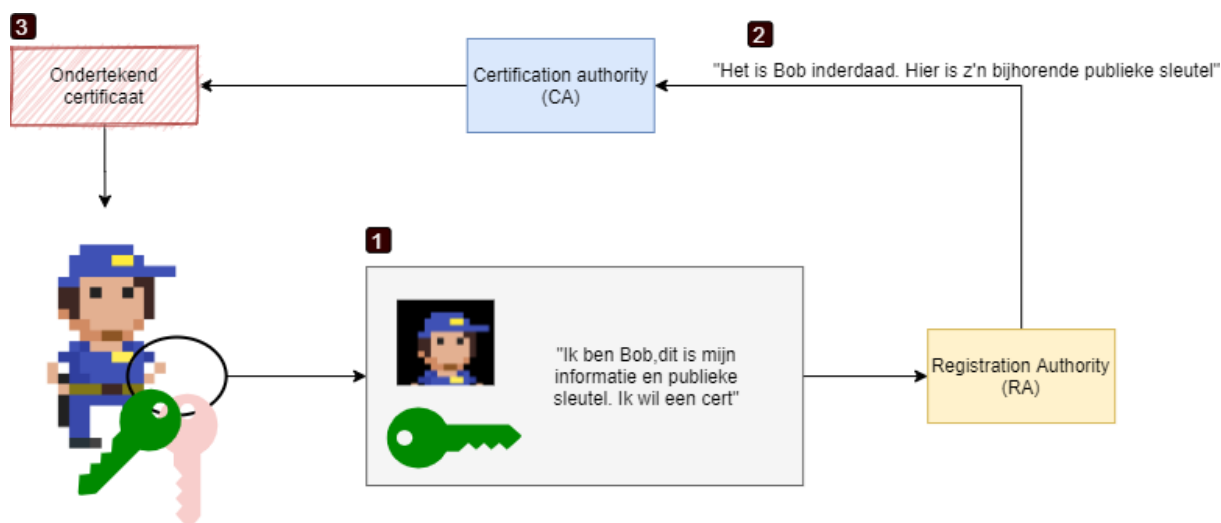
Digitale certificaten

Een certificaat is een (digitaal) document dat de identiteit van een gebruiker bindt aan een publiek sleutel. Dit document werd digitaal ondertekent door een vertrouwde derde partij (**trusted third party**) zodat bij twijfel van de echtheid van het certificaat men altijd bij deze derde partij terecht kan. Uiteraard is het belangrijk dat we deze derde partij kunnen vertrouwen, anders kunnen we ook niet de het certificaat vertrouwen dat zijn onderschrijven.



Certificaten worden beschreven in de **X.509** standaard.

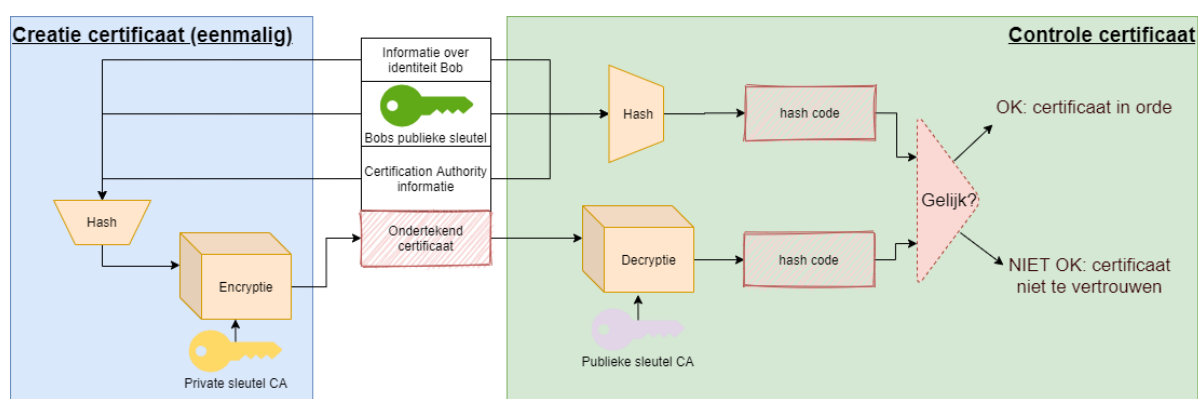
Om een certificaat aan te maken dient Bob naar een **Registration authority** (RA) gaan die zijn identiteit zal verifiëren. Dit gebeurt aan de hand van de typische documenten die ook buiten het Internet worden gebruik om iemands identiteit te bewijzen: identiteitskaart, paspoort, rijbewijs, etc. In sommige gevallen zal de RA zelfs eisen dat Bob zich naar een fysiek kantoor begeeft om daar z'n identiteit *in te flesh* te bewijzen. Indien de RA de identiteit heeft bevestigd zal deze de aanvraag van Bob doorsturen naar een **Certification authority** (CA), inclusief Bobs publieke sleutel.



Het gehele systeem van CA's, RA's, etc. dat bestaat om certificaten uit te geven, beheren en bewijzen heet een **public key infrastructure (PKI)**.

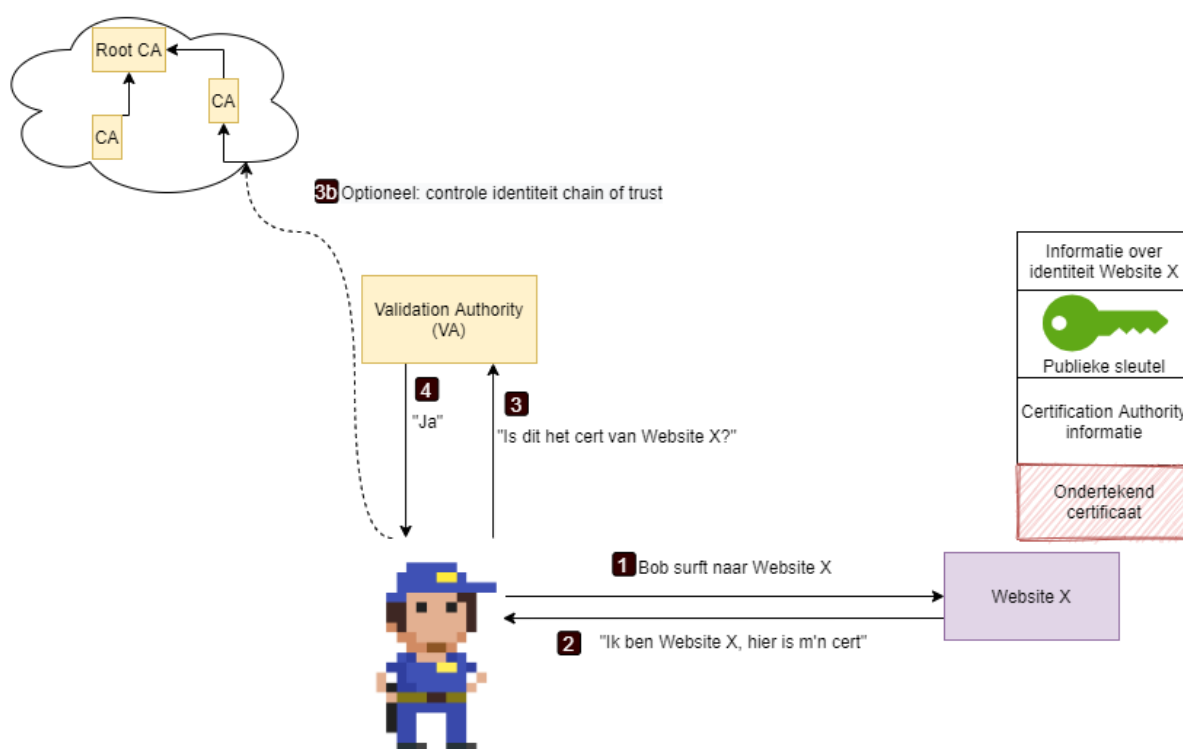
De CA zal deze informatie gebruiken om een certificaat, van een bepaalde levensduur, te genereren. Hierbij zal de echtheid van de CA achter af bewezen kunnen worden door de CA:

- Het certificaat is een geëncrypteerde hash van Bobs publieke sleutel, informatie over de CA en over Bob. De encryptie van de hash gebeurt aan de hand van de private sleutel van de CA.
- Om later de echtheid van een certificaat te testen voldoet het om dezelfde hash te genereren (publieke sleutel, info over Bob en CA) en deze te vergelijken met het certificaat na decryptie met de publieke sleutel van de CA. Als deze gelijk zijn weten we dat het certificaat door de gegeven CA werd ondertekend (enkel hun private/publiek sleutel paar zal terug de originele hash geven).

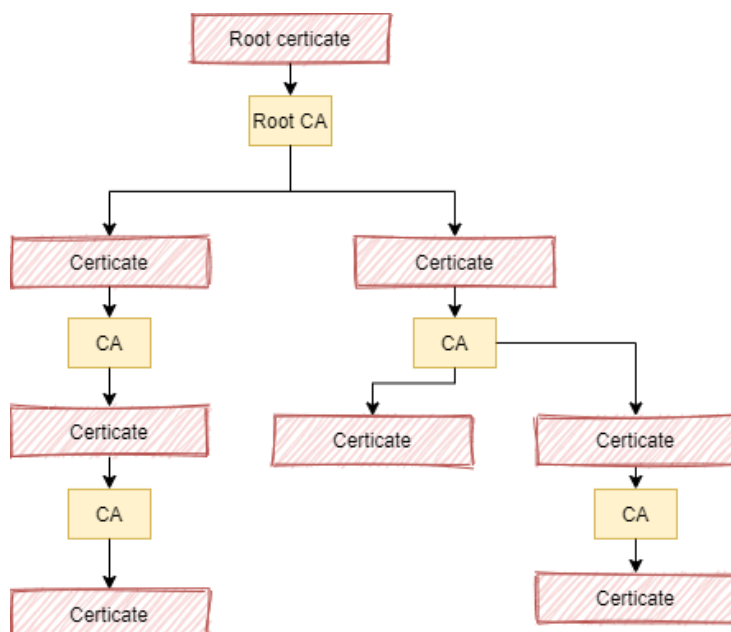


Voorgaande proces zal plaatsvinden wanneer je browser via een **https** verbinding surft naar een website en zo wil controleren of wel degelijk met de website wordt gecommuniceerd en niet met een imposter. Indien de browser (of de gebruiker) twijfelt aan de echtheid van de publieke sleutel van de CA die het

certificaat van de website ondertekent, dan zla voorgaande proces zich herhalen, maar deze keer om het certificaat van de CA te controleren met behulp van een bovenliggende CA. Op die manier kan het dus zijn dat een keten van CA's ontstaan die telkens CA's onder zich bewijzen. Uiteraard zal er steeds bovenaan zo'n ketting een **root CA** staan. Als je die vertrouwt, dan kan je al de CA's er onder dus ook vertrouwen... maar ook vice versa!



Het ergste voor een CA dat kan voorvallen is dat de betrouwbaarheid van de CA in het gedrang komt. Als een CA bijvoorbeeld weet heeft van een potentiële inbraak op hun systemen dan bestaat er de kans dat aanvallers de private sleutel van de de CA hebben bemachtigd en dus zelf certificaten *op naam van de CA* kunnen genereren, met alle gevolgen van dien! Indien dus deze kan bestaat dan is er een *breach of trust* en zullen alle certificaten van deze CA als ongeldig worden bestempeld, inclusief alle certificaten van sub-CA's! Dit kan verregaande gevolgen hebben.

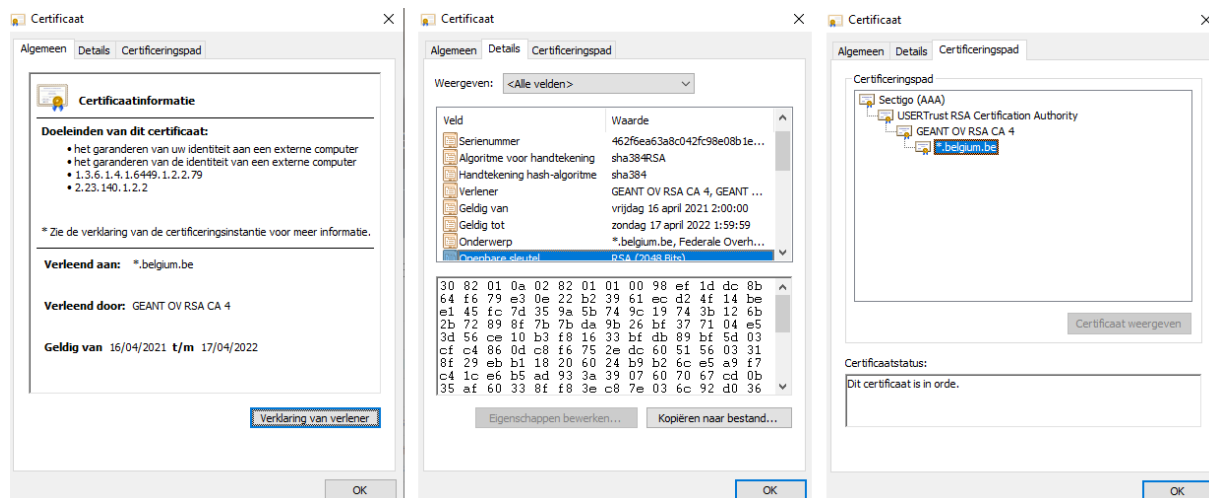


Certificaten bekijken

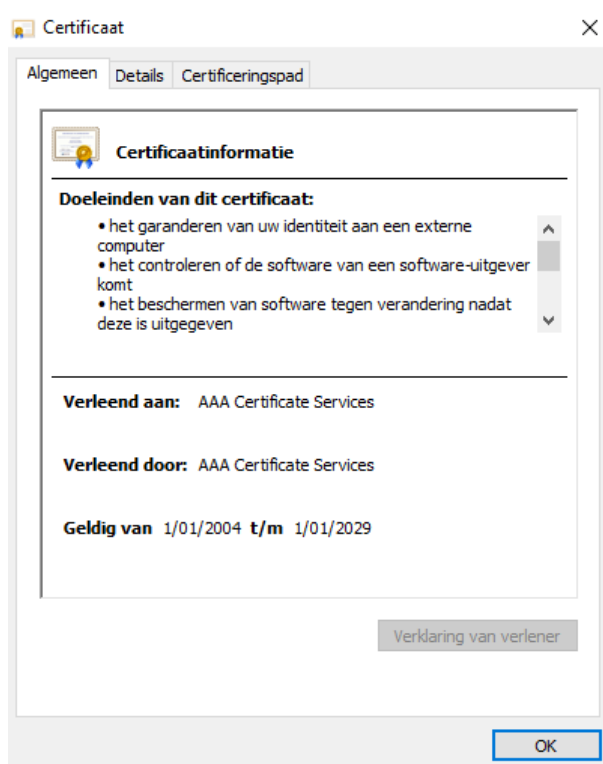
In iedere moderne browser kan je snel bekijken hoe zo'n certificaat er juist uit ziet. Als je via een https verbinding naar een website surft dan op het slotje naast de URL n de adresbalk klikt kan je doorklikken om het certificaat te openen. Als je naar <https://www.belgium.be> surft en dit doet dan krijg je eerst wat samenvattende informatie:

Zo zien we onder andere de geldigheidsduur, alsook de CA die dit certificaat heeft gegenereerd. Onder details kunnen we onder andere de publieke sleutel zien van de website alsook de gebruikte algorithmes voor de hash, e.d.

En op de laatste tab, Certificeringspad, zien we de chain of trust. We kunnen vervolgens hier de bovenliggende certificaten bekijken.



Het certificaat van Sectigo is uiteraard een **selfsigned certificate**, daar zij “bovenaan de hiërarchie staan”. Als we Sectigo niet vertrouwen dan kunnen we ook de communicatie met *belgium.be* niet vertrouwen daar.

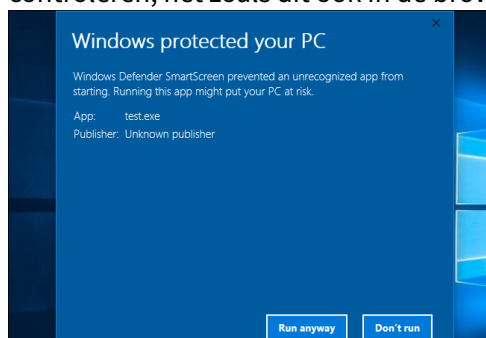


Figuur 0.14: Sectigo heeft een self-signed certificaat wat je herkent aan het feit dat de velden *Verleend aan* en *Verleend door* dezelfde waarde hebben.



Naast certificaten voor webserver (zogenaamde **SSL certificaten**) kan je ook een persoonlijk certificaat aankopen om je eigen identiteit aan derden te bewijzen tijdens bijvoorbeeld email-communicatie. Voorts heb je ook **code signing** certificaten die de echtheid van een applicatie bewijzen zodat je zeker bent dat je geen malware installeert als je programma X hebt gedownload.

Als je in Windows 10 een applicatie of installer probeert uit te voeren dan zal de ingebouwde *SmartScreen* service ogenblikkelijk de echtheid (of ontbreken van) het certificaat controleren, net zoals dit ook in de browser zou gebeuren.



HTTPS en TLS



TODO

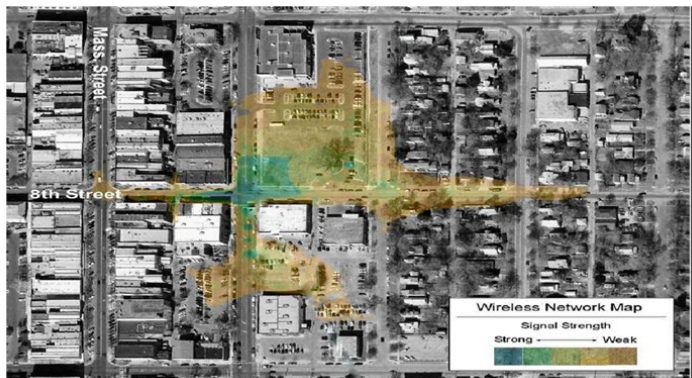
Wifi security

Alhoewel dit hoofdstuk integraal na het crypto hoofdstuk komt, is het toch interessant om dit hoofdstuk geschrinkt met het crypto hoofdstuk door te nemen als volgt:

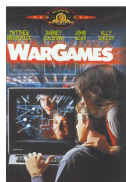
- “Crypto” tot en met “Symmetric stream ciphers”.
- “Wifi security” tot en met “WEP en waarom het faalde”.
- De rest van “Crypto”.
- De rest van “Wifi security”

De problemen van Wifi

We kunnen draadloze netwerken, specifiek wifi-netwerken, niet meer uit ons leven inbeelden. De opkomst van de IEEE 802.11b standaard (spreek IEEE uit als “*Ai-trippel-i*”) in 1999 veroorzaakte een kleine revolutie in de manier waarop bedrijven en privégebruikers konden werken. Plots kon je met een laptop van overal in het gebouw - en zelfs er buiten- op het netwerk geraken. Die vrijheid voor de gebruikers betekende wel een nachtmerrie voor de cyberboswachters. Een netwerkkabel heeft een intrinsieke extra beveiliging: enkel daar waar de kabel ligt kunnen gebruikers aan het netwerk geraken. Zolang je dus geen netwerkkabel bijvoorbeeld naar de publieke parking brengt kan niemand van daar illegaal het netwerk benaderen. Met wifi leek het alsof plotseling het hele netwerk in een straal van tientallen meters rond het gebouw beschikbaar was, met alle gevolgen van dien.



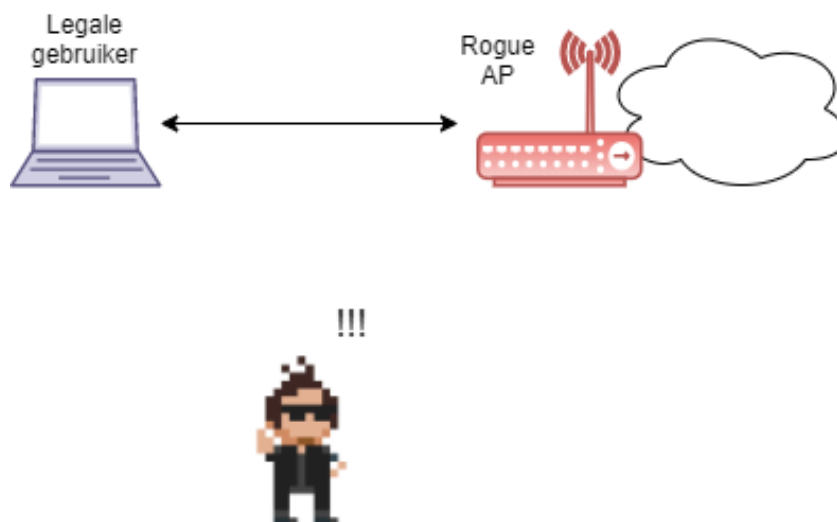
Al gauw werd een nieuwe sport uitgevonden door hobbyist hackers en professionele cybercriminelen: **wardriving**. De idee is eenvoudig: rijdt rond in de stad en laat de laptop naast je in de auto scannen naar alle netwerken, met extra aandacht voor die netwerken die geen of zwakke beveiliging hadden.



De term wardriving komt van de term *wardialing* die op zijn beurt gebaseerd is op de klassieke cyber-cult film “Wargames” uit 1983. Voor de geschiedkundigen onder ons, wardialing was het opbellen van willekeurige telefoonnummers met je modem in de hoop een zogenaamd *bulletin board system* oftewel **BBS** (een pre-internet forum zeg maar) te vinden.

Draadloze netwerken die gevonden worden hebben dan ook een schare aan problemen:

- **Eavesdropping:** iedereen kan “meeluisteren” wat er door de lucht vliegt. Dit heb je niet met een bedrade kabel.
- **Invasion:** je kan eenvoudig verbinden met het netwerk indien er geen beveiliging voorzien werd. Iets dat je dus bij een bedraad netwerk enkel kunt als je fysiek toegang hebt tot een netwerkkabel.
- **Man-in-the-middle aanvallen:** hier gaan we zo meteen dieper op in.
- **Backdoor:** een veelvoorkomend probleem zijn zogenaamde **rogue access points** die worden bijgeplaatst op het netwerk door goedbedoelde werknemers die zou het bereik van het netwerk wat willen uitbreiden. Vaak zijn echter de beveiligingsinstellingen van zo’n (meestal goedkoop) access point niet zo sterk als die van het bedrijf. Bijgevolg is dit de ideale backdoor voor malafide gebruikers die het netwerk aan het surveilleren zijn. Wanneer ze een netwerkscan doen zullen ze tientallen goed beveiligde access points zien en 1 zwak beveiligd.



Figuur 0.1: Een rogue access point is de ideale manier om binnen te geraken voor hacker.

- **Denial-of-service op fysiek niveau:** om een gebruiker toegang tot een bedraad netwerk te ontfen op fysiek niveau dien je de kabel door te knippen. Bij wifi is dit nog eenvoudiger: de “kabel” bij wifi zijn de frequentiebanden in de lucht waarbinnen de apparaten mogen werken (circa 2.4 Ghz bij de oudere wifi apparaten, nu meestal rond de 5 Ghz band). De wifi-apparaten kunnen enkel met elkaar communiceren indien zij een signaal naar elkaar over die frequentieband kunnen sturen op een moment dat niemand anders in de buurt die band gebruikt (zie note hierna). Als een malafide gebruiker dus die frequentieband vult met andere signalen, dan zullen de legale gebruikers nooit iets kunnen uitsturen. Wil je dus een wifi netwerk *Dos’n*, koop dan een signaalgenerator die op de juiste frequentieband de nodige ruist uitzend en klaar is kees.



Bedrade netwerk apparaten werken volgens het CSMA/CD (Carrier Sense Multiple Access / Collision Detection) om met elkaar over de draad te communiceren, hierbij detecteren ze wanneer er ‘botsingen’ tussen pakketen voordoen en de informatie dus opnieuw moet uitgestuurd worden. Bij draadloze netwerken is het echter onmogelijk om *botsingen in de lucht* te detecteren, daarom werken ze met een variant: **CSMA/CA**, oftewel CSMA/ **Collision Avoidance**. Een wifi-apparaat dat iets wil uitsturen zal eerst controleren of de frequentieband waarop ze werken vrij is, enkel dan zal er vervolgens een pakketje de lucht worden ingestuurd.



Sommige gebruikers schakelen het broadcasten van hun netwerknaam (het zogenaamde **SSID**) uit in de hoop dat zo dat burens, voorbijgangers en wardrivers hun netwerk niet kunnen zien. Helaas is dit zogenaamde *snake's oil*: het geeft een vals gevoel van veiligheid. Je kan weliswaar SSID-broadcasting uitzetten (dit is een pakketje dat je access point elke paar seconden uitstuurt om aan iedereen die luistert te zeggen “*Hallo, hier is het netwerk met naam X, en dit zijn de parameters die je nodig hebt om met mij te verbinden*”) maar het SSID wordt ook in een hoop andere pakketjes de lucht in gestuurd. Een beetje wifi hacker kan dus het SSID ogenblikkelijk uit de lucht plukken, ongeacht dat broadcasting werd uitgeschakeld of niet.



Om de huidige, en betere, beveiliging van wifi te appreciëren gaan we wederom even terug in de tijd om te kijken hoe de originele IEEE wifi standaard de beveiliging beschreef. Het zal een ietwat horror-achtige tocht worden waarin we gaan ontdekken dat enkele stevige hiaten ervoor gezorgd hebben dat illegale toegang tot bijna ieder wifi netwerk rond de eeuwwisseling binnen enkele minuten kon gebeuren. Lees verder en huiver mee.

Onbeveiligde managementframes

Veel keuzes die in de IEEE standaard werden gemaakt zijn vermoedelijk het gevolg dat men leentje buur is gaan spelen bij de reeds bestaande IEEE standaarden voor bedrade netwerken. Echter, bij bedrade netwerken had je niet de inherente onveilige omgeving van het draadloze aspect, waardoor op gebied van beveiliging hier weinig tot geen aandacht aan werd besteed.

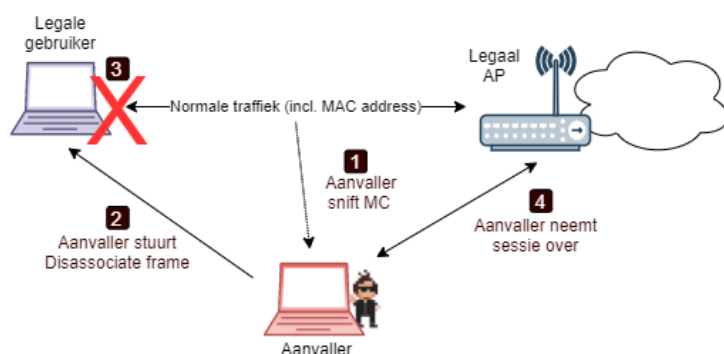
Management frames in wifi zijn frames die ervoor zorgen dat alle aanwezigen op het netwerk op een ordentelijke manier met elkaar kunnen communiceren. Deze frames zorgen bijvoorbeeld voor:

- Om een client verbinding te laten maken met een netwerk.
- Om SSIDs te broadcasten.
- Clients de opdracht te geven het netwerk te verlaten.
- Om te verbinden met een ander access point van hetzelfde netwerk (*handover*).

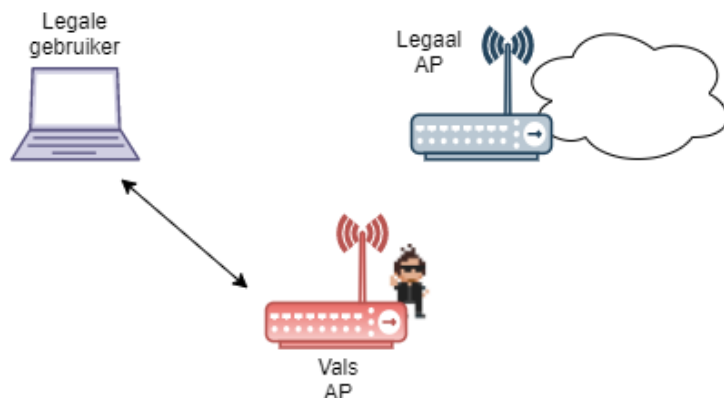
Net zoals bij bedrade netwerken, koos men bij de IEEE wifi standaard om deze managementframes **zonder enige vorm van beveiliging** te gebruiken (er werd geen *CIA* voorzien). Dit zorgde ervoor dat niet legale gebruikers zelfs management frames konden uitzenden op een netwerk, zonder dat de ontvangers ervan konden controleren of de bron wel een legaal access point of client was.

Dit resulteerde in onder andere volgende scenario's:

- Een hacker kan legale gebruikers DoS'n door constant zogenaamde *disassociation* naar hen te sturen. Dit frame, gebruikt door access points, geeft aan clients de opdracht dat ze het netwerk moeten verlaten. De hacker kan zo'n frame uitzenden en daarbij het "source" veld instellen op het MAC-adres van het access points. Dit spoofing kan ongecontroleerd, waardoor legale gebruikers dit frame altijd zullen aanvaarden én vervolgens uitvoeren: de gebruiker kan niet meer verbinden met het netwerk zolang de disassociation frames blijven verstuurd worden (**disassociation flooding**)
- **Identity spoofing** was ook eenvoudig daar de hacker eender welk veld in de frames kan aanpassen. Van zodra hij een legale gebruiker met voorgaande techniek van het netwerk heeft geschopt, kan hij vervolgens zichzelf voordoen als deze gebruiker. Hiervoor moet hij gewoon het MAC-adres spoofing van de legale gebruikers tijdens de communicatie met het access point.



- En *last but not least* laten de onbeveiligde management frames toe dat we eenvoudig een access point kunt nabootsen (**impersonation**). Vervolgens kunnen we een **man-in-the-middle aanval** uitvoeren daar een hacker zich kan plaatsen tussen de gebruiker en het internet en zo informatie kan ontfutselen.





De linux tool *AirSnarf* laat toe om fake hotspots (publiek wifi netwerk) op te zetten. Hierbij maakt het gebruik van de onbeveiligde management frames. Een scenario in België dat gegarandeerd success heeft (vanuit het standpunt van de hacker) is een fake Telenet Wifree hotspot op te zetten. Hierbij zal je eerst de login pagina van Telenet Wifree kopiëren en via een lokale webserver aanbieden aan de gebruikers die op jouw access point met de naam “Telenet Wi-Free” verbinden. Je kan nu van iedere gebruiker de gebruikersnaam en paswoord stelen telkens deze die informatie op jouw fake pagina invoert.

De 802.11 standaard qua beveiliging

De originele “ANSI/IEEE Std. 802.11” die de wifi specificaties beschrijft werd geschreven in 1999. Het had als doel *“to develop a medium access control (MAC) and physical layer (PHY) specification for wireless connectivity for fixed, portable, and moving station within a local area.”* Hoofdstuk 8 van deze standaard had als titel “Authentication and privacy” en was maar 10 miezerige pagina’s lang, in vergelijking de totale grootte van het document (528 pagina’s) was dit misschien wel een voorbode hoe weinig aandacht er aan beveiliging zou worden gegeven.



Vanaf nu zullen we geregeld het woord access point afkorten naar **AP**. In deze tekst kan een AP zowel een eenvoudig access point zijn als een complexe draadloze router of modem.

Voor we dat kleine hoofdstuk gaan openbreken -en ontdekken hoe WEP in de eerste generatie wifi-apparaten een navenante beveiliging aanbod- zullen we eerst bekijken hoe gebruikers met een draadloos netwerk effectief kunnen verbinden. Zoals reeds vermeld gebeurt dit gebruik makende van de onbeveiligde management frames.

Verbinden met een netwerk

Voor aanvallers actieve aanvallen kunnen starten op een netwerk dienen ze verbinding te maken met het netwerk. Dit proces bestaat uit 4 stappen die doorlopen moeten worden voor legale en illegale gebruiker effectief gebruik kunnen maken van het netwerk en *higher-layer* datatraffiek kunnen verwerken.

1. **Scannen:** Passief of actief zoeken naar de netwerken in de buurt.
2. **Verbinden (*joining*):** Kiezen met welk netwerk zal verbonden worden.
3. **Authenticatie:** Bewijzen dat de gebruiker toegang heeft tot het netwerk.

4. **Associatie (*association*)**: Bijhouden met welk AP de gebruiker verbonden is van het netwerk. Pas vanaf deze stap kan het netwerk gebruikt en misbruikt worden.

Scannen

In deze stap zal de client ontdekken welke netwerk er in de omgeving zijn. Oftewel zoekt de client specifiek naar een netwerk, oftewel wil hij gewoon een olijsting van alle aanwezige netwerken. Ieder draadloze netwerk heeft een netwerdnaam, de **SSID** (*service set identifier*) die bestaat uit een 32 byte ASCII character string, en zal deze op geregeld tijdstip broadcasten voor iedereen die hier nood aan heeft (we merkten al eerder op dat dit broadcasten kan uitgezet worden, maar dat dat een vals gevoel van veiligheid zal geven). Naast het SSID zal ieder netwerk ook meerdere fysische parameters uitsenden die de client in de volgende stap zal nodig hebben.

Er zijn meerdere kanalen beschikbaar in het spectrum waar binnen een netwerk mag werken. De client zal bij het scannen daarom steeds enkele milliseconden op een bepaald kanaal luisteren om potentiële SSID broadcasts op te vangen, om dan naar het volgende kanaal hetzelfde te doen.

Verbinden

Wanneer de gebruiker gekozen heeft met welk SSID er moet verbonden worden zal in deze stap de wifi-netwerkaart ingesteld worden op de juiste fysische parameters (frequentie kanaal, snelheid, etc.) zodat als het ware AP en client synchroon lopen (en dus op de juiste moment iets tegen elkaar kunnen zeggen zonder elkaars signalen te storen). In de scan-stap keken we als het waren goed rond, in deze stap richten we onze blik nu op een bepaald netwerk.

Authenticatie



Deze en de voorgaande stappen gebeurt volledig automatisch indien de gebruiker eerder heeft geopteerd om automatisch met een netwerk te verbinden.

In een bedraad netwerk zit authenticatie impliciet vervat in het fysisch aspect: wanneer je de kabel in je laptop kan steken dan is de kans groot dat je een legale gebruiker bent in het gebouw en dus is er geen extra authenticatie nodig (kort door de bocht gezien, weliswaar). Dat is niet zo met draadloze netwerken die voorbij de grenzen van het gebouw hun signaal uitsenden. Er is daarom een extra stap nodig voor we het netwerk kunnen betreden.

In de 802.11 standaard staan 2 mogelijke authenticatie-methoden beschreven (die we verderop zullen toelichten):

- **Open-system authentication**
- **Shared-key authentication**

Origineel was deze authenticatie een enkelrichtingsstraat. Enkel de client dient zich te authenticeren. Hierdoor bestaat dus de kans dat de gebruiker verbind met een rogue of fake AP.

Naast voorgaande 2 methoden is er nog een derde die niet in de standaard staat beschreven maar die wel door veel fabrikanten werd aangeboden in hun wifi-apparaten:

- **MAC Address Authentication gebruik makend van een MAC-ACL:** hierbij kan de beheerder van een AP een *access control list* (ACL) aanleggen waarin alle MAC-adressen staan van toegelaten apparaten.

Open-system authentication

Deze vorm van authenticatie werd eerst gezien als de “er is geen authenticatie nodig op dit netwerk”-versie van authenticatie. Echter, de problemen met WEP op gebied van beveiliging hebben er voor gezorgd dat sinds de WEP-opvolger WPA, deze vorm de veiligere van de twee modes is.

In netwerken met deze modus om te authenticeren worden er exact 2 frames tussen client en AP uitgewisseld:

1. Eerst vraagt de de client aan het AP toegang.
2. Vervolgens stuurt het AP naar de client een “Welkom” frame.

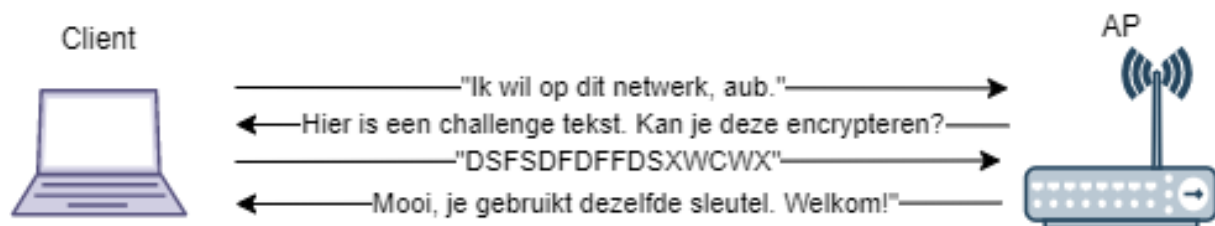
Meer gebeurt er niet in deze modus.

Als er in dit soort netwerk geen encryptie wordt toegepast dan kan dus eender welk apparaat in de omgeving verbinding maken met dit netwerk. Als er wél WEP encryptie wordt gebruikt dan zal het bezit van de WEP-sleutel als een soort toegangscontrole werken: je kan namelijk wel authenticeren (daar het AP iedereen toelaat in deze authenticatie-fase) maar vervolgens kan je geen data lezen en versturen tenzij je een geldige WEP-sleutel hebt.

Shared-key authentication

In deze modus moet je bewijzen dat je in het bezit bent van een geldige WEP-sleutel voor dit netwerk. Het AP zal daarom een **challenge-response** authenticatie opstarten bestaande uit volgende sequentie van frames:

1. Het AP maakt een random string aan, de *challenge*, en stuurt deze in plaintext naar de client.
2. De client encrypteert deze string met z’n WEP-sleutel en stuurt dit terug naar het AP (de *response*).
3. Het AP zal nu de response decrypteren met z’n eigen WEP-sleutel en het resultaat vergelijken met de challenge-string. Als beide gelijk zijn weet het AP dat de client een geldige WEP-sleutel heeft en dus toegelaten mag worden op het netwerk.



Merk op dat stap 1 en 2 ervoor zorgen dat aanvallers die deze *handshake* sniffen 2 interessante frames zien passeren. Eerst zien ze een plaintext, ogenblikkelijk gevolgd door de bijhorende ciphertext ervan (indien ze een gebruiker sniffen met een geldige WEP-sleutel). Dit zal interessante informatie blijken verderop in dit horror-verhaal waarin we zullen tonen waarom WEP niet zo veilig bleek te zijn als gehoopt.

Associatie

Na een succesvolle authenticatie krijgt de client een *association ID* toegewezen. Dit ID gebruikt het netwerk om te weten waar in het netwerk de client zich bevind. Veel draadloze netwerken bestaan namelijk uit meerdere AP's en via dit id weet het netwerk met welk AP de client momenteel verbonden is en zal alle data voor de client dan naar dat AP sturen.

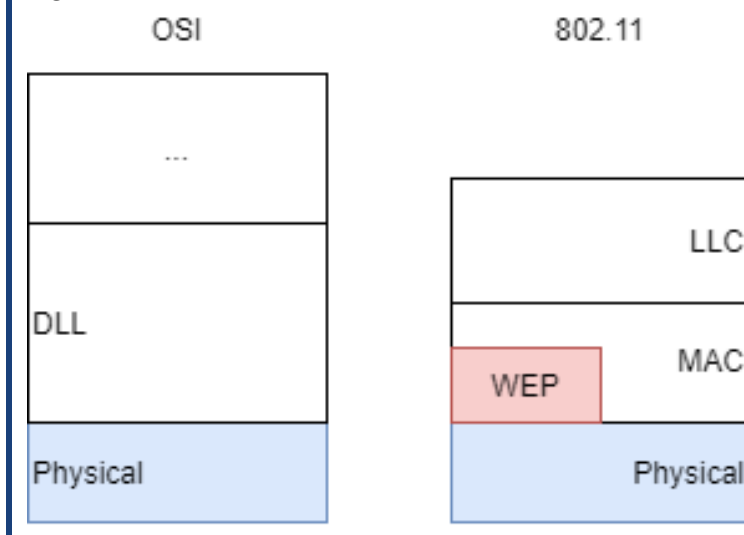
Vanaf dit punt kan de gebruiker dus de bronnen van het netwerk beginnen gebruiken en wordt het tijd om deze communicatie te beveiligen (tenzij het om een publieke hotspot gaat waar iedereen alles van elkaar kan zien).

WEP

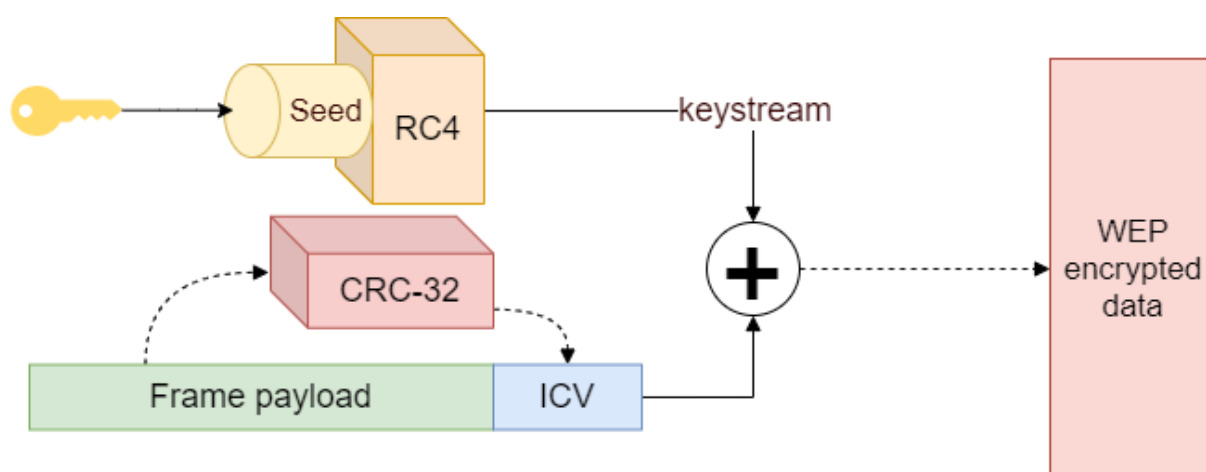
Om potentiële aanvallers ervan te weerhouden dat ze traffic kunnen sniffen (of zelf op het netwerk zetten) voorziet de 802.11 vanaf de associatie de optie om encryptie te voorzien. Dit gebeurt aan de hand van **WEP**, wat staat voor *wired equivalent privacy*. Een naam die veel beloofde maar niet zo goed was: de idee was dat WEP even veilig zou zijn als een bedraad netwerk.



WEP is optioneel en bevindt zich vlak voor frames naar de fysische laag (*PHY*) worden gestuurd die de frames “in de lucht” zal sturen. Het IEEE werkt met lagen die ongeveer overeen komen met de OSI-lagen maar met iets andere namen. In volgende figuur zie je waar WEP, optioneel, zich bevindt ten opzichte van de onderliggen en bovenliggende lagen.

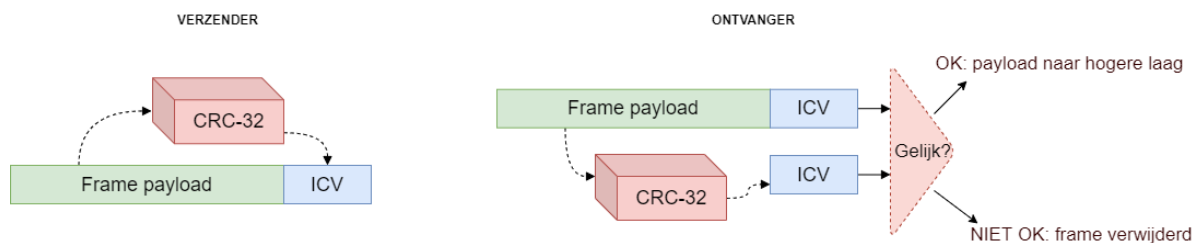


Hoe werkt WEP?



Figuur 0.2: WEP in z'n geheel

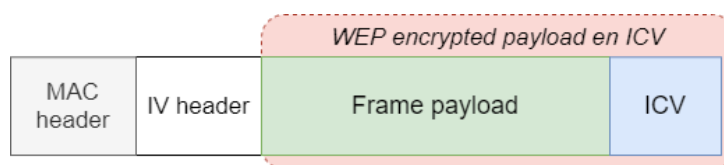
Het hart van WEP is het RC4-algoritme dat we reeds zagen in het crypto-hoofdstuk. De WEP-sleutel zal dienst doen als de seed voor de keystream generatie. Deze keystream zal op zijn beurt ge-XOR'd worden met het te encrypteren frame.



Figuur 0.3: Integrity check

Confidentiality en integriteit worden tegelijkertijd afgehandeld in WEP. Vlak voor dat het frame via de XOR-operatie wordt geëncrypteerd zal het frame eerst door een *integrity check algoritme* gestuurd worden. Deze integrity check gebeurt met behulp van CRC-32, een oude getrouwe op dit gebied. CRC-32 zal een hash genereren die de ontvanger bij ontvangst kan gebruiken om te zien of het frame werd aangepast na verzenden (bewust door een aanvaller, of door bijvoorbeeld ruis in het netwerk). Deze hash, de **Integrity Check VALUE (ICV)**, zal mee worden geëncrypteerd door RC4 voor hij verstuurd wordt. Op deze manier kunnen ordinaire aanvallers het frame niet aanpassen zonder dat ze daarmee ook de ICV ongeldig maken.

Finaal verkrijgen we dus volgende WEP-frame dat kan verstuurd worden:



Figuur 0.4: WEP frame layout



In de originele standaard zat de mogelijkheid om tot 4 WEP-sleutels in een netwerk te gebruiken. Via de *keyid* kon een client of AP dan aangeven met welk van de geïnstalleerde sleutels een frame werd geëncrypteerd.

Sleutellengte en de IV

Origineel ondersteunde WEP enkel 40-bit WEP sleutels. Ondertussen is dat opgetrokken maar toen de standaard werd geschreven besepte men al dat er sowieso een probleem met de sleutel zou zijn als die zo zou gebruikt worden: ieder frame dat met dezelfde sleutel wordt geëncrypteerd zal dezelfde keystream hebben gebruikt. Dat was natuurlijk geen optie. Om die reden werd gekozen om te werken met een **initialisatie vector (IV)** van 24-bit. Deze IV werd mee als seed aan RC4 gegeven. Door ieder frame een andere IV te kiezen zorgden men er zo voor dat ieder frame een andere keystream gebruikte

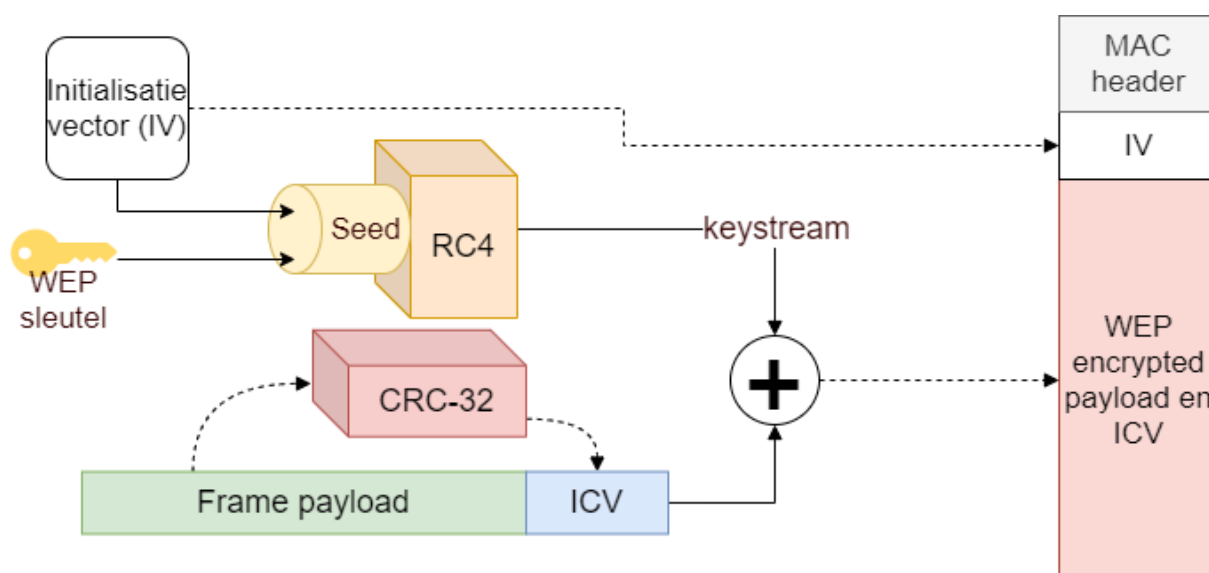
(WEP-sleutel+IV werd de nieuwe seed per frame). In de originele standaard werd echter niet beschreven hoe deze IV moest veranderen wat nefaste gevolgen zal hebben verderop.

Omdat ook de ontvanger dezelfde keystream moet kunnen genereren tijdens decryptie is het natuurlijk belangrijk dat de IV ook bij de ontvanger gekend is. De enige manier om dit op te lossen is door de IV mee in de header van het frame te plaatsen en door te sturen. Uiteraard moet deze IV als plaintext door het leven gaan.

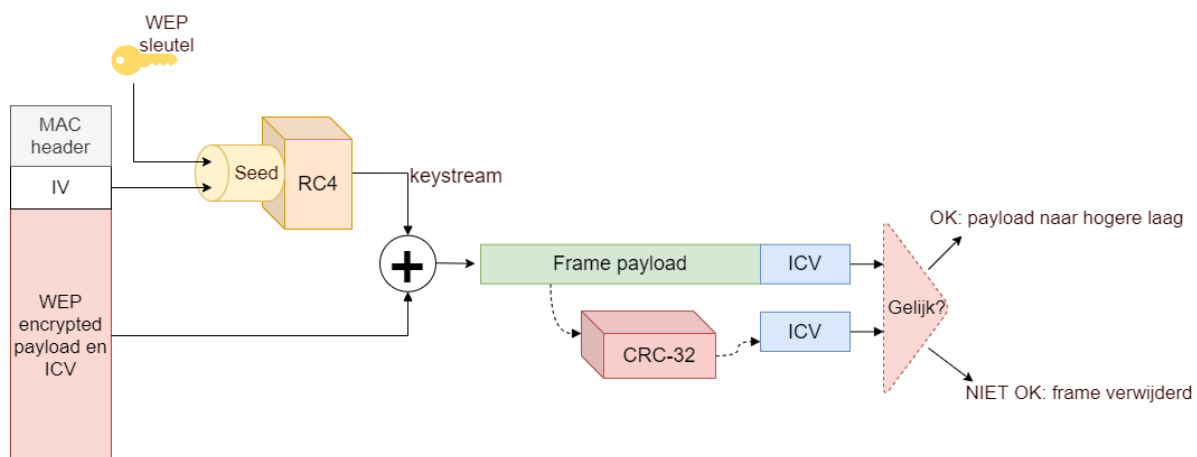


Dit concept van een sleutel verlengen met een arbitrair getal heet *salting* en zullen we in het hoofdstuk omtrent paswoorden en authenticatie verderop in de cursus nog zien terugkomen.

Finaal krijgen we dus de volgende werking, encryptie en decryptie, als volgt:



Figuur 0.5: WEP encryptie



Figuur 0.6: WEP decryptie

Hoe WEP faalde

Het leek een verbonden vat: hoe populairde wifi werd over de hele wereld, hoe meer papers er verschenen die fouten identificeerden in WEP. Al vrij snel werd duidelijk dat WEP hoegenaamd géén CIA kon aanbieden. De meeste fouten die werden gevonden kunnen gegroepeerd worden in volgende 4 zaken:

1. Het **RC4 algoritme** is helemaal niet gemaakt voor een datagramnetwerk zoals wifi.
2. De manier waarop de **IV** in de standaard is beschreven is ontoerijkend en verhoogd de kans op foute implementaties door fabrikanten.
3. **CRC-32** kan omzeilt worden en kan dus geen integriteit van frames garanderen.
4. Er is **geen sleutel-management systeem**.

Er is echter nog een vijfde fout die de voorgaande 4 als het ware nog versterkt:

5. Er is geen **replay protection**.

Hierdoor heeft de aanvaller dus vrij spel en kan hij draadloze netwerk als een soort experimenteertuin gebruiken en duizenden pakketjes te pas en te pas onpas heruitzenden. We zullen nu de eerste 4 grote problemen beschrijven. De *replay protection* behandelen we niet apart maar zullen we geregeld bij de andere problemen zien opduiken.

Probleem 1: RC4

De meeste problemen met WEP komen van een verkeerd gebruik van het RC4 algoritme. RC4 wordt in erg veel moderne beveiligingsapparaten toegepast omdat het een sterk én efficient algoritme is (het

verbruikt weinig energie omdat er geen dure vermenigvuldigingsoperaties in voorkomen). Echter, stream ciphers in het algemeen, RC4 specifiek, zijn eigenlijk geen goede keuze voor datagram netwerken waarin transmissies onbetrouwbaar zijn.

In een datagram netwerk worden pakketjes vaak opnieuw verstuurd wanneer er een fout optrad en de ontvanger voor een *retransmission* vraagt (dit gebeurt voor ongeveer 20% van de verstuurde data). Dit is volledig normaal gedrag in zowel bedrade als draadloze netwerken, echter voor een stream cipher is dit nefast. Specifiek 2 eigenschappen van stream ciphers (en dus ook RC4) zorgen voor stevige gebreken in het WEP-protocol inzake confidentiality:

1. RC4 heeft **geen random access mogelijkheden**.
2. RC4 staat **geen sleutel hergebruik** toe.

Laten we die 2 eigenschappen eens bekijken en welke cascade van problemen ze met zich meebrengen.

RC4 heeft geen random access mogelijkheden Deze eigenschap is niet zo zeer een probleem vanuit beveiligingsperspectief, maar wel vanuit performantieperspectief. RC4 had eigenlijk nooit gekozen mogen worden door het IEEE om in WEP gebruikt te worden. Het verlies van één bit van de datastroom zal ervoor zorgen dat alle bits erna met RC4 ook verloren zijn, daar we met een stream cipher werken waarbij de synchronisatie van de stroom bits tussen verzender (encryptie) en ontvanger (decryptie) gelijk moet blijven. Bij het minste dataverlies moeten beide zijden hun *RC4-motortje* resetten en opnieuw beginnen.

AES bijvoorbeeld heeft wél die random access mogelijkheid: hierdoor kan steeds herbegonnen worden aan het punt van dataverlies en niet helemaal opnieuw, wat natuurlijk veel efficiënter is (daar we werken in een datagram omgeving waar bitverlies bijna continue voorkomt).

RC4 staat geen sleutel hergebruik toe Stream ciphers hebben een tweede erg belangrijke eigenschappen: **het is uiterst onveilig om een zelfde sleutel twee keer te gebruiken!**

Stel dat je volgende 2 plaintext byte sequenties hebt: $p_1, p_2, p_3, \dots$ en $q_1, q_2, q_3, \dots$. Beide worden met dezelfde keystream $k_1, k_2, k_3, \dots$ geëncrypteerd. Dit geeft ons vervolgens volgende 2 ciphertext sequenties:

$$p_1 \oplus k_1, p_2 \oplus k_2, p_3 \oplus k_3$$

$$q_1 \oplus k_1, q_2 \oplus k_2, q_3 \oplus k_3$$

Als we nu veronderstellen dat een aanvaller deze 2 ciphertexts capteert, wat dan volgt is een grove schending van de confidentiality die RC4 zou moeten garanderen:

$$(p_i \oplus k_i) \oplus (q_i \oplus k_i) = p_i \oplus q_i$$

Of in andere woorden, wanneer we de beide ciphertexts met elkaar XOR'n krijgen we een sequentie die **niet afhankelijk is van de gebruikte sleutel!** Een stevige hoeveelheid informatie over beide plaintext wordt zo onthult. Als 1 van beide plaintexts gekend is dan volstaat een eenvoudige XOR-operatie om ook de andere plaintext te kunnen zonder dat hierbij de gebruikte sleutel moet gekend zijn. Deze fout zullen we verderop misbruiken.

Kortom, stream ciphers zijn niet veilig in een datagram omgeving indien er geen vorm van sleutelmanagement bestaat die de sleutels kan vervangen voor ze herbruikt worden. WEP probeert dit gebrek aan sleutelmanagement te omzeilen door met een IV te werken zodat er geen collisions zoals eerder beschreven kunnen optreden... maar ook dat zal een resem problemen met zich meebrengen.

Probleem 2: IV

Het IEEE had dus weet van voorgaand probleem met RC4 en introduceerde daarom het IV. Vanuit cryptografisch standpunt is dit een solide oplossing. Echter, door het gebrek aan replay protection krijgen we helaas een hoop fouten met de IV.

De IV veroorzaakt weak keys In een paper van 2001 door Scott Fluhrer, Itsik Mantin en Adi Shamir werd aangetoond dat het key scheduling algoritme (KSA) van RC4 een hiaat bevat:

1. Wanneer een deel van de gebruikte RC4 keystream gekend is dan kan een groep *RC4 weak keys* gevonden worden.
2. Wanneer deze weak keys gebruikt worden om een keystream te genereren dan zal er informatie van de gebruikte WEP sleutel in deze keystream gelekt worden.

Of anders gezegd: sommige IV's zorgen voor een sleutel-lekkage naar de keystream, wat natuurlijk nefast is voor eender welk cryptografisch cipher.

Een gevolg van die weak keys is dat, indien de eerste 2 bytes van genoeg keystreams (ongeveer 60) geweten is, men de gebruikte WEP sleutel kan achterhalen door middel van een FMS aanval (de afkorting staat voor de eerste letters van de 3 onderzoekers uit de paper).

De FMS aanval werkt indien:

1. We ongeveer 60 keystreams kunnen capteren waarvan geweten is dat ze *weak* zijn.
2. We de eerste 2 bytes van de plaintext van de bijhorende frames kennen die met deze weak keystreams zijn geencrypteerd.

Dat tweede is geen probleem, met dank aan de netwerkspecificaties: de payload van een met WEP geëncrypteerd pakket bevat de LLC header (de header van de logical link layer). Volgens de standaard (RFC 2684) moeten "IP datagram pakketten altijd zichzelf in de header identificeren via de SNAP header"

En laten de eerste 2 bytes van die header toch wel niet altijd starten met 0xAA. Kortom, quasi alle frames die over een WEP-netwerk vliegen zullen altijd met de hexadecimale waarde AA starten. Nu volstaat het om de bijhorende keystream te achterhalen aangezien we de plaintext kennen, kunnen we ook de eerste 2 bytes van de keystream kennen daar we weten dat:

$$c_i = k_i \oplus p_i$$

(waarbij p_i AA is)

En dus:

$$c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$$

We hebben zo ook deel 1 van de FMS in onze handen en kunnen nu het algoritme de gebruikte WEP-sleutel laten berekenen (de manier waarop dat gebeurt zou ons te ver brengen)/



De FMS aanval is geïmplementeerd in volgende 2 erg populaire Linux tools: Aircrack-ng & WEPCrack. Beide kunnen dus gebruikt worden om de WEP-sleutel van een draadloos netwerk te achterhalen.

IV collisions treden op Op zich is de FMS aanval al dramatisch, maar helaas stopt het hier niet. Doordat de IV maar 24 bit groot is treden er veel sneller collisions op dan intuïtief wordt verwacht. Van zodra 2 pakketjes met dezelfde IV zijn verstuurd treedt er een collision op, en die zijn erg interessant voor aanvallers. Pakketjes met dezelfde IV zijn pakketjes waarvan de payload met dezelfde keystream werd geëncrypteerd.

Een 24-bit IV kan 2^{24} oftewel 16 777 216 mogelijke waarden hebben. Een kleine berekening toont hoe snel collisions optreden:

Gegeven: een eerste generatie AP die aan een miezerige 11 Mbps werkt en continue 1.500-byte pakketjes in de lucht stuurt:

- $\frac{11 \text{ Mbps}}{(1500 \text{ bytes/pakket}) * 8 \text{ bits/byte}} = 916.67 \text{ pakketjes/seconde}$
- $\frac{16.777.216 \text{ IV's}}{916.67 \text{ pakketjes/seconde}} \approx 18302 \text{ seconden}$

Dat wil dus zeggen dat na ongeveer 5uur alle IV's opgebruikt zijn en er dan ten laatste collisions optreden.

Deze fout kunnen aanvallers op 2 manieren misbruiken: met een passieve of met een actieve aanval.

Passieve IV aanval

Een aanvaller kan passief meeluisteren (*eavesdropping*) en stilletjes alle traffic onderscheppen tot er een IV collision optreedt. Door twee pakketjes met eenzelfde IV te XOR'n verkrijgt de aanvaller een

pakket dat bestaat uit de XOR van beide plaintext'en van de gecapteerde pakketten. Als dus één van beide plaintexten gekend is, is de inhoud van het andere pakket ook gekend.

IP traffic is vaak erg voorspelbaar en bevat aardig wat redundantie (om fouten op te vangen). Hierdoor wordt het makkelijker voor een aanvaller om via cryptanalysis te achterhalen wat de inhoud, of een deel, van het pakket bevat. Een voorbeeld hiervan toonden we bij de FMS aanval waarbij steeds de LLC header gekend was van de meeste pakketten.

Omdat collisions redelijk snel optreden (vergeet niet dat het voorbeeld hierboven maar sprak over één client en één AP. Als er dus meerdere clients actief zijn in een netwerk treden collisions véél sneller op) is het voor een aanvaller dus maar een kwestie van lang genoeg te sniffen om zo een grote hoeveelheid pakketten met gelijke IV's op te vangen, waardoor de cryptanalysis ongelooflijk vereenvoudigd wordt.



In al deze voorbeelden gaan we er vanuit dat de gebruiker geen encryptie toepast op de hogere lagen waar z'n data vandaan komt. Uiteraard wordt cryptanalyse een pak moeilijker als de payload van gecapteerde pakketten geëncrypteerd blijkt te zijn.

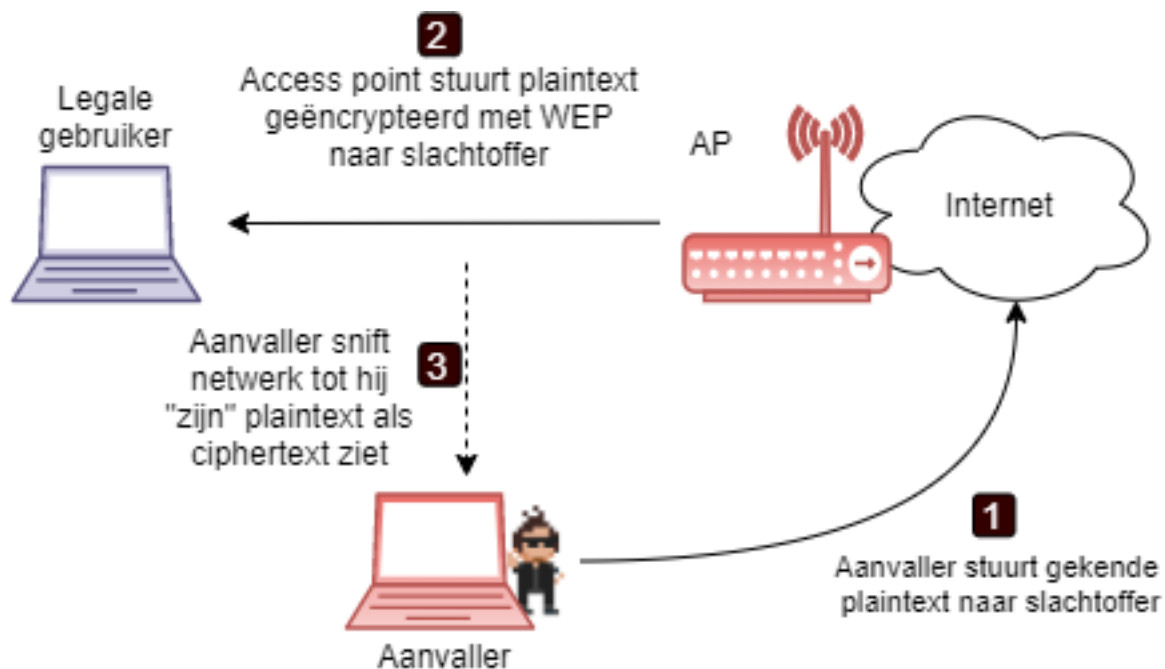
Actieve IV aanval

De passieve aanval heeft als nadeel dat we als aanvaller:

1. moeten wachten op collisions, en dus bijgevolg op traffic over het netwerk.
2. we enkel door weloverwogen gokken (cryptanalyse) kunnen proberen te weten te komen wat de originele plaintext juist is.

Beide problemen kunnen we als aanvaller echter te niet doen door een actieve rol te gaan spelen. Doordat een AP braaf alle traffic encrypteert dat het van het bedrade netwerk krijgt om naar een client te sturen, is het voor een aanvaller een kwestie van "gekende" plaintext van buitenuit naar het slachtoffer te sturen. Als volgt:

1. Een gekende plaintext boodschap (bijvoorbeeld een emailbericht of ping) wordt naar het AP gestuurd (via het internet bijvoorbeeld), dat vervolgens door de aanvaller in het oog wordt gehouden. Noot: als de aanvaller enkel het draadloze netwerk ter beschikking heeft (en niet het internet) dan zal een bitflip-aanval moeten gebruikt worden, wat we verderop zullen uitleggen.
2. De aanvaller blijft sniffen tot het de ciphertext ziet passeren waarin (vermoedelijk) z'n gestuurde plaintext zit.
3. Vervolgens kan de aanvaller een keystream te pakken krijgen door z'n plaintext te XOR'en met de gecapteerde ciphertext: $c_i = k_i \oplus p_i \Leftrightarrow k_i = c_i \oplus p_i$.



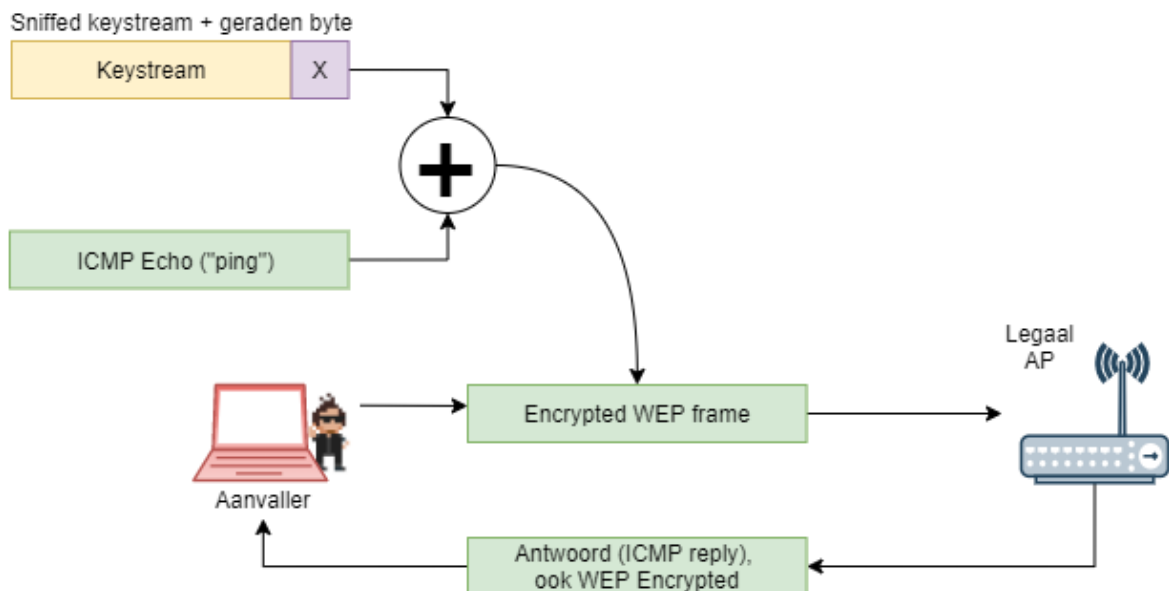
Keystreams groeien Wanneer een aanvaller met voorgaande IV collisions keystreams kan capteren kan hij in principe data op het netwerk beginnen plaatsen (aangezien het netwerk ervan uitgaat dat het gebruiken van geldige keystreams, wil zeggen dat de gebruiker geauthenticeerd is omdat hij de bijhorende WEP-sleutel heeft). De aanvaller kan nu plaintext XOR'n met deze gevonden keystream en op het netwerk zetten. Echter, hij is beperkt tot pakketten die even lang zijn als de keystream die gevangen werd. Het zou véél nuttiger zijn als de aanvaller als het ware een bibliotheekje heeft van geldige keystreams van allerlei lengtes.

Omdat er geen replay protection aanwezig is, kan de aanvaller heel eenvoudig z'n gecapteerde keystreams doen *groeien* en zo byte per byte een langere keystream genereren. Dit gaat als volgt te werk:

1. De aanvaller maakt een plaintext pakketje aan dat 1 byte langer is dan de keystream die hij al heeft. Het ping-commando (ICMP) kan je met de "-l" optie bijvoorbeeld een ping van eender welke bytesize laten genereren. Het voordeel van het ping-commando gebruiken is ook dat we exact weten wat voor response er kan verwacht worden.
2. De aanvaller plakt nu 1 byte achter de gecapteerde keystream. Hij kiest hierbij een willekeurige waarde en heeft dus 1 kans op 256 om de juiste te kiezen.
3. De aanvaller XOR'r deze keystream met het commando uit stap 1 en stuurt dit op het netwerk.
4. Indien de aanvaller in stap 2 de juiste byte gekozen heeft dan zal er een reactie op de ping volgen (daar het pakket werd gedecrypteerd door het AP en dan hoger in de OSI-stack door het netwerk werd gestuurd). Als de keystream fout is zal er geen reactie komen daar het AP het pakketje als

foutief heeft weggegooid en dus heeft genegeerd.

5. Als een verkeerde byte werd gekozen in stap 2 dan zal de gebruiker dit proces opnieuw starten en nu een andere byte-waarde kiezen. Hij zal dit blijven herhalen tot hij in stap 4 reactie krijgt en dus weet dat hij nu z'n keystream met succes heeft doen groeien met 1 byte.



We hebben bij deze aanval 1 belangrijk concept genegeert waardoor deze aanval op eerste zich niet mogelijk is: het aanpassen van een payload resulteert ook in een nieuwe CRC. Deze is echter niet gecrypteerd waardoor het niet duidelijk is hoe we dit kunnen omzeilen. Wacht nog even tot we aan de bitflip aanval komen en alles zal duidelijk worden (dat dit dus geen probleem is).

IV selectie De vierde fout met de Initialisatie Vectors is de manier waarop de selectie ervan moet gebeuren in de hardware. De 802.11 gaf enkel aan dat het IV “*geregeld moest geupdate*” worden. Dat is uiteraard te vaag en heeft ervoor gezorgd dat fabrikanten zelf moesten bepalen welke IV selectie strategie ze in hun hardware zouden implementeren. Hierdoor waren er 3 strategieën die hun weg in de verschillende apparaten vonden:

- **Vast IV:** Sommige fabrikanten hadden geen flauw benul wat het doel van de IV was vanuit cryptografisch standpunt en kozen daarom zelfs gewoon om alle pakketten steeds met het zelfde IV te versturen.
- **Willekeurig IV:** Andere fabrikanten verkozen het om hun hardware bij ieder pakketje een willekeurig IV te laten selecteren. Alhoewel dit uiteraard veel veiliger is dan een “vaste IV”-strategie, treden er toch veel sneller collisions op dan verwacht. Dit valt te verklaren door het zogenaamde

verjaardagenparadox (zie kader verder) dat verklaart waarom er reeds 50% kans op een collisions is na 4823 pakketjes. Dat wil dus zeggen dat al na enkele seconden er meestal collisions optreden.

- **Incrementele IV:** In deze strategie wordt een circulaire teller gebruikt waarbij het IV telkens met 1 wordt verhoogd wanneer een pakket moet worden verstuurd. Meestal begint deze teller op een vaste waarde. Dit zal er dan ook voor zorgen dat er een collisions optreedt van zodra een tweede apparaat zich op het netwerk begeeft en dus begint uit te zenden met het IV gelijk aan het IV van het allereerste pakketje dat het eerste apparaat gebruikte.

Kortom, een 24-bit *salt* is véél te klein in een omgeving met erg hoge data-rates zoals een draadloos netwerk. Dit probleem zou nog beperkt kunnen worden indien de originele WEP geregeld sleutels kon verversen, maar door het gebrek aan enig key management was dat dus uit den boze (want herinner je: de enige reden dat we IV's nodig hadden was omdat anders steeds dezelfde WEP-sleutel als seed werd gebruikt en dus alle keystreams gelijk zouden zijn. Door geregeld een andere sleutel te gebruiken zou onze kleine IV-lengte minder precair zijn, als we maar tijdig de sleutels verversen).



Volgende tekst uit Wikipedia legt de **verjaardagenparadox** uit: “De verjaardagenparadox is een paradox uit de kansrekening, die een resultaat toont dat tegen de verwachting ingaat. Het gaat om de vraag hoe groot de kans is dat in een groep willekeurig gekozen mensen er (minstens) twee dezelfde verjaardag hebben. Het blijkt dat, onder enkele lichte veronderstellingen, deze kans al meer dan 50% is voor een groep van maar 23 mensen. Bij 57 mensen is de kans zelfs meer dan 99%.”

Beeld je nu in dat in plaats van mensen, je honderden pakketten hebt, niet met een verjaardag maar met een eigen IV: de kans op collisions, ook al is de IV 24 bit, wordt dus 50% bij reeds een 5000 tal pakketten.

Probleem 3: CRC

WEP gebruikt een *integrity checksum field* om te voorkomen dat een pakket wordt aangepast tijdens transmissie, namelijk een CRC-32 checksum. Dit was niet zo'n wijze keuze: CRCs zijn in het algemeen vooral bedoeld om random transmissiefouten te detecteren, niet bewuste aanpassingen. Daarnaast heeft de CRC-32 ook nefaste gevolgen in combinatie met RC4 waardoor bitflipsaanvallen mogelijk zijn.

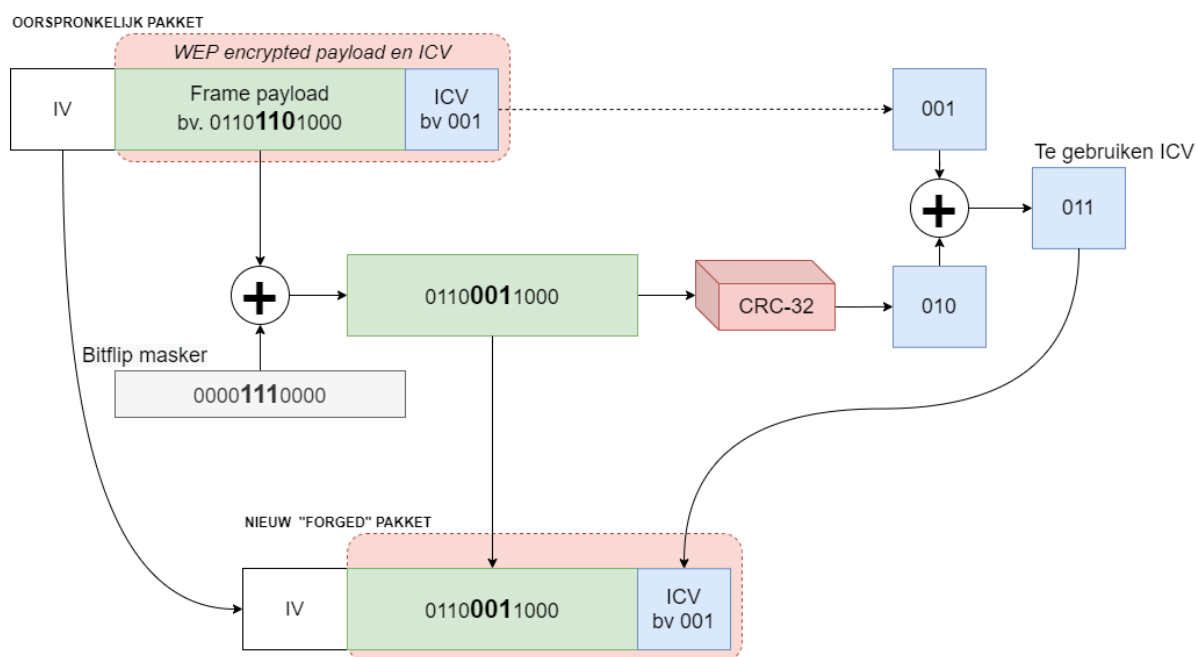
CRC-32 is een zogenaamde lineaire functie. Zonder in detail hierover in te gaan volstaat het te begrijpen dat als gevolg hiervan het volgende hebben: De CRC van 1 boodschap geXOR'd met de CRC van een andere boodschap is hetzelfde als de CRC nemen nadat beide boodschappen samen werden geXOR'd. Of beter gezegd:

$$CRC(\text{boodschap}_1) \oplus CRC(\text{message}_2) = CRC(\text{message}_1 \oplus \text{message}_2)$$

Door deze eigenschap kunnen we gecontroleerd aanpassingen aan pakketten doen, zonder daarmee de CRC te breken. Welgekome, bitflip aanval.

Bitflip aanval Om een bitflip aanval te doen heeft de aanvaller enkel 1 geldig WEP frame nodig. Hij hoeft zelfs niet te weten wat de inhoud ervan is, zolang het maar een geldig frame is.

Vervolgens maakt de aanvaller een bitflip masker: dit bestaat uit een reeks 0's, met 1 of 2 bits op 1. Dit zijn de bits die geflipt zullen worden in de volgende stap: de XOR nemen het bitflip masker met het payload gedeelte van het oorspronkelijke pakket. Welke bits geflipt worden doet er niet toe, integendeel: we willen net een geldig WEP-frame maken mét een foute data, zoals we zo meteen zullen zien. Deze nieuwe payload willen we nu in een geldig frame plaatsen, en dus hebben we een geldige ICV nodig. We doen dit door de ICV van de nieuwe payload te berekenen en deze te XOR'n met de, geëncrypteerde originele ICV. Het resultaat is een geldig, geëncrypteerde ICV voor de nieuwe payload. Bijgevolg hebben we een geldig frame kunnen maken dat door access points op het netwerk aanvaard zullen worden.

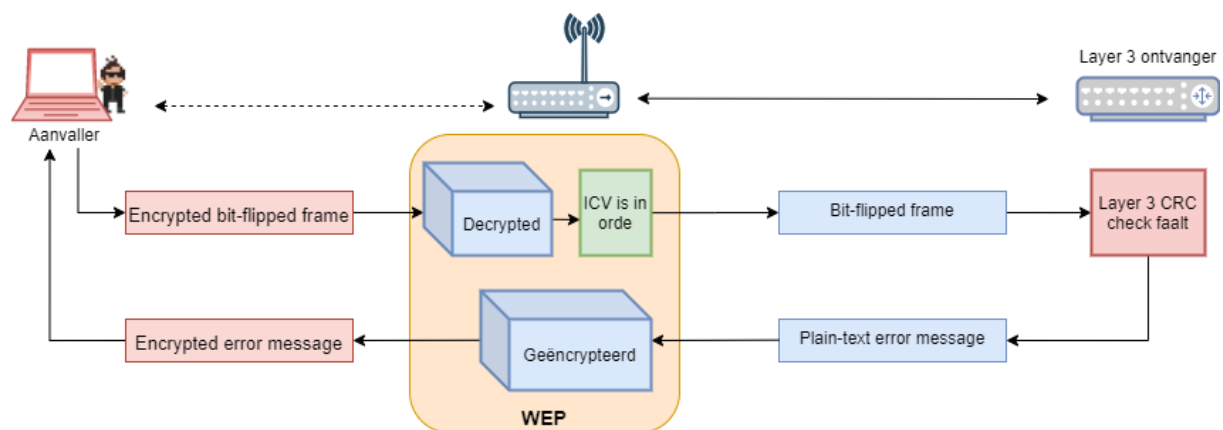


The attacker can now send a forged frame that is considered genuine by the receiver. The receiver (AP) dutifully decapsulates the bit flipped data and the LLC discovers that the data is 'gibberish'. Yet, since the ICV was valid, the receiver simply thinks some higher layer CRC error has occurred and thus sends an encrypted error-message to the attacker.

Wanneer een AP dergelijk pakket krijgt zal het dit pakket braaf naar de volgende laag sturen, ook al bevat de inhoud *rommel*, dankzij de geldige ICV. Op de volgende laag komt nu een pakket aan dat duidelijk stuk is, en deze laag zal bijgevolg een foutboodschap genereren en terugsturen. De

aanvaller krijgt deze boodschap in een WEP-frame aan. En alhoewel dit pakket geencrypteerd is, weet de aanvaller de inhoud van dit pakket (daar hij heeft opgezocht wat de foutboodschap zal bevatten op de volgende laag volgens de standaard van die laag) en kan dus een geldige keystream te pakken krijgen:

$$c = k \oplus p \Leftrightarrow k = c \oplus p$$



Dankzij de bitflip aanval kan de aanvaller dus zonder problemen keystreams groeien zoals eerder uitgelegd.

Probleem 4: Sleutelmanagement

Het moge duidelijk zijn: constant dezelfde WEP-sleutel gebruiken, op meerdere apparaten, gedurende meerdere dagen, is vragen om problemen. Werknemers die ontslagen worden kunnen een potentiële sleutel-lekkage veroorzaken met alle gevolgen van dien. Administrators moeten manueel nieuwe sleutels in voeren bij de werkgevers, etc. Kortom, 2 belangrijke oorzaken zorgen voor een nog lagere beveiligingsgraad van WEP dan er al was ten gevolge van de voorbije 3 problemen (IV, RC4, CRC):

1. Er is **geen geautomatiseerd sleutel verversmechanisme**: idealiter worden de sleutels binnen 1 sessie vervangen voor dat de pool van mogelijke IV's is opgebraakt.
2. Er is **geen gecentraliseerd sleutelmanagementsysteem**.

Hoe WEP oplossen?



Bart Preneel van de KUL/Cosic, één van België's meest vooraanstaande crypto-experts, zei ooit over WEP dat het het perfecte schoolvoorbeeld is van wat er allemaal kan fout lopen wanneer je beslist om zelf een nieuw crypto-algoritme te ontwikkelen.

Rond 2001, nog geen 2 jaar nadat de eerste wifi-standaard de wereld “het wonder van draadloze netwerken” bracht, werd duidelijk dat er dringend een oplossing moest verschijnen voor de vele veiligheidsproblemen. Wifi was alomtegenwoordig, zowel bij particulieren als in bedrijven, en dus moest een oplossing gezocht worden voor al die duizenden bestaande apparaten reeds in huizen en gebouwen. Het IEEE kon moeilijk een nieuwe standaard uitschrijven die alle bestaande gebruikers in de kou zette. Er werd daarom besloten om 2 pistes uit te werken:

1. WPA1: een (tijdelijke) oplossing voor bestaande apparaten, rekening houdend met enkele duidelijke beperkingen.
2. WPA2: de “ultieme oplossing” die een volledig nieuwe suite aan veiligheidsprotocollen zou bevatten voor toekomstige Wifi-producten, maar die niet compatibel zou zijn met bestaande apparatuur.



WPA staat voor Wi-Fi protected standard.

De beide oplossingen zouden ook de IEEE 802.1X standaard omarmen. Deze standaard zou sleutelmanagement en gebruikersauthenticatie voorzien in combinatie met de nieuwe encryptie en integriteitsoplossingen van WPA1 of WPA2.

Omdat sleutelmanagement én de bijhorende 802.1X infrastructuur redelijk overkill kon zijn voor huistuin- en keukegebruikers van wifi, besloot het IEEE om 2 versies van zowel WPA1 en WPA2 te maken:

- **Personal:** de thuisversie voor gewone gebruikers waarbij met een gedeelde sleutel of passphrase wordt gewerkt (een zogenaamde **PSK** oftewel *pre-shared key*). Hierbij wordt géén 802.1X gebruikt.
- **Enterprise:** de versie voor (grote) bedrijven waar sleutelmanagement belangrijk is en dus 802.1X wordt gebruikt

We vatten hier alvast de belangrijkste verschillen tussen WEP, WPA1 en WPA2 samen.

Standaard	Encryptie	Integrity	Authenticatie & sleutelmanagement
WEP	RC4	CRC-32	Geen
WPA1-Personal	TKIP	Michael	pre-shared key
WPA1-Enterprise	TKIP	Michael	802.1X
WPA2-Personal	CCMP	CBC-MAC	pre-shared key
WPA2-Enterprise	CCMP	CBC-MAC	802.1X



Ondertussen bestaat er ook WPA3, die we op het einde van dit hoofdstuk zullen bespreken. WPA1 en WPA2 zijn echter, historisch gezien, gerelateerd aan WEP en daarom bekijken we deze beiden samen in functie van WEP.

WPA1

Voor de tijdelijke oplossing, WPA1n werden volgende beperkingen geïdentificeerd:

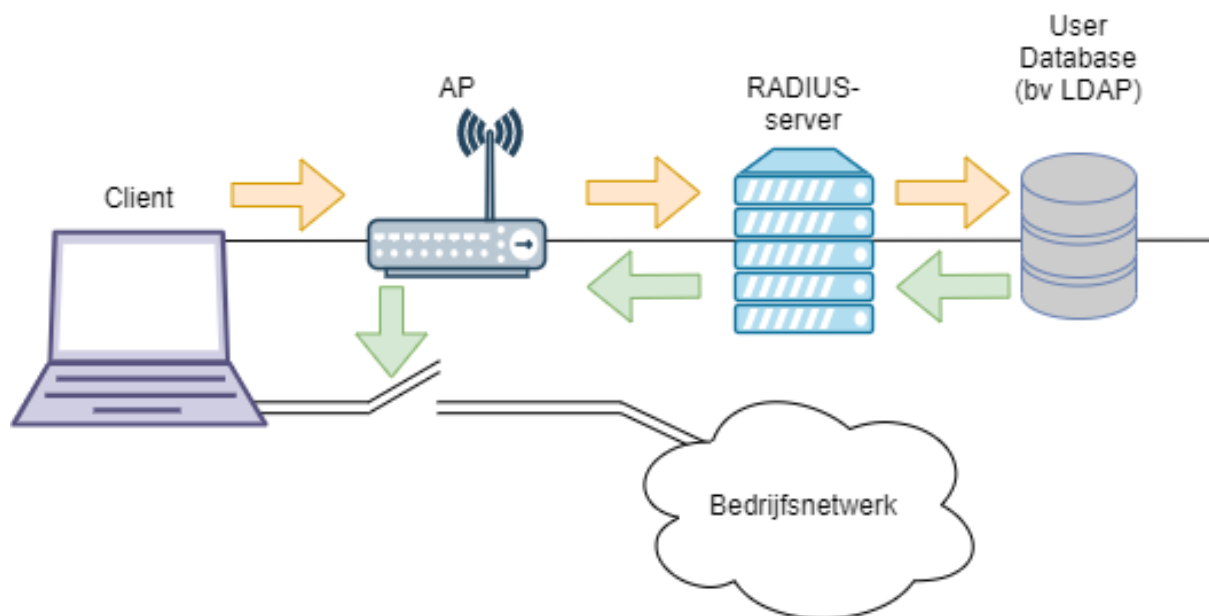
- Zoals verteld, miljoenen WEP-gebaseerde apparaten waren reeds in gebruik. Deze apparaten zouden met behulp van een firmware upgrade gepatcht moeten kunnen worden naar de tussentijdse oplossing.
- De meeste AP's werkten met processoren die reeds quasi volcontinue tegen hun maximum capaciteit werkten. De extra algorithmes die het AP moesten draaien mocht dus maar een beperkte overhead creëren.
- Delen van de RC4 encryptie zijn *hardwired* voor een deel in de hardware van de AP. Hierdoor kunnen bepaalde delen van WEP onmogelijk 'omzeilt' worden en hangen we dus inherent vast aan WEP.

802.1X

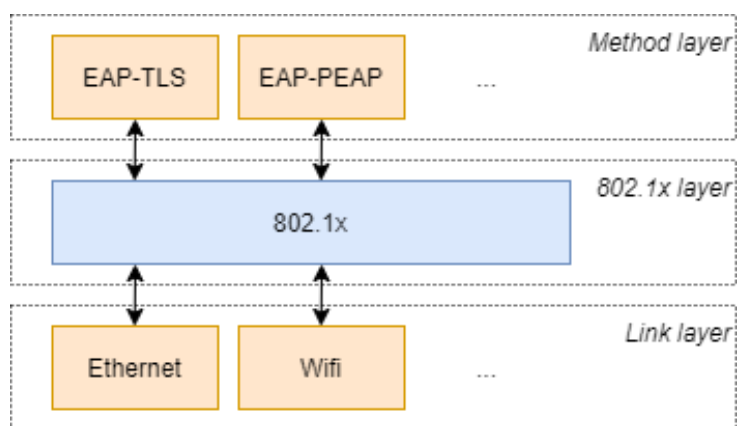
De IEEE 802.1X standaard is, ondertussen, een veelgebruikte standaard in veel draadloze én bedrade netwerken. Het voorziet volgende zaken aan WPA1 en WPA2 netwerken die in *Enterprise-mode* werken:

- (Mutual) **authentication**.
- Een **centraal user management** systeem.
- Een **veilige manier om geheime sleutels uit te wisselen**.

We gaan de volledige werking van de 802.1X standaard hier niet uit de doeken doen, dat zou ons te ver brengen. Het is echter nuttig om te begrijpen dat deze standaard werd gekozen omdat hij ervoor zorgt dat de AP niet meer zelf de authenticatie moeten doen, maar dat ze gebruik maken van de bestaande authenticatie-infrastructuur van het bedrijf. De APs zullen gewoon als een doorgeefluik aan de start tussen gebruiker en de authenticatie-server optreden en de boodschappen tussen beiden uitwisselen. Enkel wanneer het AP toestemming krijgt van de authenticatieserver (meestal een RADIUS server) zal het AP de eindgebruiker toegang tot het draadloze netwerk verschaffen.



802.1X zelf beschrijft niet hoe de authenticatie moet plaatsvinden: het is geen algoritme. Integendeel: het is een **framework** waar binnen andere algorithmen en standaarden, op maat van het bedrijf, kunnen ingeplugd worden. Hierdoor ontstaat een flexibel concept dat bedrijven (of diehard eindgebruikers) niet verplicht om een bepaalde manier van authenticatie (en bijhorende soft-en hardware) te omarmen. 802.1X zorgt voor de vertaling van de authenticatie- boodschappen tussen enerzijds het netwerkprotocol (bv Ethernet, Wifi, maar ook Token Ring, etc.) en de *methode laag*. De methode-laag bevat het te gebruiken authenticatie-protocol en dient **EAP**-compatibel zijn.

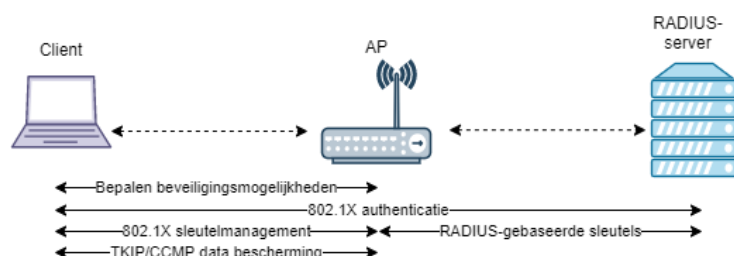


EAP oftewel *Extensible Authentication Protocol* is, zoals de naam doet vermoeden, een uitbreidbaar protocol van te gebruiken authenticatie-methoden. Afhankelijk van de keuze van het bedrijf kan voor een bepaald EAP-protocol gekozen worden, het enige is gebruiksvriendelijker en of veiliger dan het andere. Uiteraard dienen zowel de client als de netwerkinfrastructuur compatibel te zijn met de gekozen EAP-methoden van het netwerk. Koop je dus een AP dat WPA2-Enterprise compatibel is, moet

je nog steeds controleren of het AP compatibel is met de gekozen EAP-methode van het bedrijf.

De meest gebruikte EAP-methoden zijn:

- **EAP-TLS**: gebruikt een TLS-tunnel om op een beveiligde manier te communiceren (we zagen TLS ook reeds aan het einde van crypto waar het gebruikt werd om https-traffic te beveiligen). Hierbij gebeurt een certificaat-gebaseerde authenticatie.
- **EAP-TTLS** (*Tunneled TLS*): Omdat niet alle eindgebruikers zich kunnen authenticeren aan de hand van een certificaat, voorziet PEAP authenticatie met behulp van een username/paswoord login, waarbij wel nog steeds een TLS tunnel wordt gebruikt voor veilige communicatie. Ter info: EAP-TTLS is quasi hetzelfde als *Protected EAP* (PEAP) een ander EAP-protocol dat je soms zal zien passeren.



De voorgaande figuur toont de typische uitwisseling van boodschappen die plaatsvinden wanneer een client voor het eerst verbinding wil maken op een WPA1 of WPA2 Enterprise netwerk:

1. Client en access zoeken samen welke EAP-methoden zij beide kunnen gebruiken om de authenticatie te starten. Enkel indien ze samen tot 1 keuze kunnen komen wordt over gegaan naar stap 2.
2. Vanaf nu zal het AP enkel dienst doen als doorgeefluik tussen de client en de authenticatie-server.
3. Als de server genoeg bewijs heeft om de client te authenticeren zal de server de nodige sleutels genereren en deze aan het AP geven.
4. Het AP zal vanaf nu instaan voor het verdere sleutelbeheer en wanneer nodig de sleutels verversen tijdens de sessie.
5. Wanneer de sleutels tussen client en AP zijn verwerkt krijgt de client toegang tot het netwerk.

In wifi-netwerken met 802.1X worden 2 sets van sleutels aangemaakt:

- Sessie sleutels, ook wel "*pairwise keys*" genoemd: deze is uniek per client en is enkel gekend door die client en het AP.
- Groepsleutels, ook wel "*group keys*" genoemd: deze wordt tussen alle clienten van hetzelfde AP gedeeld en worden gebruikt voor multi-cast traffic.

Indien het *dynamic key exchange* is geconfigureerd dan zal de authenticatie de sessiesleutels eenmalig aanmaken en doorsturen. Daarna zullen specifieke encryptie-sleutels gegenereerd worden bij de

client en AP gebaseerd op deze sessiesleutel. De client (en AP) kan dan zelf, automatisch, op gepaste momenten en nieuwe encryptie-sleutel genereren.

TKIP

Wanneer de nodige sleutels zijn uitgewisseld is het tijd om data te encrypteren. Dit gebeurt door middel van het **Temporal Key Integrity Protocol** (TKIP), wat een suite van algorithms is dat als een wrapper rond WEP wordt gelegd om zo encryptie aan te bieden en daarbij de fouten van WEP minimaliseert.

TKIP omhult WEP met volgende zaken:

1. Een nieuwe **integrity check genaamd Michael** dat een *message integrity code* (MIC) genereert die wél bestand is tegen bitflip aanvallen.
2. Een nieuwe manier van **IV selectie** die replay aanvallen voorkomt.
3. Een **per-pakket sleutel mixing** algoritme dat het probleem met *weak keys* in RC4 oplost.
4. Een *re-keying* mechanisme dat ongeveer elke 10000 pakketjes een nieuwe sleutel doet genereren.

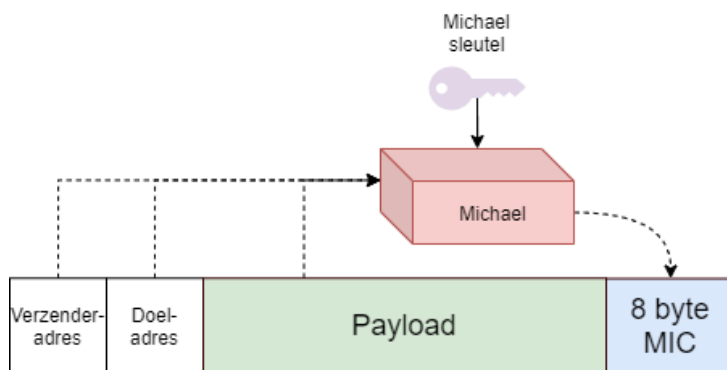
Michael the Mic

Om bewuste aanpassingen aan de payload van een frame te detecteren gebruikt WPA1 een *message integrity check* (**MIC**). Dit is een cryptografisch sterker concept dan de originele CRC checks en wordt verzorgd door het “Michael” algoritme.



In de literatuur wordt meestal gesproken over “Message authentication codes” of MAC’s. Echter, in de IEEE 802 standaarden wordt MAC reeds gebruikt voor *message access control* en werd dus gekozen voor MIC.

“Michael” berekent de MIC van een payload maar gebruikt hierbij ook de authenticatie sleutel, het adres van de verzender én ontvanger. Hierdoor wordt het voor een aanvaller veel moeilijker om een dergelijke MIC na te bootsen, laat staan te *replayen*.



Wanneer TKIP een 2 foute MICs na elkaar detecteert gaat het er van uit dat er een aanval bezig is. Volgende stappen worden dan ogenblikkelijk uitgevoerd door de client:

1. Alle huidige sleutels worden verwijderd.
2. De client verbreekt de verbinding.
3. Er wordt 1 minuut gewacht voor een nieuwe verbinding (*association*) wordt opgezet.

IV selectie verbetering

Om te voorkomen dat fabrikanten weer naïeve oplossingen voor de IV-selectie implementeerden, legde WPA1 nu de regels op. Een ontvangen pakket zal pas aanvaard worden indien het IV van het pakket op het IV van het vorige pakket volgt. Uiteraard zit er een kleine marge om hertransmissies toe te staan, maar een pakket met IV 12933 zal nooit aanvaard worden als het AP vlak ervoor een pakket met IV 4 heeft aangekregen.

Daarnaast wordt ook het IV gevoelig vergroot. TKIP hanteert namelijk een 48-bit IV genaamd de *TKIP sequence counter* (TSC). Deze wordt opgebouwd door de eerste en tweede byte van de originele WEP IV te combineren met 4 bytes van een speciaal gegenereerde *extended IV*.

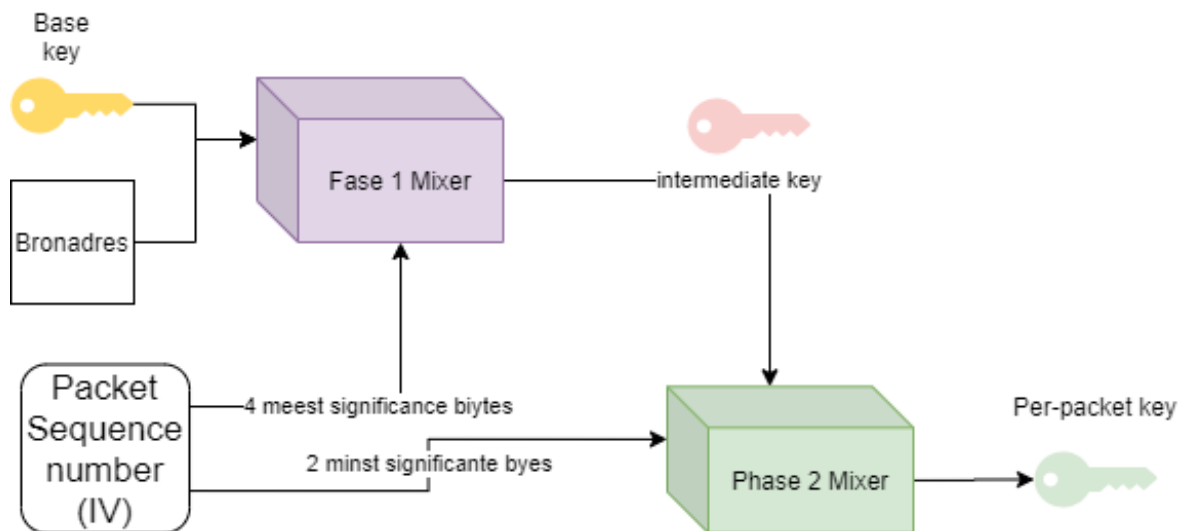
Key mixing en re-keying

Om weak keys te voorkomen gebruikt een TKIP een *key mixing function* dat zal resulteren in een **temporal of per-pakket sleutel** die dienst zal doen als vervanger voor de WEP-sleutel. Deze sleutel zal geregeld ververs worden (vandaar *temporal* oftewel tijdelijk) en heeft dus een beperkte levensduur voor actieve aanvallen.

Het mixen van de sleutel gebeurt in twee fases, waarbij iedere fase een specifieke zwakte van WEP indijkt:

- Fase 1: zorgt ervoor dat alle clients een eigen sleutel hebben doordat het verzender adres (TA, *transmitter address*) wordt toegevoegd aan de basis sleutel.

- Fase 2: zorgt voor een ‘per-pakket’ sleutel waarbij kennis van de IV niet meer door de aanvallers kan misbruikt worden.



Fase 1 mix

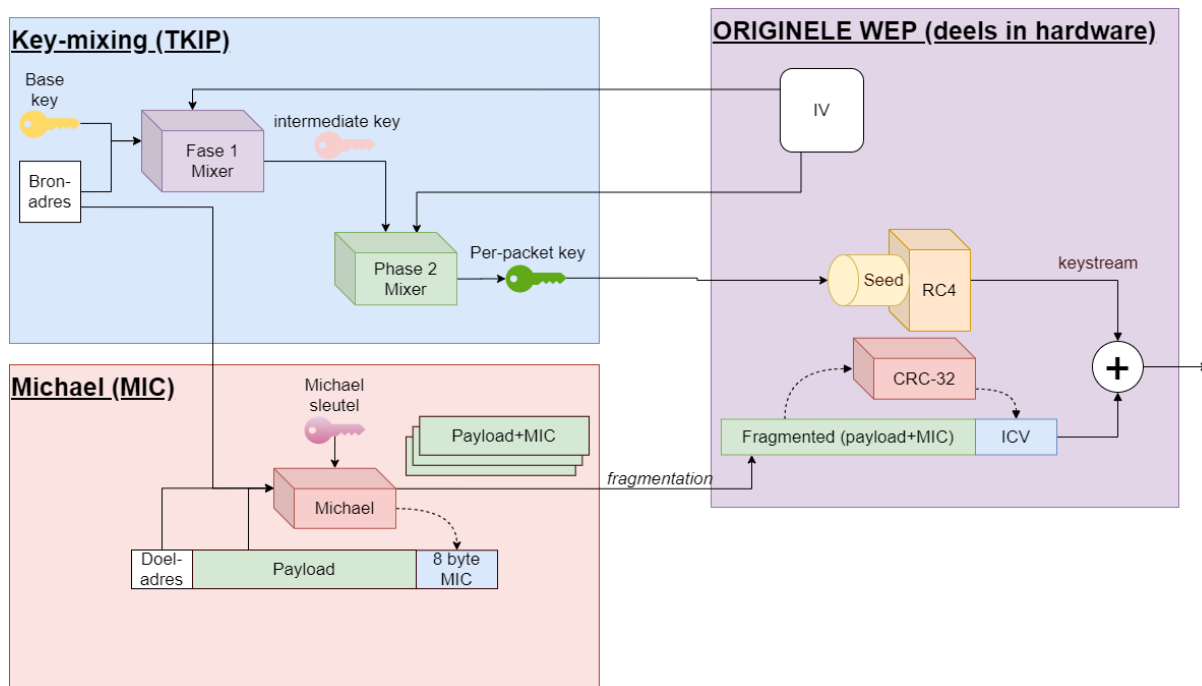
Het MAC address van de client wordt ge-XOR'd met de basissleutel (zijnde ofwel de PSK sleutel in Personal modus of de *temporal* sleutel verkregen van 802.1X tijdens de authenticatie). Dit resultaat wordt door een S-box (substitutie) gestuurd, resulterend in een tussentijdse (*intermediate*) sleutel.

Phase 2 mixing

In deze fase wordt de TSC (de pakket-teller, uitgelegd in de “IV selectie verbetering” sectie) geëncrypteerd samen met de tussentijdse sleutel. De encryptie gebeurt door middel van een kleine Feistel-structuur (zie hoofdstuk crypto) en resulteert in een 128-bit per-pakket sleutel. Vervolgens wordt deze sleutel, samen met delen van de TSC als “WEP-sleutel” en IV aan het originele WEP-gedeelte aangeboden. Hierbij wordt ervoor gezorgd dat er geen RC4 weak keys meer mogelijk zijn omdat die IV worden weggefilterd voor ze aan de hardware worden aangeboden.

WPA1: alle blokjes samen

Finaal kunnen we vervolgens WPA1 visualiseren, waarbij duidelijk is dat we vooral een wrapper rond WEP hebben verkregen, maar dat WEP nog steeds het hart van het systeem is.



In 2009 verschenen er al enkele exploits die WPA1-Personal misbruikten waardoor aanvallers de WPA passphrase (de PSK) konden achterhalen door de handshake aan de start van een sessie te capteren.



Bekijk coWPAtty en Aircrack om WPA1-Personal te *hacken*.

WPA2

De finale oplossing voor het WEP-veiligheidsprobleem werd gegoten in WPA2 waarvan AES het hart vormt. Toen de standaard werd ontwikkeld was AES zonet verschenen en het bleek de ideale toepassing voor WPA2. Hierbij werden 2 onderdelen van AES gebruikt namelijk:

1. AES met *counter mode* voor de encryptie.
2. AES in CBC-MAC mode voor integriteit en authentication.

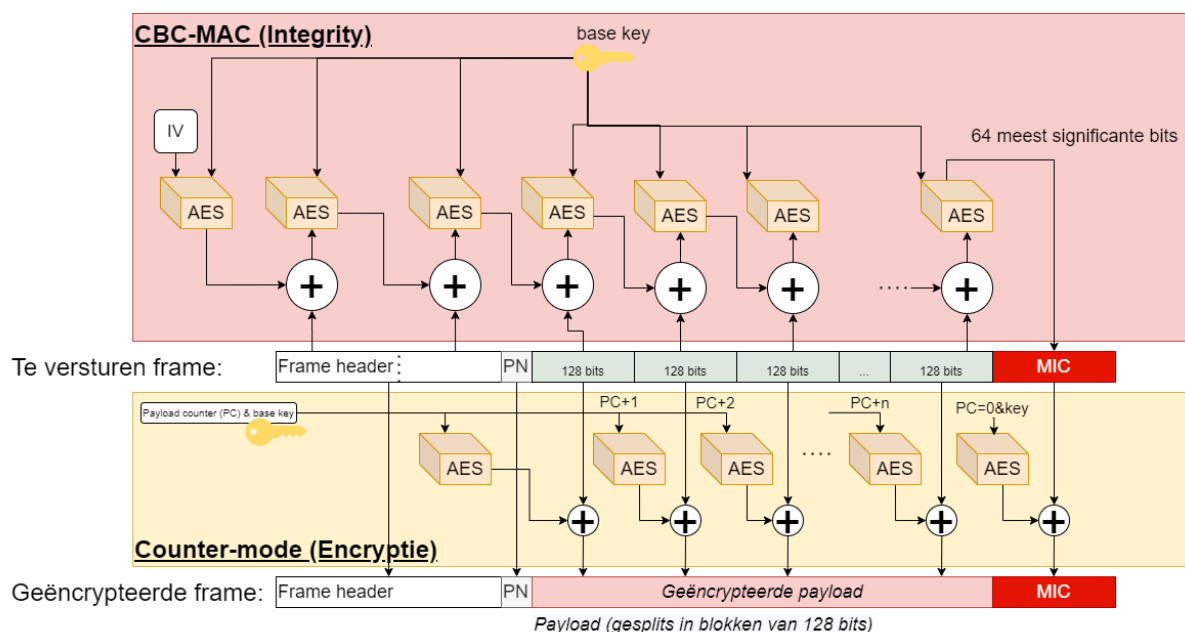
Voor de sleuteldistributie en management werd, zoals vermeld, gebruik gemaakt van de 802.1X standaard.



Toen WPA2 uitkwam ontdekte men dat toch aardig wat bestaande hardware kon gepatch'd worden om ook WPA2-compatibel te zijn. Dit was een opsteker voor velen, daar de originele premise van WPA2 net was dat ze geen rekening zouden houden met de bestaande hardware die reeds op de markt was.

Encryptie en MIC creatie

CCMP gebruikt, net als TKIP, een 48-bit IV die *packet number* (PN) werd genoemd. Deze PN wordt, samen met andere informatie, gebruikt om de AES encryptie van een seed te voorzien. Hierbij wordt het frame (en de header) in blokken van 128 bit verdeeld en zo blok per blok verwerkt. De encryptie gebeurt parallel met het berekenen van de MIC (de ICV in WEP) die finaal achteraan als een laatste blok ook mee wordt geëncrypteerd. Ook nu wordt met een tijdelijke sleutel gewerkt die gebaseerd is op de hoofdsleutel verkregen via 802.11X (enterprise mode) of de passphrase (in personal mode)



In de figuur zie je duidelijk de elegantie van het systeem en hoe de integriteit in parallel wordt berekend met de encryptie. De MIC wordt berekend met behulp van CBC-MAC, wat staat voor **cipherblock chaining - message authentication code**, een veelzeggende naam. Zoals we uit het hoofdstuk crypto weten, zal er bij CBC een keten van encrypties starten, waarbij de encryptie van het huidige blok geXOR'd wordt met het resultaat van de encryptie van het vorige blok. Het finale geëncrypteerde cipherblock zal daarbij dienstdoen als MIC. Bij de minste bitfout ergens in de payload of header zal een totaal andere MIC gegenereerd worden.

Om de payload te encrypteren (*merk op dat de header niet geëncrypteerd wordt, het wordt enkel mee betrokken om de MIC te berekenen*) wordt de **counter mode** van AES gebruikt. Hierbij worden de blokken onafhankelijk van de andere blokken geëncrypteerd en zal een *payload counter* (PC) telkens met 1 verhoogd worden en als extra seed voor de AES-motor van het huidige blok dienen (samen met de te gebruiken sleutel).

Helaas, aan alles komt een einde

Tot 2017 ging alles goed. De AES standaard was al jaren een robuuste standaard gebleken en werd op vele plekken nog steeds gebruikt. En dit zou ook bij Wifi zou zijn geweest, waren het niet dat in mei 2017 een Belgische onderzoeker, Mathy Vanhoef, de bevindingen van z'n onderzoek publiceerde. Hij had helaas een belangrijke fout gevonden in de WPA2 standaard. Deze had niets te maken met AES - dat blijft een stevige standaard zijn- maar wel de manier waarop een bepaalde uitwisseling van berichten tijdens de initiële handshake tijdens de authenticatie plaatsvinden tijdens de *sleutel reinstallatie fase*. Deze aanvallen worden beschreven én gedemonstreerd op krackattacks.com en verplichtten de IEEE om te beginnen werken aan een opvolger voor WPA2.

WPA 3 (“Wifi 6”)

In 2018 kwam de Wifi Alliance uit met de opvolger van WPA2, de titel, je raadt het nooit, was uiteraard WPA3, maar werd ook wel Wifi 6 genoemd. De standaard gaan we hier niet zo gedetailleerd bespreken, het volstaat te begrijpen dat ze

- op veiligheidsniveau state-of-the-art was.
- rekening hield met de noden van 21e eeuwste draadloze netwerken (denk maar aan Internet-of-things apparaten, beveiligde publieke hotspots, etc.).



De Wifi Alliance is in het leven geroepen als een organisatie die ervoor zorgde dat producten wel officieel konden claimen dat hun producten conform een IEEE standaard waren. Enkel wanneer hun producten de nodige tests van de Wifi Alliance aflegden kon het product het label van “Wifi alliance compatibel” product dragen.

Ook in WPA3 werden 2 modes voorzien: een personal en een enterprise mode. Enkele van de interessantste verbeteringen zijn:

- *Simultaneous Authentication of Equals (SAE)*: een nieuw cryptografisch concept waarbij in de personal mode authenticatie veel veiliger kan plaatsvinden dan voorgeen.
- Resistant tegen offline dictionary attacks: iets waar zowel WPA1 en WPA2 last van hadden voor aanvallers met geduld.
- *Forward secrecy*: zelfs als de aanvaller de wifi-sleutel van oude gecapteerde pakketten vindt zal hij deze toch niet kunnen decrypteren. Het aloude “safe now, decrypt later” is dus niet van toepassing op WPA3.
- *Wifi easy connect*: een gebruiksvriendelijke manier om internet-of-things apparaten met het netwerk te verbinden.

- *Wifi enhanced open*: publieke hotspots blijven publiek, maar iedere client heeft z'n eigen veilige kanaal met het AP. Gedaan zijn de dagen van je in de Starbucks zetten om zo prive-traffic van omstaanders te sniffen.
- *Geauthenticeerde encryptie* gebruik maken van “256-bit Galois/Counter Mode Protocol (GCMP-256)” een cryptocipher dat we hier niet uit de doeken gaan doen.
- Gebruikt de meest secure authenticatie en sleuteldistributie methoden mogelijk binnen 802.1X (HMAC, HMAC-SHA384 en ECDH)

Authenticatie

Bewijzen wie je bent om toegang te krijgen tot een website of applicatie heet **authenticatie**. Vervolgens, afhankelijk van wie je bent, zal je bepaalde rechten toegewezen krijgen die bepalen wat je wel en niet kunt doen op de website of applicatie, dit heet **authorizatie**. In dit hoofdstuk gaan we ons toespitsen op het eerste deel van dit proces: de authenticatie. Hierbij gaan we vooral kijken hoe we op een veilige manier moeten omgaan als web-of applicatiebeheerders met de login-informatie van gebruikers.

We weten al dat je geheime sleutel een belangrijk onderdeel is in heel veel aspecten van cybersecurity. Het **weten van de geheime sleutel** is een eerste vorm van authenticatie (maar uiteraard niet de beste). Je logingegevens die je gebruikt om toegang te krijgen tot een website bestaan in primaire vorm meestal uit een combinatie van gebruikersnaam en paswoord. Het paswoord in dit verhaal is kortom gewoon een ander woord voor je geheime sleutel. Als beheerder is het dan ook essentieel dat we uitermate veilig omgaan met de paswoorden van gebruiker. We zouden niet willen dat alle login gegevens van onze gebruikers in verkeerde handen vallen.

Veilige paswoorden

Ongeacht de veiligheden die we inbouwen als cyberboswachter, veel blijft afhangen van de manier waarop eindgebruikers omgaan met hun paswoorden. Volgende regels worden continue **niét** gehanteerd, met alle gevolgen van dien:

- Hergebruik nooit een wachtwoord. In principe heb je één wachtwoord *pér* service (applicatie, website, etc.).
- Gebruik geen wachtwoorden die in *dictionaries* staan. Maar overweeg volledig random gegenereerde wachtwoorden.
- Zorg ervoor dat je wachtwoorden lang genoeg zijn (minimum 12 tekens).
- Zorg ervoor dat je wachtwoorden steeds een combinatie van cijfers, letters (grote én kleine) en leestekens zijn.

Trouwens, herinner je de McCumber kubus waarin we benadrukten dat technologie maar één aspect is om C.I.A. toe te passen op je data in z'n 3 primaire vormen? Het zal je niet verbazen dat cybercriminelen

niet altijd gaan proberen databanken aan te vallen om paswoorden van gebruikers te pakken te krijgen. Als zij een specifiek doelwit hebben dan gaan we vaak op andere manieren te werk:

- **Password spraying:** hierbij gaat de hacker een (beperkte) lijst van veelgebruikte paswoorden testen op een grote groep *useraccounts* van een bepaalde website, in de hoop een *hit* te hebben (*"spray and pray"*).
- **(Spear) phishing:** bij phishing hanteert de aanvaller de goedgelovigheid of onoplettendheid van de gebruiker om een ogenschijnlijk betrouwbare mail of bericht te sturen met daarin een link naar een pagina die malware installeert of een fake login scherm toont. Bij spear phishing gebruikt de aanvaller geen massmail, maar gaat hij juist gericht één specifiek doelwit een op maat gemaakte mail of bericht sturen. Spear phishing is heden ten dage een van dé **social engineeringen** aanvallen bij uitstek.
- **Key loggers:** als de aanvaller toegang heeft tot de computer (wat uiteraard van over het netwerk kan) dan kan hij een permanente key logger installeren die continue alle toetsaanslagen op het systeem opneemt. Nadien kan de aanvaller deze logs dan analyseren in de hoop zo ook het paswoord of andere gevoelige informatie terug te vinden.

Hoe paswoorden opslaan

Hoe moet je nu als cyberboswachter de login-informatie van je gebruikers bewaren? We gaan een soort *bottom-up* aanpak hanteren, waarbij we beginnen met de meest naïeve oplossing en telkens verbeteringen zullen aanbrengen.

Paswoorden als plaintext bewaren

In het prille begin van het internet gebeurde dit quasi overal: de login-databank had 2 kolommen:

1. gebruikersnaam
2. gebruikerspaswoord

De paswoorden in kolom 2 stonden er zoals ze waren. Als een gebruiker wilde inloggen op dit soort websites dan moest hij z'n paswoord verzenden en dan ging de *backend* controleren of het ingezonden paswoord overeen kwam met het paswoord in de database. Het spreekt voor zich dat dit soort databanken van gigantische waarden zijn voor aanvallers: van zodra ze de databank hebben te pakken hebben ze alle paswoorden van alle gebruikers! Profit!

Paswoorden mogen nooit in leesbare vorm in een databank staan! Wanneer dit wel zo is dan kan je beter ogenblikkelijk je account bij die service deleten. Want alhoewel deze aanpak al lang bestaat

en al bijna even lang van geweten is dat deze erg onveilig is, toch zijn er nog steeds ontelbare websites en applicaties die hieraan zondigen.

Een goede manier om te weten of een service op deze manier werkt is gebruik maken van de “*Ik ben m’n wachtwoord vergeten*”-knop. Als je deze knop gebruikt en je krijgt een email met daarin jouw originele paswoord, dan kan je er zeker van zijn dat de service jouw paswoord op deze manier bewaard. In principe zou een service NOOIT jouw wachtwoord moeten kunnen zien. We gaan zelfs zien dat **jouw wachtwoord nooit je computer mag verlaten**, laat staan dat deze beschikbaar is in een plaintext database.

Paswoord hashing

Door een paswoord te hashen kunnen we de paswoorden al iets veiliger bewaarden. Door een secure hash van een paswoord te genereren creëren we een stuk tekst die niet terug naar het originele paswoord kan omgezet worden. In praktijk zal ieder paswoord een andere hash creëren uiteraard kunnen er toch 2 zaken zich voordoen:

1. Twee totaal verschillende paswoorden genereren dezelfde hash, een zogenaamde *collision*.
2. Twee mensen kiezen hetzelfde paswoord en zullen dus ook dezelfde hash genereren.

Een gebruiker die zich wenst aan te melden bij een systeem dat met paswoord hashes werkt zal nu op zijn lokale systeem z’n hash moeten genereren en dit over het netwerk doorsturen. De service zal deze ontvangen hash vergelijken met de waarde die in de database staan, en indien deze gelijk is dan wordt veronderstelt dat de gebruiker het juiste paswoord kende.

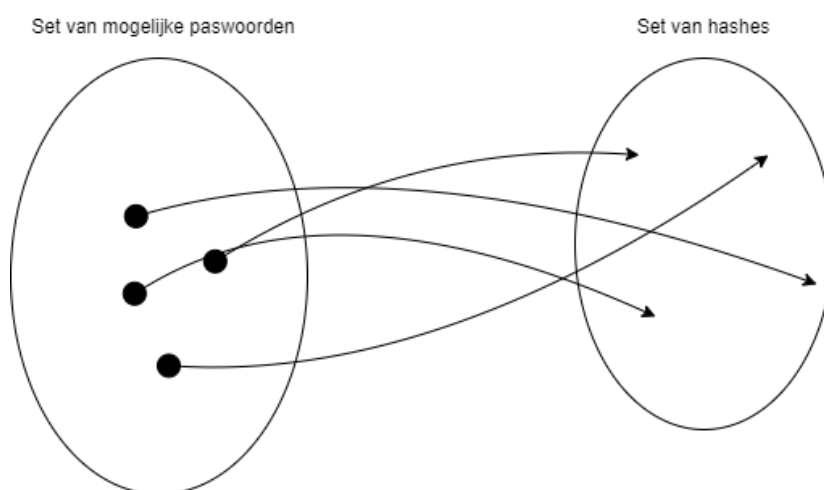
We versturen dus niet meer het paswoord over het netwerk, maar we zijn nu wel vatbaar voor een **pass-the-hash** aanval. Het volstaat om een geldige combinatie van gebruikersnaam en has te capteren en deze vervolgens te gebruiken om ergens in te loggen. De aanvaller heeft hierbij geen kennis nodig van het originele paswoord.

Rainbow table attack

Als de database door aanvallers gestolen wordt dan zitten we ook een tikkeltje veiliger als voorheen (de pass-the-hash aanval zal uiteraard nu zeker werken) indien de aanvaller de paswoorden van gebruiker nodig heeft (om bijvoorbeeld vervolgens op een ander systeem te gebruiken). De aanvaller zal een brute-force of dictionary attack moeten uitvoeren om te ontdekken welk paswoord resulteert in welke hash. Dit kan een erg tijdrovend proces zijn want enkel veelgebruikte hashing algoritmen (onder andere *script* en *bcrypt*) zijn *by design* zodanig geschreven dat deze erg traag werken. Dit zorgt ervoor dat de tijd om 1 hash te genereren geen voelbaar verschil geeft, maar wanneer een aanvaller er duizenden

per seconden wil kunnen testen, dan zal het algoritme als een stevige **timebottleneck** optreden. De aanvaller zou dan in de plaats vooraf alle hashen kunnen *precomputen* als alternatief. Dit heeft dan weer voor gevolg dat zo'n lijst gigantisch groot is en er dus een **memorybottleneck** optreedt.

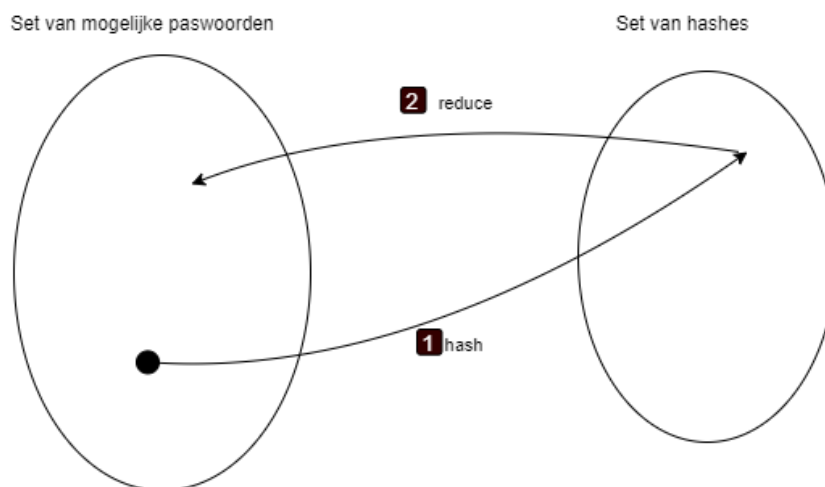
De aanvaller zit dus met het dilemma tussen hashen berekenen ter plekke, wat erg traag zal gaan, oftewel alle mogelijke hashen op voorhand berekenen, wat veel geheugenplek vereist. Via een **rainbow table attack** krijgt de aanvaller echter een handig instrument in handen dat een soort compromis tussen beide bottlenecks aanbiedt. Een rainbow table is een tabel van precomputed hashen, maar waarvan we ze niet allemaal moeten bewaren om toch een grotere set dan die dat in de tabel bewaard worden, te hebben.



Figuur 0.1: Ieder paswoord mapt naar exact 1 hash

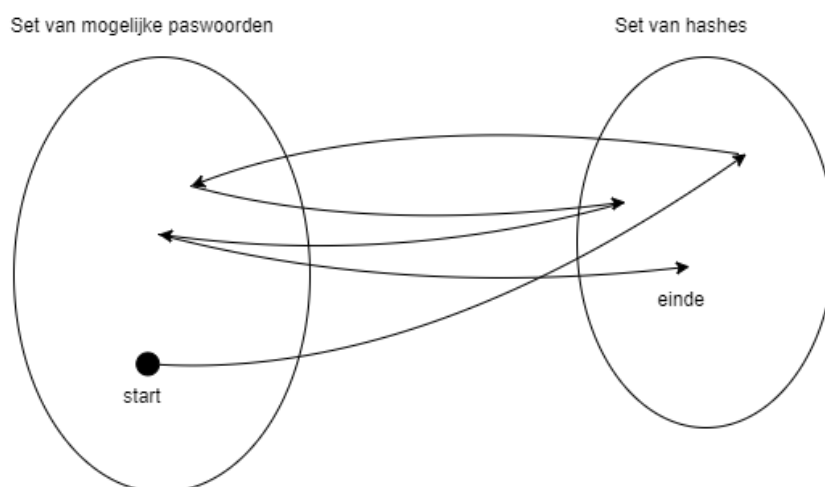
Een rainbow table stel je als volgt op:

- Je kiest een startpunt, zijnde 1 van de mogelijk paswoorden uit de set van paswoorden waarvoor je een rainbow table wil opstellen (zie figuur hier voor)
- Je genereert de hash van dit gekozen paswoord.
- Je pas nu op deze hash een *reduction* functie toe. Dit is een zelfgekozen mapping van de verkregen hash terug naar een paswoord uit de set van mogelijk paswoorden.



Figuur 0.2: Van de hash via de reductie functie terug naar een ander paswoord

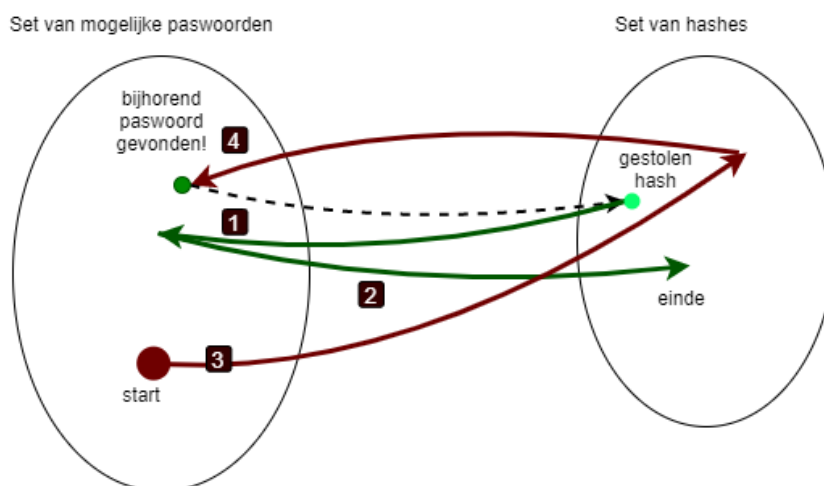
- Van dit nieuwe paswoord genereert je weer een hash en pas je weer de reduction functie toe.
- Die combinatie hash+reductie blijf je X aantal keer herhalen tot je een lekker lange lijst hebt.
- Finaal hou je nu van deze lijst enkel het startpunt bij (het gekozen paswoord uit de set) en de allerlaatste gegenereerde hash.



Figuur 0.3: Voorbeeld van een lijst opeenvolgende paswoorden en hun hashes

Wanneer de aanvaller nu van een gestolen hash terug het paswoord te pakken wil krijgen dan zal hij deze hash als startpunt gebruiken en hier telkens weer de combinatie reductie+hash op toepassen **totdat** een hash wordt gevonden die als eindpunt voor een van de lijsten werd ingesteld. De aanvaller zal vervolgens het startpunt van deze lijst nemen (een paswoord) en weer de combinatie hash+reductie toepassen hierop (als het ware terug een rainbow table generen) en op gegeven zal hij terug op de

gestolen hash uitkomen, als hij dan één stapje terug kijkt dan zal hij daar het paswoord zien die bij deze gestolen hash hoort.



Figuur 0.4: Ieder paswoord mapt naar exact 1 hash

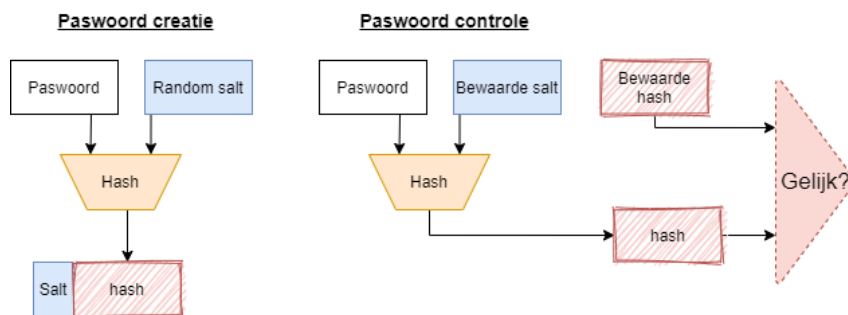


De uitleg, en vooral de manier van afbeeldingen, is gebaseerd op volgende uitstekende uitleg op het internet: [bron](#).

Salting

Om bestand te zijn tegen de rainbow attack dienen we de set van mogelijke paswoorden gevoelig te vergroten waardoor het niet meer realistisch is om voor die set rainbow tables te genereren. We kunnen helaas niet verwachten van de eindgebruiker dat zij met véél langere, meer willekeurige, paswoorden op de proppen komen en zullen dus een 'oude' truc moeten gebruiken die we ook al bij Wifi hebben gezien. Bij wifi hanteerden we een initialisatie vector (IV) om eigenlijk de WEP-sleutel met 24 bits te verlengen zodat zelfs bij dezelfde sleutel, iedere IV eigenlijk zorgt voor een unieke seed.

Wel nu, dit concept kan je ook toepassen bij wachtwoorden en heet **salting**. Een salt is een extra stuk dat je toevoegt aan het paswoord **voor je de hash** berekent. Dit extra stukje is een willekeurig getal dat je uiteraard zal mee moeten opslaan in de database. Wanneer twee gebruikers hetzelfde paswoorden zouden hebben, dan zouden ze , dankzij hun unieke salt, toch beide totaal verschillende hashes genereren. Niet alleen dat, maar de salt zorgt er dus ook voor dat de set van mogelijk paswoorden véél groter wordt.



In de database bewaren we dus nu volgende informatie:

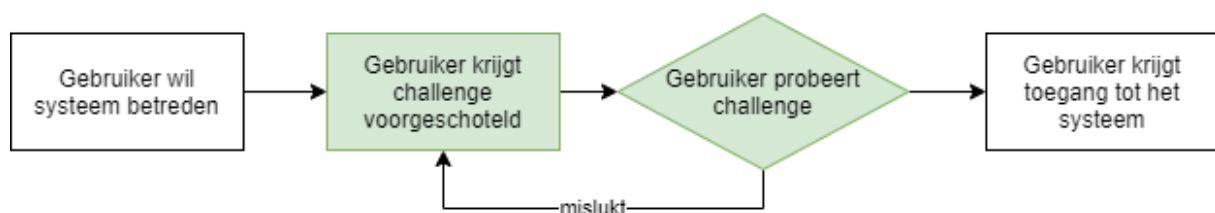
- gebruikersnaam.
- gebruikte salt (minimum 32 bits).
- hash van het paswoord en de salt samen.



Merk op dat ook nu we nog steeds niet beschermd zijn tegen pash-the-hash aanvallen.

CRAM en SCRAM

Om iemand te authenticeren spraken we totnogtoe enkel over een username/paswoord systeem. Echter, er zijn vele andere manieren om iemand te authenticeren. We spreken over "**challenge-response authentication (SCRAM)**" wanneer de gebruiker een vraag gesteld krijgt (de *challenge*) en hij hierop een geldig antwoord (de *response*) moet geven voor hij wordt toegelaten. Authenticeren met een paswoord is dus een vorm van CRAM. Er zijn er echter nog vele andere denk maar aan de gehekkelde CAPTCHAs -de ambetante vraag om te bewijzen dat je geen robot bent door alle boten in een afbeeldingen aan te duiden- of inloggen met behulp van je iris-scan.



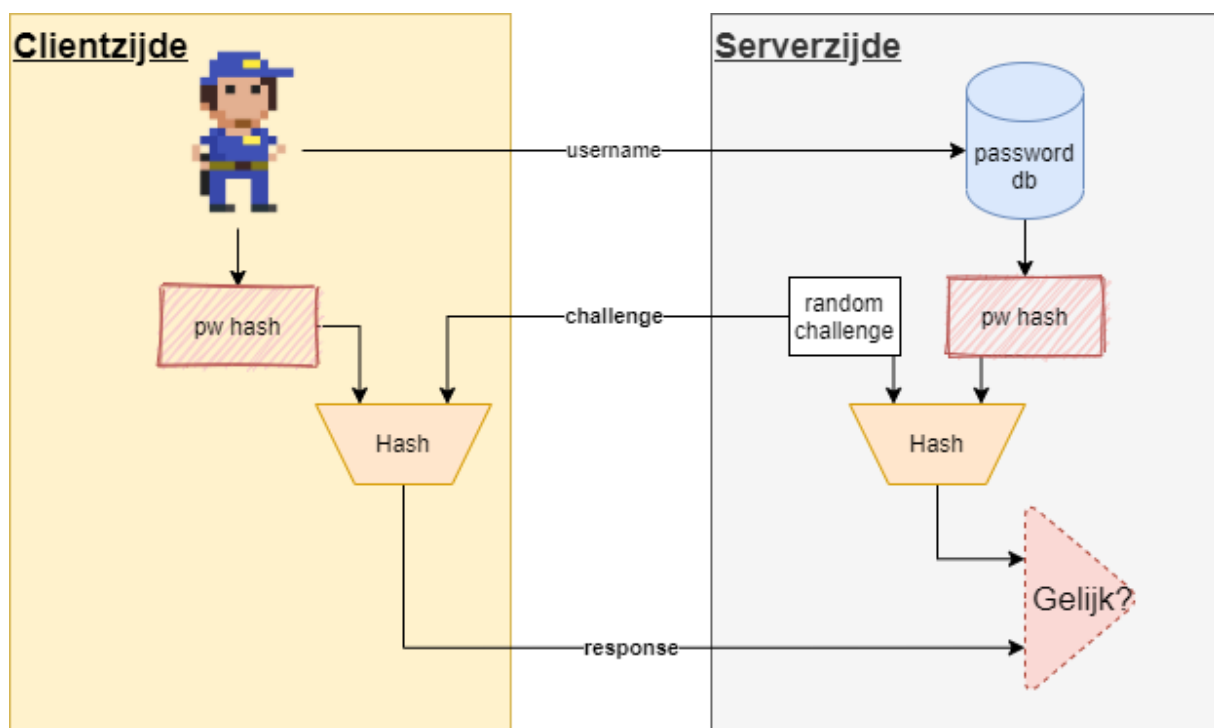
Om het probleem van *pash-the-hash* op te lossen kan je gebruiken maken van een **SCRAM**, een *salted challenge response authentication mechanism*. We bespreken een vereenvoudigde versie (een echte SCRAM voorziet ook *mutual authentication*) waarbij we hoofdzakelijk willen uitleggen waarom een SCRAM systeem veiliger is dan een klassieke salted paswoord logging van daarnet. Met dit systeem zorgen we ervoor dat

1. de salted hash van de gebruiker NOOIT moet verzonden worden.

2. geen replay aanval m.b.v. pass-the-hash mogelijk is.

Het mechanisme werkt als volgt:

1. De gebruiker stuurt z'n username met de vraag om in te loggen.
2. De server genereert een random challenge en stuurt deze terug.
3. Server en client creëren nu een hash van deze challenge met de hash van het paswoord (de server heeft dit bewaard, de client genereert de hash door z'n paswoord in te voeren)
4. De client stuurt deze hash, de response, terug naar de server.
5. De server vergelijkt of zijn gegenereerde response hash dezelfde is als die van de gebruiker.

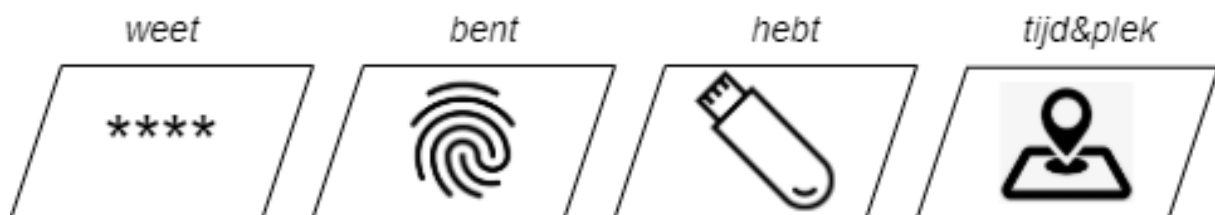


Merk op dat we ook hier nog steeds met een salted paswoord kunnen werken. Het enige dat dan verandert is dat de server naast de challenge, ook de te gebruiken salt doorstuurt die reeds in de database bewaard werd samen met de salted hash van de gebruiker.

Multifactor authentication

We hebben enkel nog maar over paswoorden gesproken in dit hoofdstuk, maar uiteraard zijn er ook andere zaken die je kan gebruiken om je te identificeren. Er zijn verschillende **factoren** die kunnen gebruikt worden om te controleren of een persoon wel degelijk toegang mag krijgen tot een systeem:

- iets wat je **weet**: je paswoord, je pincode, je rijksregisternummer, etc.
- iets wat je **bent**: een eigenschap die uniek is per persoon en onder de noemer "biometrics" valt, zoals je vingerafdruk, irisscan, etc.
- iets wat je **hebt**: een stuk hardware zoals een smartphone, USBkey, etc.
- **Waar** of **wanneer** je bent: je IP-adres, het moment van de dag dat je probeert in te loggen.



Figuur 0.5: Multifactor authenticatie factoren

We zien meer en meer systemen verschijnen die aan zogenaamde *multifactor authentication* (MFA) doen waarbij het systeem minstens 2 factoren (2FA*) wil controleren voor het je toegang tot het systeem geeft. Hoe meer verschillende factoren er worden gebruikt bij de authenticatie hoe veiliger het systeem is, maar ook hoe minder gebruiksvriendelijk het wordt. Het blijft dus een afweging tussen die 2 eigenschappen om in te schatten wat de ideale hoeveelheid veiligheid en gebruiksvriendelijkheid je wenst te hebben.

Systemen die MFA aanbieden doen dit vaak op een gecontroleerde manier: afhankelijk van de gebeurtenissen zal het systeem beslissen of meerdere factoren moeten getest worden of niet. Als je bijvoorbeeld woont in België, maar Google zit plots dat iemand met jouw paswoord probeert in te loggen vanuit een IP-adres op de Azoren, dan zal Google beslissen dat "iets wat je weet" (het paswoord) niet genoeg controle is en extra informatie vragen.

Iets wat je weet: paswoorden

Deze factor hebben reeds uitvoerig behandeld doorheen deze cursus. Het grote probleem met dingen weten is dat

- Je ze kan vergeten en daardoor niet meer kan inloggen.
- Anderen die informatie kunnen te weten komen en dus zich als jou voordoen.

Kortom, alhoewel deze factor vaak vanuit technologisch standpunt het eenvoudigst te implementeren is, is dat ook de minst veilige.

Iets wat je bent: biometrics

“We zijn allemaal uniek”. Iedere mens heeft een hele hoop eigenschappen die uniek zijn per persoon. Zelfs als bepaalde van onze eigenschappen gelijkaardig zijn met andere personen dan zal zeker een combinatie van 2 of meerdere eigenschappen dat niet zijn. Door dus menselijke eigenschappen van je te gebruiken als authenticatie hebben we een factor gevonden die zeer moeilijk na te bootsen valt: op voorwaarde dat je een echt unieke eigenschap kiest én deze op deze juiste manier meet.

Enkel veel gebruikte biometrieën als authenticatievorm zijn:

- vingerafdruk
- iris
- stem
- gezicht (vaak met behulp van “stereo camera” voor 3D beeld)

Maar ook andere metrieën kunnen erg interessant zijn zoals de manier waarop je je paswoord invoert, de manier waarop je wandelt (*gait*) etc.

Om een biometrie in de paswoord database te bewaren hebben we een manier nodig om deze te digitaliseren op een zodanige manier dat de unieke aspecten ervan bewaard worden. Voorts moet er rekening mee gehouden worden dat het “registreren” van een biometrische eigenschap nooit 100% accuraat kan. Denk maar aan een tijdelijk krasje op je vinger, je baard die anders geschoren is, etc. De zogenaamde *feature points* van een biometrische eigenschap worden in de database bewaard: dit zijn de unieke waarden waarvan geweten is dat deze per persoon anders zijn. We gaan deze niet per biometrische eigenschap bespreken, het volstaat te begrijpen dat in de gebruikersdatabase meestal een korte sequentie van getallen (of letters, denk maar aan een DNA-sample) wordt bewaard die als het ware jouw unieke paswoord voorstel voor die specifieke biometrische eigenschap van je. Enkel wanneer je bij het opnieuw inloggen (quasi) dezelfde *feature points* genereert bij de registratie zal deze factor aanvaardt worden als correct.



Biometrische eigenschappen kunnen niet alleen dienst doen als een extra factor bij het authenticeren, ze zijn uiteraard ook erg handig voor identificatie. In principe kan iemand nog steeds de gebruikersnaam van een ander persoon gebruiken (*impersonation*). Als de biometrische eigenschappen als identificatie dienen kunnen aanvallers dat niet meer doen: ze kunnen onmogelijk aan het systeem zeggen “*ik ben persoon x*” terwijl de vingerafdrukscanner duidelijk een vingerafdruk registreert van *persoon y*.

Iets wat je hebt: hardware

Een fysiek object, zeker als het complex is, kan moeilijk nageemaakt worden en is dus een ideale factor. De elektronica van de 21e eeuw behoort tot de meest complexe dingen ooit die de mensheid heeft kunnen vervaardigen. Het is dan ook logisch dat we deze elektronica gebruiken als extra authenticatiefactor.

Er zijn twee grote families van hardware-gebaseerde authenticatie-vormen:

- Een smartphone, met daarop een *authenticator* app.
- Een USB sleutel

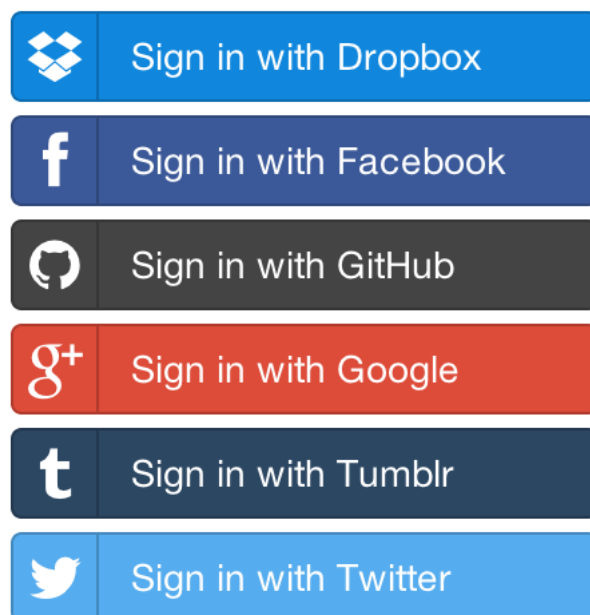
Een nadeel van deze groep is dat het om een fysiek object gaat dat je kan verliezen of dat stuk kan gaan.

Federation en single-sign on (SSO)

Bij cryptografie wordt het ten stelligste afgeraden om zomaar op de *wilde boef* een eigen crypto-algoritme te ontwikkelen. De kans dat je fouten met verstekkende gevolgen maakt is te groot. Ook bij het omgaan van login-data van gebruikers en hou je ze authenticiteit is het aangeraden om even te bezinnen voor je er zelf aan begint. Dankzij het concept **federation** hoeft je niet wakker te liggen hoe je je gebruikersspaswoorden gaat opslaan: gebruikers kunnen inloggen gebruik makend van hun bestaande Google, Facebook en andere accounts. Via federatie zal de gebruiker op jouw site (of app), de *service provider*, kunnen inloggen waarbij een *thirdparty* -die jij en je gebruiker vertrouwt- voor de eigenlijke authenticatie zorgt (de *identity provider*), gebruik makend van zogenaamde *single-sign on* (sso) authenticatie.



Federation via sso is een onderdeel van *federated identity management*, een grote groep technologieën en concepten die ervoor zorgen dat de identiteit van een gebruiker over meerdere, onafhankelijke systemen wordt bewaard en gebruikt.



Figuur 0.6: Enkele van de vele typische SSO knoppen die je geregeld zal tegenkomen.



OAuth (*open authorization*) is een gestandaardiseerde manier om aan authenticatie te doen.



Je zal de termen **delegation** en **federation** soms door elkaar zien tegenkomen wanneer je meer informatie over SSO opzoekt. Samengevat gaan we bij *delegation* een gebruiker verplichten in te loggen met een bepaalde third-party die dit ondersteunt (bv inloggen met je Facebook account). Bij *federation* gaat het breder: je website zal éénder welke third-party account aanvaarden, zolang deze maar compatibel is met het authenticatie systeem van je website (een voorbeeld hiervan is OpenID).

Dankwoord

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)

Bronnen

- 802.11 Wireless Network (first edition), Gast, O'Reilly, ISBN: 0-596-0018-3
- Meerdere boeken van William Stalling (Computer Security, Cybersecurity, etc.)

